HIS · GRUPO 3 · METODOLOGÍA Y PROGRAMACIÓN

Informe por módulo: qué hacer y con qué código

Una sección por módulo, en el orden en que conviene construirlos. Cada una trae el estado actual, los ajustes al modelo, los pasos con el código completo, los endpoints, los tests y una lista de "terminado cuando".

Backend NestJS + TypeORM | **Base** PostgreSQL | **Frontend** Vue 3 + Pinia | **Módulos** 6 + frontend

Cómo usar este informe

Cada módulo tiene su sección con el mismo orden: qué hay hoy en el repo, pasos de implementación con el código completo, endpoints, tests y una lista de "terminado". El código está escrito para el proyecto tal como está hoy: NestJS + TypeORM + PostgreSQL, capas `domain / application / infrastructure / presentation`, entidades y módulos que ya existen.

Tres cosas para leer antes de empezar:

- **El código no se ejecutó.** Está escrito contra los archivos reales del repo, pero hay que compilarlo y probarlo. Si algo no compila, se corrige en el momento: es parte del trabajo.
- **Los controllers `ping` no se tocan.** Cada módulo agrega controllers nuevos al lado de los `ping`. Así los tests existentes (`npm test` y `npm run test:e2e`) siguen pasando sin cambios.
- **La base compartida ya está hecha.** Las excepciones, los guards, los eventos y los cambios a las entidades ya están en el repo (sección "Lo que ya trae el repo"). No hay que escribirlos de nuevo: solo usarlos.

Orden de trabajo y dependencias

Los módulos se llaman entre sí a través de lo que cada uno exporta. Por eso el orden importa:

FASE	MÓDULO	DEPENDE DE	PATRÓN QUE SE APRENDE
1	IAM	Base compartida	JWT, guards, puertos y adaptadores
2	Admisiones	IAM	Locking optimista, State, cola de prioridad
3	Camas e inventario	Admisiones	Reutiliza State y concurrencia; stock atómico
4a	Historia Clínica	Admisiones	CRUD con reglas de integridad
4b	Atención y Enfermería	Admisiones, Camas	Builder, umbrales, eventos
5	Facturación	Admisiones, eventos de Atención	Strategy, listener de eventos

FASE	MÓDULO	DEPENDE DE	PATRÓN QUE SE APRENDE
6	Frontend	Todos	Pinia, router con roles

Admisiones va primero porque concentra más requisitos técnicos. Lo que se aprende ahí se copia casi tal cual en Camas.

Al clonar el repositorio

Esto lo hace **cada integrante**, una vez, en su máquina. Al terminar tiene el backend corriendo, las 21 tablas creadas en su PostgreSQL local y los tests en verde. Recién ahí empieza con su módulo.

1. Programas instalados

Node.js LTS (20 o 22), Git, PostgreSQL 17 y DataGrip (o pgAdmin). El paso a paso está en la *Guía de Instalación HIS*.

2. Clonar y crear la base

```
git clone <url-del-repositorio>
cd Tp-final-metologia-y-programacion---grupo-3
```

En DataGrip, sobre el servidor local, abrir una consola y ejecutar (con las comillas, para que respete las mayúsculas):

```
CREATE DATABASE "HospitalHIS";
```

3. Backend: dependencias y variables de entorno

```
cd backend
npm install
copy .env.example .env
```

(En Mac o Linux: `cp .env.example .env`.) Abrir `backend/.env` y completar:

```
DB_PASSWORD=la-password-que-elegiste-al-instalar-postgres
JWT_SECRET=una-frase-larga-cualquiera
ADMIN_EMAIL=admin@hospital.test
ADMIN_PASSWORD=cambiar-esta-clave-1
```

`DB_USERNAME`, `DB_HOST`, `DB_PORT` y `DB_NAME` ya vienen con el valor correcto. El `.env` **no se sube a git**: cada uno tiene el suyo.

`npm install` ya trae todo lo que necesita la base compartida (`bcryptjs`, `@nestjs/event-emitter`, `@types/passport-jwt`). Si por algún motivo falla alguno, instalarlo a mano:

```
npm i bcryptjs @nestjs/event-emitter
npm i -D @types/passport-jwt
```

4. Frontend

```
cd ../frontend
npm install
```

5. Arrancar y verificar

```
cd ../backend
npm run start:dev
```

El backend queda en `http://localhost:3000` y Swagger en `http://localhost:3000/api`. Al arrancar por primera vez, TypeORM crea las tablas solo (`synchronize: true`, temporal mientras el

modelo se estabiliza). En DataGrip: `HospitalHIS` → schemas → `public` → `tables`. Tienen que aparecer 21 tablas.

Si no ves ninguna, en el selector de esquemas ("No schemas selected" → `...`) marcar `public`.

Después, en otra terminal y dentro de `backend`:

```
npm test
npm run test:e2e
```

Los dos tienen que terminar sin fallas. Si algo falla *antes* de que hayas tocado código, avisar al grupo: es un problema de la base y no de tu módulo.

6. Trabajar en tu módulo

Cada integrante trabaja en su propia rama y la integra con un Pull Request:

```
git checkout -b modulo/admisiones     # el nombre de tu módulo
# ... trabajar, commitear ...
git pull origin main                  # traer lo que subieron los demás, seguido
git push origin modulo/admisiones     # y abrir el Pull Request en GitHub
```

Reglas para no pisarse: cada persona toca solo la carpeta de su módulo (más su archivo de módulo y, si hace falta, un agregado a `shared/`); lo que otro módulo necesita de vos se expone con `exports` en tu módulo, nunca importando sus archivos internos.

Lo que ya trae el repo (base compartida)

Esto ya está escrito, probado con tests unitarios y subido. Está acá para que se sepa qué hay y cómo usarlo.

Qué hay en `shared/`

ARCHIVO	PARA QUÉ SIRVE
`exceptions/domain.exception.ts`	Excepciones de negocio: `DomainException` (422), `ConflictoDeConcurrenciaException` (409), `RecursoNoEncontradoException` (404), `TransicionInvalidaException` (422), `CredencialesInvalidasException` (401)
`filters/domain-exception.filter.ts`	Traduce cada excepción a su código HTTP. Los casos de uso lanzan excepciones de dominio y nunca conocen HTTP
`database/errores-pg.ts`	`esViolacionDeUnicidad(error)`: detecta el error `23505` de PostgreSQL (choque de índice único)
`guards/jwt-auth.guard.ts`	`JwtAuthGuard`: exige un token válido. La estrategia `jwt` la registra el módulo IAM (fase 1)
`guards/roles.guard.ts` y `decorators/roles.decorator.ts`	`RolesGuard` y `@Roles(...)`: restringen un endpoint por rol
`decorators/usuario-actual.decorator.ts`	`@UsuarioActual()`: entrega `{ id, email, rol }` del token dentro de un handler
`events/`	`IEventPublisher` + `EVENT_PUBLISHER` (puerto), `EventosModule` (global, publica con `@nestjs/event-emitter`) y las constantes y payloads en `eventos.ts`

Además `main.ts` ya tiene `.addBearerAuth()`, así que Swagger muestra el botón **Authorize** para pegar el token.

Cómo se protege un endpoint

```
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(RolUsuario.RECEPCIONISTA, RolUsuario.ADMIN)
@Post()
reservar(...) { ... }
```

El orden importa: `JwtAuthGuard` primero (llena `request.user`), `RolesGuard` después (lo lee).

Cómo se publica un evento

El caso de uso recibe el puerto por constructor y publica; no sabe quién escucha:

```
constructor(@Inject(EVENT_PUBLISHER) private readonly eventos: IEventPublisher) {}

this.eventos.publicar(EVENTOS.STOCK_BAJO_UMBRAL, payload);
```

Quien escucha (por ejemplo Facturación) lo hace con `@OnEvent(EVENTOS.MEDICACION_ADMINISTRADA)` en un provider de su módulo.

Patrón de test sin base de datos

Todos los casos de uso se testean con un repositorio en memoria que implementa la misma interfaz que el real. Es la prueba de que la inversión de dependencias funciona. Ejemplo del esquema (se repite en cada módulo):

```
class RepoEnMemoria implements IAlgoRepository {
  private readonly items = new Map<string, Algo>();
  async findById(id: string) { return this.items.get(id) ?? null; }
  async save(a: Algo) { this.items.set(a.id, a); return a; }
}

const useCase = new HacerAlgoUseCase(new RepoEnMemoria());   // sin Nest, sin Postgres
```

Cambios al modelo de datos ya aplicados

Las entidades ya tienen estas columnas. Se crean solas al arrancar el backend, así que no hay que hacer nada; solo saber que existen.

ENTIDAD	CAMBIO	PARA QUÉ
`Paciente`, `Medico`	`usuarioId` (uuid, opcional, único)	Saber qué médico o paciente es el usuario logueado
`Turno`	Estado inicial `programado` e índice único parcial `uq_turno_medico_horario_activo` (médico + horario, ignora cancelados)	Defensa real contra la doble reserva
`Insumo`	`precioUnitario` (numeric)	Facturación necesita cuánto cobrar por unidad
`IndicacionMedica`	`insumoId` y `cantidad` (opcionales)	Qué insumo se descuenta al cumplir una medicación
`Factura`	`montoCubierto` (numeric)	Lo que absorbe la cobertura (el `total` es lo que paga el paciente)

Para comprobar el índice del turno, en DataGrip:

```
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'turnos';
```

Tiene que aparecer `uq_turno_medico_horario_activo` con la condición `WHERE estado <> 'cancelado'`. Si no está (por ejemplo porque la tabla ya tenía filas repetidas), se crea a mano:

```
CREATE UNIQUE INDEX uq_turno_medico_horario_activo
  ON turnos ("medicoId", "fechaHora")
  WHERE "estado" <> 'cancelado';
```

Dos advertencias que valen para todo el proyecto

1. La concurrencia optimista no es automática. TypeORM, con `@VersionColumn()`, **solo incrementa** la columna en cada `save()`; no compara ni lanza error si dos peticiones chocan. Para que la segunda petición pierda, hay que hacer la comparación a mano con un `UPDATE` condicional:

```
const r = await this.repo
  .createQueryBuilder()
  .update(Turno)
  .set({ estado: nuevoEstado, version: () => 'version + 1' })
  .where('id = :id AND version = :versionLeida', { id, versionLeida })
  .execute();

return r.affected === 1;   // false => alguien modificó la fila antes: responder 409
```

Todo el código de este informe usa ese patrón. El informe de arquitectura (sección 06) ya fue corregido para decir lo mismo.

2. Las entidades llevan decoradores de TypeORM dentro de `domain/`. Para no duplicar cada tabla en dos clases (entidad de dominio y entidad de persistencia), el proyecto usa la misma clase. Lo que sí se mantiene limpio es la lógica: las máquinas de estado, los Builders, las estrategias y las reglas de umbrales son TypeScript puro, sin imports de Nest ni de TypeORM, y los casos de uso solo dependen de interfaces.

IAM: identidad y accesos

Objetivo. Que cualquiera pueda iniciar sesión, que el backend reconozca quién es y con qué rol, y que el resto de los módulos puedan restringir endpoints por rol. Todo lo demás depende de esto.

Roles: `admin`, `medico`, `enfermero`, `recepcionista`, `paciente` (enum `RolUsuario`, ya existe).

Qué hay hoy

YA EXISTE	FALTA
Entidad `Usuario` (con `passwordHash`, `rol`, `activo`, `version`)	Hash de contraseñas
Interfaz `IUsuarioRepository` + token + implementación TypeORM	Casos de uso `RegistrarUsuario` y `Login`
`RolesGuard` y `@Roles()` en `shared/`	Estrategia JWT (el `JwtAuthGuard` ya está en `shared/`)
Endpoint `GET /iam/ping`	`POST /iam/login`, `POST /iam/registro`, `POST /iam/usuarios`, `GET /iam/yo`
	Un primer usuario administrador

Estructura final

```
iam/
├── domain/
│   ├── entities/usuario.entity.ts             (existe)
│   ├── enums/rol.enum.ts                      (existe)
│   ├── repositories/usuario-repository.interface.ts   (existe)
│   └── services/
│       ├── password-hasher.interface.ts       NUEVO
│       └── token-service.interface.ts         NUEVO
├── application/
│   ├── registrar-usuario.use-case.ts          NUEVO
│   └── login.use-case.ts                      NUEVO
├── infrastructure/
│   ├── repositories/typeorm-usuario.repository.ts   (existe)
│   └── security/
│       ├── bcrypt-password-hasher.ts          NUEVO
│       ├── jwt-token.service.ts               NUEVO
│       ├── jwt.strategy.ts                    NUEVO
│       └── sembrar-admin.service.ts           NUEVO
├── presentation/
│   ├── iam.controller.ts                      (existe: ping)
│   ├── auth.controller.ts                     NUEVO
│   └── dto/                                   NUEVO
└── iam.module.ts                              (se modifica)
```

Paso 1. Puertos (interfaces del dominio)

`iam/domain/services/password-hasher.interface.ts`:

```ts
export interface IPasswordHasher {
  hashear(plano: string): Promise<string>;
  comparar(plano: string, hashGuardado: string): Promise<boolean>;
}

export const PASSWORD_HASHER = Symbol('PASSWORD_HASHER');
```

`iam/domain/services/token-service.interface.ts`:

```ts
import { RolUsuario } from '../enums/rol.enum';

export interface PayloadToken {
  sub: string;      // id del usuario
```

```
  email: string;
  rol: RolUsuario;
}

export interface ITokenService {
  firmar(payload: PayloadToken): Promise<string>;
}

export const TOKEN_SERVICE = Symbol('TOKEN_SERVICE');
```

Paso 2. Adaptadores de infraestructura

`iam/infrastructure/security/bcrypt-password-hasher.ts`:

```
import { Injectable } from '@nestjs/common';
import { compare, hash } from 'bcryptjs';
import { IPasswordHasher } from '../../domain/services/password-hasher.interface';

@Injectable()
export class BcryptPasswordHasher implements IPasswordHasher {
  private static readonly RONDAS = 10;

  hashear(plano: string): Promise<string> {
    return hash(plano, BcryptPasswordHasher.RONDAS);
  }

  comparar(plano: string, hashGuardado: string): Promise<boolean> {
    return compare(plano, hashGuardado);
  }
}
```

`iam/infrastructure/security/jwt-token.service.ts`:

```
import { Injectable } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { ITokenService, PayloadToken } from '../../domain/services/token-service.interface';

@Injectable()
export class JwtTokenService implements ITokenService {
  constructor(private readonly jwt: JwtService) {}

  firmar(payload: PayloadToken): Promise<string> {
    return this.jwt.signAsync(payload);
  }
}
```

`iam/infrastructure/security/jwt.strategy.ts`. Es lo que llena `request.user` en cada request autenticado:

```
import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { UsuarioAutenticado } from '../../../shared/decorators/usuario-actual.decorator';
import { PayloadToken } from '../../domain/services/token-service.interface';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(config: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: config.getOrThrow<string>('JWT_SECRET'),
    });
  }

  /** Lo que devuelve acá queda disponible como request.user. */
  validate(payload: PayloadToken): UsuarioAutenticado {
    return { id: payload.sub, email: payload.email, rol: payload.rol };
  }
}
```

Paso 3. Casos de uso

`iam/application/registrar-usuario.use-case.ts`:

```
import { Inject, Injectable } from '@nestjs/common';
import { DomainException } from '../../shared/exceptions/domain.exception';
import { Usuario } from '../domain/entities/usuario.entity';
```

```ts
import { RolUsuario } from '../domain/enums/rol.enum';
import {
  IUsuarioRepository,
  USUARIO_REPOSITORY,
} from '../domain/repositories/usuario-repository.interface';
import { IPasswordHasher, PASSWORD_HASHER } from '../domain/services/password-hasher.interface';

export interface RegistrarUsuarioCommand {
  email: string;
  password: string;
  nombre: string;
  rol: RolUsuario;
}

@Injectable()
export class RegistrarUsuarioUseCase {
  constructor(
    @Inject(USUARIO_REPOSITORY) private readonly usuarios: IUsuarioRepository,
    @Inject(PASSWORD_HASHER) private readonly hasher: IPasswordHasher,
  ) {}

  async ejecutar(cmd: RegistrarUsuarioCommand): Promise<Usuario> {
    const email = cmd.email.trim().toLowerCase();
    if (await this.usuarios.findByEmail(email)) {
      throw new DomainException('Ya existe un usuario con ese email');
    }

    const usuario = new Usuario();
    usuario.email = email;
    usuario.passwordHash = await this.hasher.hashear(cmd.password);
    usuario.nombre = cmd.nombre.trim();
    usuario.rol = cmd.rol;
    usuario.activo = true;

    return this.usuarios.save(usuario);
  }
}
```

`iam/application/login.use-case.ts`:

```ts
import { Inject, Injectable } from '@nestjs/common';
import { CredencialesInvalidasException } from '../../shared/exceptions/domain.exception';
import { RolUsuario } from '../domain/enums/rol.enum';
import {
  IUsuarioRepository,
  USUARIO_REPOSITORY,
} from '../domain/repositories/usuario-repository.interface';
import { IPasswordHasher, PASSWORD_HASHER } from '../domain/services/password-hasher.interface';
import { ITokenService, TOKEN_SERVICE } from '../domain/services/token-service.interface';

export interface LoginCommand {
  email: string;
  password: string;
}

export interface LoginResultado {
  accessToken: string;
  usuario: { id: string; nombre: string; rol: RolUsuario };
}

@Injectable()
export class LoginUseCase {
  constructor(
    @Inject(USUARIO_REPOSITORY) private readonly usuarios: IUsuarioRepository,
    @Inject(PASSWORD_HASHER) private readonly hasher: IPasswordHasher,
    @Inject(TOKEN_SERVICE) private readonly tokens: ITokenService,
  ) {}

  async ejecutar(cmd: LoginCommand): Promise<LoginResultado> {
    const usuario = await this.usuarios.findByEmail(cmd.email.trim().toLowerCase());

    // Mismo error para "no existe", "inactivo" y "clave mala": no revelar cuál fue.
    if (!usuario || !usuario.activo) throw new CredencialesInvalidasException();
    if (!(await this.hasher.comparar(cmd.password, usuario.passwordHash))) {
      throw new CredencialesInvalidasException();
    }

    const accessToken = await this.tokens.firmar({
      sub: usuario.id,
      email: usuario.email,
      rol: usuario.rol,
    });

    return {
      accessToken,
      usuario: { id: usuario.id, nombre: usuario.nombre, rol: usuario.rol },
    };
  }
}
```

Paso 4. Primer administrador

Sin esto nadie puede crear usuarios con rol. Se crea solo al arrancar si `ADMIN_EMAIL` y `ADMIN_PASSWORD` están en el `.env` y el usuario no existe.

`iam/infrastructure/security/sembrar-admin.service.ts`:

```
import { Inject, Injectable, Logger, OnApplicationBootstrap } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { RegistrarUsuarioUseCase } from '../../application/registrar-usuario.use-case';
import { RolUsuario } from '../../domain/enums/rol.enum';
import {
  IUsuarioRepository,
  USUARIO_REPOSITORY,
} from '../../domain/repositories/usuario-repository.interface';

@Injectable()
export class SembrarAdminService implements OnApplicationBootstrap {
  private readonly logger = new Logger(SembrarAdminService.name);

  constructor(
    private readonly config: ConfigService,
    private readonly registrar: RegistrarUsuarioUseCase,
    @Inject(USUARIO_REPOSITORY) private readonly usuarios: IUsuarioRepository,
  ) {}

  async onApplicationBootstrap(): Promise<void> {
    const email = this.config.get<string>('ADMIN_EMAIL');
    const password = this.config.get<string>('ADMIN_PASSWORD');
    if (!email || !password) return;
    if (await this.usuarios.findByEmail(email.trim().toLowerCase())) return;

    await this.registrar.ejecutar({ email, password, nombre: 'Administrador', rol: RolUsuario.ADMIN });
    this.logger.log(`Usuario administrador creado: ${email}`);
  }
}
```

Paso 5. DTOs y controller

`iam/presentation/dto/login.dto.ts`:

```
import { ApiProperty } from '@nestjs/swagger';
import { IsEmail, IsString, MinLength } from 'class-validator';

export class LoginDto {
  @ApiProperty({ example: 'admin@hospital.test' })
  @IsEmail()
  email: string;

  @ApiProperty()
  @IsString()
  @MinLength(8)
  password: string;
}
```

`iam/presentation/dto/registro.dto.ts`:

```
import { ApiProperty } from '@nestjs/swagger';
import { IsEmail, IsEnum, IsNotEmpty, IsString, MinLength } from 'class-validator';
import { RolUsuario } from '../../domain/enums/rol.enum';

export class RegistroDto {
  @ApiProperty() @IsEmail() email: string;
  @ApiProperty() @IsString() @MinLength(8) password: string;
  @ApiProperty() @IsString() @IsNotEmpty() nombre: string;
}

/** Solo lo usa el admin: puede elegir cualquier rol. */
export class CrearUsuarioDto extends RegistroDto {
  @ApiProperty({ enum: RolUsuario })
  @IsEnum(RolUsuario)
  rol: RolUsuario;
}
```

`iam/presentation/auth.controller.ts`:

```
import { Body, Controller, Get, HttpCode, Post, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { Roles } from '../../shared/decorators/roles.decorator';
import {
```

```
  UsuarioActual,
  UsuarioAutenticado,
} from '../../shared/decorators/usuario-actual.decorator';
import { JwtAuthGuard } from '../../shared/guards/jwt-auth.guard';
import { RolesGuard } from '../../shared/guards/roles.guard';
import { LoginUseCase } from '../application/login.use-case';
import { RegistrarUsuarioUseCase } from '../application/registrar-usuario.use-case';
import { Usuario } from '../domain/entities/usuario.entity';
import { RolUsuario } from '../domain/enums/rol.enum';
import { LoginDto } from './dto/login.dto';
import { CrearUsuarioDto, RegistroDto } from './dto/registro.dto';

/** Nunca se devuelve la entidad completa: el hash no sale del backend. */
function aRespuesta(u: Usuario) {
  return { id: u.id, email: u.email, nombre: u.nombre, rol: u.rol, activo: u.activo };
}

@ApiTags('iam')
@Controller('iam')
export class AuthController {
  constructor(
    private readonly loginUseCase: LoginUseCase,
    private readonly registrarUseCase: RegistrarUsuarioUseCase,
  ) {}

  @Post('login')
  @HttpCode(200)
  login(@Body() dto: LoginDto) {
    return this.loginUseCase.ejecutar(dto);
  }

  /** Autoregistro público: siempre crea un paciente. */
  @Post('registro')
  async registro(@Body() dto: RegistroDto) {
    return aRespuesta(await this.registrarUseCase.ejecutar({ ...dto, rol: RolUsuario.PACIENTE }));
  }

  /** El personal (médicos, enfermeros, recepción) lo da de alta un admin. */
  @Post('usuarios')
  @ApiBearerAuth()
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(RolUsuario.ADMIN)
  async crearUsuario(@Body() dto: CrearUsuarioDto) {
    return aRespuesta(await this.registrarUseCase.ejecutar(dto));
  }

  @Get('yo')
  @ApiBearerAuth()
  @UseGuards(JwtAuthGuard)
  yo(@UsuarioActual() usuario: UsuarioAutenticado) {
    return usuario;
  }
}
```

Paso 6. Módulo

`iam/iam.module.ts` (reemplazar):

```
import { Module } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { JwtModule, JwtModuleOptions, JwtSignOptions } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { TypeOrmModule } from '@nestjs/typeorm';
import { LoginUseCase } from './application/login.use-case';
import { RegistrarUsuarioUseCase } from './application/registrar-usuario.use-case';
import { Usuario } from './domain/entities/usuario.entity';
import { USUARIO_REPOSITORY } from './domain/repositories/usuario-repository.interface';
import { PASSWORD_HASHER } from './domain/services/password-hasher.interface';
import { TOKEN_SERVICE } from './domain/services/token-service.interface';
import { TypeOrmUsuarioRepository } from './infrastructure/repositories/typeorm-usuario.repository';
import { BcryptPasswordHasher } from './infrastructure/security/bcrypt-password-hasher';
import { JwtStrategy } from './infrastructure/security/jwt.strategy';
import { JwtTokenService } from './infrastructure/security/jwt-token.service';
import { SembrarAdminService } from './infrastructure/security/sembrar-admin.service';
import { AuthController } from './presentation/auth.controller';
import { IamController } from './presentation/iam.controller';

@Module({
  imports: [
    TypeOrmModule.forFeature([Usuario]),
    PassportModule,
    JwtModule.registerAsync({
      inject: [ConfigService],
      useFactory: (config: ConfigService): JwtModuleOptions => ({
        secret: config.getOrThrow<string>('JWT_SECRET'),
        signOptions: {
          expiresIn: config.get<string>('JWT_EXPIRES_IN', '8h') as JwtSignOptions['expiresIn'],
        },
      }),
    }),
```

```
  ],
  controllers: [IamController, AuthController],
  providers: [
    { provide: USUARIO_REPOSITORY, useClass: TypeOrmUsuarioRepository },
    { provide: PASSWORD_HASHER, useClass: BcryptPasswordHasher },
    { provide: TOKEN_SERVICE, useClass: JwtTokenService },
    JwtStrategy,
    RegistrarUsuarioUseCase,
    LoginUseCase,
    SembrarAdminService,
  ],
  exports: [USUARIO_REPOSITORY],
})
export class IamModule {}
```

Endpoints

MÉTODO	RUTA	ACCESO	QUÉ HACE
POST	`/iam/login`	Público	Devuelve `accessToken` y datos del usuario
POST	`/iam/registro`	Público	Autoregistro de paciente
POST	`/iam/usuarios`	Admin	Alta de personal con rol
GET	`/iam/yo`	Cualquier logueado	Devuelve `{ id, email, rol }` del token
GET	`/iam/ping`	Público	Ya existe

Tests

`iam/application/login.use-case.spec.ts`:

```
import { CredencialesInvalidasException } from '../../shared/exceptions/domain.exception';
import { Usuario } from '../domain/entities/usuario.entity';
import { RolUsuario } from '../domain/enums/rol.enum';
import { IUsuarioRepository } from '../domain/repositories/usuario-repository.interface';
import { IPasswordHasher } from '../domain/services/password-hasher.interface';
import { ITokenService } from '../domain/services/token-service.interface';
import { LoginUseCase } from './login.use-case';

class UsuariosEnMemoria implements IUsuarioRepository {
  constructor(private readonly lista: Usuario[]) {}
  async findByEmail(email: string) { return this.lista.find((u) => u.email === email) ?? null; }
  async findById(id: string) { return this.lista.find((u) => u.id === id) ?? null; }
  async save(u: Usuario) { return u; }
}

const hasherFalso: IPasswordHasher = {
  hashear: async (p) => `hash:${p}`,
  comparar: async (p, h) => h === `hash:${p}`,
};

const tokensFalsos: ITokenService = { firmar: async (p) => `token-de-${p.email}` };

function usuario(overrides: Partial<Usuario> = {}): Usuario {
  return Object.assign(new Usuario(), {
    id: 'u1', email: 'dra@hospital.test', passwordHash: 'hash:clave-segura',
    nombre: 'Dra. López', rol: RolUsuario.MEDICO, activo: true, version: 1, ...overrides,
  });
}

describe('LoginUseCase', () => {
  const crear = (usuarios: Usuario[]) =>
    new LoginUseCase(new UsuariosEnMemoria(usuarios), hasherFalso, tokensFalsos);

  it('devuelve token y datos del usuario con credenciales correctas', async () => {
    const r = await crear([usuario()]).ejecutar({ email: 'DRA@hospital.test', password: 'clave-segura' });

    expect(r.accessToken).toBe('token-de-dra@hospital.test');
    expect(r.usuario).toEqual({ id: 'u1', nombre: 'Dra. López', rol: RolUsuario.MEDICO });
  });

  it('rechaza una contraseña incorrecta', async () => {
    await expect(crear([usuario()]).ejecutar({ email: 'dra@hospital.test', password: 'otra-clave' }))
      .rejects.toBeInstanceOf(CredencialesInvalidasException);
  });

  it('rechaza un email que no existe', async () => {
    await expect(crear([]).ejecutar({ email: 'nadie@hospital.test', password: 'clave-segura' }))
      .rejects.toBeInstanceOf(CredencialesInvalidasException);
```

```
  });

  it('rechaza a un usuario inactivo', async () => {
    await expect(crear([usuario({ activo: false })]).ejecutar({ email: 'dra@hospital.test', password: 'clave-segura' }))
      .rejects.toBeInstanceOf(CredencialesInvalidasException);
  });
});
```

Terminado cuando

- [ ] `POST /iam/login` con el admin del `.env` devuelve un token (probarlo desde Swagger).
- [ ] Con ese token, `GET /iam/yo` devuelve el usuario; sin token devuelve 401.
- [ ] Un usuario `paciente` que llama a `POST /iam/usuarios` recibe 403.
- [ ] La respuesta de ningún endpoint incluye `passwordHash`.
- [ ] `login.use-case.spec.ts` pasa y los tests anteriores siguen en verde.

Admisiones y Turnos

Objetivo. Registrar pacientes y médicos, reservar turnos sin que dos recepcionistas puedan ocupar el mismo horario, llevar cada turno por su ciclo de vida y ordenar la cola de espera por prioridad de triaje.

Patrones de este módulo: locking optimista (HU 1.1), State para el turno (Épica 2), cola de prioridad (Épica 3).

Qué hay hoy

YA EXISTE	FALTA
Entidades `Paciente`, `Medico`, `Turno`, `ColaEntreturno`	Interfaces de repositorio y su implementación TypeORM
`AdmisionesController` con `GET /admisiones/ping`	CRUD de pacientes y médicos
Columna `version` en `Turno`	Reserva con control de concurrencia
	Máquina de estados del turno
	Cola de triaje y entreturnos

Antes de empezar: IAM terminado. Las entidades ya traen `usuarioId` en `Paciente` y `Medico`, el estado inicial `programado` y el índice único del horario en `Turno` (ver "Lo que ya trae el repo"); no hay que modificarlas.

Estructura final

```
admisiones/
├── domain/
│   ├── entities/                      (existen)
│   ├── repositories/
│   │   ├── paciente-repository.interface.ts   NUEVO
│   │   ├── medico-repository.interface.ts     NUEVO
│   │   ├── turno-repository.interface.ts      NUEVO
│   │   └── cola-repository.interface.ts       NUEVO
│   ├── state/turno-estado.ts          NUEVO  (State)
│   ├── turno-workflow.ts              NUEVO
│   └── cola-prioridad.ts              NUEVO
├── application/                       NUEVO  (un archivo por caso de uso)
├── infrastructure/repositories/       NUEVO  (implementaciones TypeORM)
├── presentation/
│   ├── admisiones.controller.ts       (existe: ping)
│   ├── pacientes.controller.ts        NUEVO
│   ├── medicos.controller.ts          NUEVO
│   ├── turnos.controller.ts           NUEVO
│   ├── cola.controller.ts             NUEVO
│   └── dto/                           NUEVO
└── admisiones.module.ts               (se modifica)
```

Paso 1. Pacientes y médicos (CRUD base)

`domain/repositories/paciente-repository.interface.ts`:

```ts
import { Paciente } from '../entities/paciente.entity';

export interface IPacienteRepository {
  findById(id: string): Promise<Paciente | null>;
  findByDni(dni: string): Promise<Paciente | null>;
  buscar(texto: string): Promise<Paciente[]>;
  save(paciente: Paciente): Promise<Paciente>;
}

export const PACIENTE_REPOSITORY = Symbol('PACIENTE_REPOSITORY');
```

`domain/repositories/medico-repository.interface.ts`:

```
import { Medico } from '../entities/medico.entity';

export interface IMedicoRepository {
  findById(id: string): Promise<Medico | null>;
  findByUsuarioId(usuarioId: string): Promise<Medico | null>;
  listarActivos(): Promise<Medico[]>;
  save(medico: Medico): Promise<Medico>;
}

export const MEDICO_REPOSITORY = Symbol('MEDICO_REPOSITORY');
```

`infrastructure/repositories/typeorm-paciente.repository.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { ILike, Repository } from 'typeorm';
import { Paciente } from '../../domain/entities/paciente.entity';
import { IPacienteRepository } from '../../domain/repositories/paciente-repository.interface';

@Injectable()
export class TypeOrmPacienteRepository implements IPacienteRepository {
  constructor(@InjectRepository(Paciente) private readonly repo: Repository<Paciente>) {}

  findById(id: string) {
    return this.repo.findOneBy({ id });
  }

  findByDni(dni: string) {
    return this.repo.findOneBy({ dni });
  }

  buscar(texto: string) {
    return this.repo.find({
      where: [{ apellido: ILike(`%${texto}%`) }, { dni: ILike(`${texto}%`) }],
      order: { apellido: 'ASC' },
      take: 20,
    });
  }

  save(paciente: Paciente) {
    return this.repo.save(paciente);
  }
}
```

`infrastructure/repositories/typeorm-medico.repository.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Medico } from '../../domain/entities/medico.entity';
import { IMedicoRepository } from '../../domain/repositories/medico-repository.interface';

@Injectable()
export class TypeOrmMedicoRepository implements IMedicoRepository {
  constructor(@InjectRepository(Medico) private readonly repo: Repository<Medico>) {}

  findById(id: string) {
    return this.repo.findOneBy({ id });
  }

  findByUsuarioId(usuarioId: string) {
    return this.repo.findOneBy({ usuarioId });
  }

  listarActivos() {
    return this.repo.find({ where: { activo: true }, order: { apellido: 'ASC' } });
  }

  save(medico: Medico) {
    return this.repo.save(medico);
  }
}
```

`application/registrar-paciente.use-case.ts`:

```
import { Inject, Injectable } from '@nestjs/common';
import { DomainException } from '../../shared/exceptions/domain.exception';
import { Paciente } from '../domain/entities/paciente.entity';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../domain/repositories/paciente-repository.interface';
```

```ts
export interface RegistrarPacienteCommand {
  dni: string;
  nombre: string;
  apellido: string;
  fechaNacimiento: string;
  telefono?: string;
  email?: string;
  coberturaId?: string;
}

@Injectable()
export class RegistrarPacienteUseCase {
  constructor(@Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository) {}

  async ejecutar(cmd: RegistrarPacienteCommand): Promise<Paciente> {
    if (await this.pacientes.findByDni(cmd.dni)) {
      throw new DomainException(`Ya existe un paciente con DNI ${cmd.dni}`);
    }

    const paciente = new Paciente();
    paciente.dni = cmd.dni;
    paciente.nombre = cmd.nombre.trim();
    paciente.apellido = cmd.apellido.trim();
    paciente.fechaNacimiento = cmd.fechaNacimiento;
    paciente.telefono = cmd.telefono ?? null;
    paciente.email = cmd.email ?? null;
    paciente.coberturaId = cmd.coberturaId ?? null;
    paciente.usuarioId = null;

    return this.pacientes.save(paciente);
  }
}
```

`application/buscar-pacientes.use-case.ts`:

```ts
import { Inject, Injectable } from '@nestjs/common';
import { Paciente } from '../domain/entities/paciente.entity';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../domain/repositories/paciente-repository.interface';

@Injectable()
export class BuscarPacientesUseCase {
  constructor(@Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository) {}

  ejecutar(texto: string): Promise<Paciente[]> {
    return this.pacientes.buscar(texto.trim());
  }
}
```

`application/registrar-medico.use-case.ts` sigue el mismo molde que `RegistrarPacienteUseCase`: valida que la matrícula no exista, arma el `Medico` con `activo = true` y `usuarioId` (opcional, para vincularlo con su login) y lo guarda con `IMedicoRepository.save`. Para validar la matrícula, agregar `findByMatricula(matricula: string)` a la interfaz del médico.

`presentation/dto/registrar-paciente.dto.ts`:

```ts
import { ApiProperty, ApiPropertyOptional } from '@nestjs/swagger';
import { IsDateString, IsEmail, IsNotEmpty, IsOptional, IsString, IsUUID, Matches } from 'class-validator';

export class RegistrarPacienteDto {
  @ApiProperty({ example: '30123456' })
  @Matches(/^\d{7,8}$/, { message: 'El DNI debe tener 7 u 8 dígitos' })
  dni: string;

  @ApiProperty() @IsString() @IsNotEmpty() nombre: string;
  @ApiProperty() @IsString() @IsNotEmpty() apellido: string;

  @ApiProperty({ example: '1990-05-21' })
  @IsDateString()
  fechaNacimiento: string;

  @ApiPropertyOptional() @IsOptional() @IsString() telefono?: string;
  @ApiPropertyOptional() @IsOptional() @IsEmail() email?: string;
  @ApiPropertyOptional() @IsOptional() @IsUUID() coberturaId?: string;
}
```

`presentation/pacientes.controller.ts`:

```ts
import { Body, Controller, Get, Post, Query, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { Roles } from '../../shared/decorators/roles.decorator';
import { JwtAuthGuard } from '../../shared/guards/jwt-auth.guard';
```

```ts
import { RolesGuard } from '../../shared/guards/roles.guard';
import { RolUsuario } from '../../iam/domain/enums/rol.enum';
import { BuscarPacientesUseCase } from '../application/buscar-pacientes.use-case';
import { RegistrarPacienteUseCase } from '../application/registrar-paciente.use-case';
import { RegistrarPacienteDto } from './dto/registrar-paciente.dto';

@ApiTags('admisiones')
@ApiBearerAuth()
@UseGuards(JwtAuthGuard, RolesGuard)
@Controller('admisiones/pacientes')
export class PacientesController {
  constructor(
    private readonly registrar: RegistrarPacienteUseCase,
    private readonly buscar: BuscarPacientesUseCase,
  ) {}

  @Post()
  @Roles(RolUsuario.RECEPCIONISTA, RolUsuario.ADMIN)
  crear(@Body() dto: RegistrarPacienteDto) {
    return this.registrar.ejecutar(dto);
  }

  @Get()
  @Roles(RolUsuario.RECEPCIONISTA, RolUsuario.MEDICO, RolUsuario.ENFERMERO, RolUsuario.ADMIN)
  listar(@Query('q') q = '') {
    return this.buscar.ejecutar(q);
  }
}
```

`MedicosController` es igual de corto: `POST /admisiones/medicos` (solo `ADMIN`) y `GET /admisiones/medicos` (todos los roles de personal) que llama a `listarActivos()`.

Paso 2. Máquina de estados del turno (State)

Los estados válidos y las transiciones permitidas del informe:

DESDE	ACCIONES PERMITIDAS
`programado`	confirmar, registrar llegada, cancelar
`confirmado`	registrar llegada, cancelar
`en_sala_espera`	atender, cancelar
`atendido`	ninguna (estado final)
`cancelado`	ninguna (estado final)

`domain/state/turno-estado.ts`. TypeScript puro, sin imports de Nest ni de TypeORM:

```ts
import { TransicionInvalidaException } from '../../../shared/exceptions/domain.exception';

export enum EstadoTurno {
  PROGRAMADO = 'programado',
  CONFIRMADO = 'confirmado',
  EN_SALA_ESPERA = 'en_sala_espera',
  ATENDIDO = 'atendido',
  CANCELADO = 'cancelado',
}

/** Estado base: toda acción está prohibida salvo que el estado concreto la permita. */
export abstract class TurnoEstado {
  abstract readonly nombre: EstadoTurno;

  confirmar(): TurnoEstado { throw this.rechazar('confirmar'); }
  registrarLlegada(): TurnoEstado { throw this.rechazar('registrar la llegada de'); }
  atender(): TurnoEstado { throw this.rechazar('atender'); }
  cancelar(): TurnoEstado { throw this.rechazar('cancelar'); }

  protected rechazar(accion: string): TransicionInvalidaException {
    return new TransicionInvalidaException(
      `No se puede ${accion} un turno en estado "${this.nombre}"`,
    );
  }
}

class Programado extends TurnoEstado {
  readonly nombre = EstadoTurno.PROGRAMADO;
  confirmar() { return new Confirmado(); }
  registrarLlegada() { return new EnSalaDeEspera(); }
  cancelar() { return new Cancelado(); }
```

```
}

class Confirmado extends TurnoEstado {
  readonly nombre = EstadoTurno.CONFIRMADO;
  registrarLlegada() { return new EnSalaDeEspera(); }
  cancelar() { return new Cancelado(); }
}

class EnSalaDeEspera extends TurnoEstado {
  readonly nombre = EstadoTurno.EN_SALA_ESPERA;
  atender() { return new Atendido(); }
  cancelar() { return new Cancelado(); }
}

class Atendido extends TurnoEstado {
  readonly nombre = EstadoTurno.ATENDIDO;
}

class Cancelado extends TurnoEstado {
  readonly nombre = EstadoTurno.CANCELADO;
}

/** Reconstruye el objeto-estado a partir del texto guardado en la base. */
export function estadoDeTurno(nombre: string): TurnoEstado {
  switch (nombre) {
    case EstadoTurno.PROGRAMADO: return new Programado();
    case EstadoTurno.CONFIRMADO: return new Confirmado();
    case EstadoTurno.EN_SALA_ESPERA: return new EnSalaDeEspera();
    case EstadoTurno.ATENDIDO: return new Atendido();
    case EstadoTurno.CANCELADO: return new Cancelado();
    default: throw new TransicionInvalidaException(`Estado de turno desconocido: "${nombre}"`);
  }
}
```

`domain/turno-workflow.ts`. Es el único lugar por el que pasa un cambio de estado; nadie asigna `turno.estado = ...` a mano:

```
import { EstadoTurno, estadoDeTurno } from './state/turno-estado';

export type AccionTurno = 'confirmar' | 'registrarLlegada' | 'atender' | 'cancelar';

/** Devuelve el estado siguiente o lanza TransicionInvalidaException (HTTP 422). */
export function siguienteEstado(actual: string, accion: AccionTurno): EstadoTurno {
  return estadoDeTurno(actual)[accion]().nombre;
}
```

Paso 3. Reserva de turnos con concurrencia (HU 1.1)

El problema. Dos recepcionistas reservan el mismo médico a la misma hora al mismo tiempo. Las dos pasan la validación "¿está libre?" y las dos guardan. La defensa no puede estar en el código de la aplicación: tiene que estar en la base. Es el índice único parcial `uq_turno_medico_horario_activo` de los ajustes al modelo: el segundo `INSERT` falla con el código `23505` y el repositorio lo convierte en un conflicto.

`domain/repositories/turno-repository.interface.ts`:

```
import { Turno } from '../entities/turno.entity';
import { EstadoTurno } from '../state/turno-estado';

export interface NuevoTurno {
  pacienteId: string;
  medicoId: string;
  fechaHora: Date;
  motivo?: string;
  prioridad?: string;
}

export interface ITurnoRepository {
  findById(id: string): Promise<Turno | null>;
  /** Lanza ConflictoDeConcurrenciaException si el horario ya está tomado. */
  crear(datos: NuevoTurno): Promise<Turno>;
  /** UPDATE condicional por versión. false => otra petición modificó el turno antes. */
  actualizarEstado(id: string, versionLeida: number, nuevoEstado: EstadoTurno): Promise<boolean>;
  listarPorMedicoYDia(medicoId: string, dia: Date): Promise<Turno[]>;
}

export const TURNO_REPOSITORY = Symbol('TURNO_REPOSITORY');
```

`infrastructure/repositories/typeorm-turno.repository.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Between, Repository } from 'typeorm';
import { esViolacionDeUnicidad } from '../../../shared/database/errores-pg';
import { ConflictoDeConcurrenciaException } from '../../../shared/exceptions/domain.exception';
import { Turno } from '../../domain/entities/turno.entity';
import { ITurnoRepository, NuevoTurno } from '../../../domain/repositories/turno-repository.interface';
import { EstadoTurno } from '../../domain/state/turno-estado';

@Injectable()
export class TypeOrmTurnoRepository implements ITurnoRepository {
  constructor(@InjectRepository(Turno) private readonly repo: Repository<Turno>) {}

  findById(id: string) {
    return this.repo.findOne({ where: { id }, relations: { paciente: true, medico: true } });
  }

  async crear(datos: NuevoTurno): Promise<Turno> {
    const turno = this.repo.create({
      paciente: { id: datos.pacienteId },
      medico: { id: datos.medicoId },
      fechaHora: datos.fechaHora,
      motivo: datos.motivo ?? null,
      prioridad: datos.prioridad ?? 'normal',
    });

    try {
      return await this.repo.save(turno);
    } catch (error) {
      if (esViolacionDeUnicidad(error)) throw new ConflictoDeConcurrenciaException('El horario');
      throw error;
    }
  }

  async actualizarEstado(id: string, versionLeida: number, nuevoEstado: EstadoTurno) {
    const resultado = await this.repo
      .createQueryBuilder()
      .update(Turno)
      .set({ estado: nuevoEstado, version: () => 'version + 1' })
      .where('id = :id AND version = :versionLeida', { id, versionLeida })
      .execute();

    return resultado.affected === 1;
  }

  listarPorMedicoYDia(medicoId: string, dia: Date) {
    const desde = new Date(dia);
    desde.setHours(0, 0, 0, 0);
    const hasta = new Date(desde);
    hasta.setDate(hasta.getDate() + 1);

    return this.repo.find({
      where: {
        medico: { id: medicoId },
        fechaHora: Between(desde, new Date(hasta.getTime() - 1)),
      },
      relations: { paciente: true },
      order: { fechaHora: 'ASC' },
    });
  }
}
```

`application/reservar-turno.use-case.ts`:

```
import { Inject, Injectable } from '@nestjs/common';
import {
  DomainException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { Turno } from '../domain/entities/turno.entity';
import {
  IMedicoRepository,
  MEDICO_REPOSITORY,
} from '../domain/repositories/medico-repository.interface';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../domain/repositories/paciente-repository.interface';
import { ITurnoRepository, TURNO_REPOSITORY } from '../domain/repositories/turno-repository.interface';

export interface ReservarTurnoCommand {
  pacienteId: string;
  medicoId: string;
  fechaHora: Date;
  motivo?: string;
}

@Injectable()
export class ReservarTurnoUseCase {
  constructor(
    @Inject(TURNO_REPOSITORY) private readonly turnos: ITurnoRepository,
```

```
    @Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository,
    @Inject(MEDICO_REPOSITORY) private readonly medicos: IMedicoRepository,
  ) {}

  async ejecutar(cmd: ReservarTurnoCommand): Promise<Turno> {
    if (cmd.fechaHora.getTime() <= Date.now()) {
      throw new DomainException('No se puede reservar un turno en el pasado');
    }

    if (!(await this.pacientes.findById(cmd.pacienteId))) {
      throw new RecursoNoEncontradoException('Paciente');
    }
    const medico = await this.medicos.findById(cmd.medicoId);
    if (!medico) throw new RecursoNoEncontradoException('Médico');
    if (!medico.activo) throw new DomainException('El médico no está disponible');

    // Si dos reservas llegan juntas, la base rechaza la segunda (409).
    return this.turnos.crear(cmd);
  }
}
```

`presentation/dto/reservar-turno.dto.ts`:

```
import { ApiProperty, ApiPropertyOptional } from '@nestjs/swagger';
import { Type } from 'class-transformer';
import { IsDate, IsOptional, IsString, IsUUID, MaxLength } from 'class-validator';

export class ReservarTurnoDto {
  @ApiProperty() @IsUUID() pacienteId: string;
  @ApiProperty() @IsUUID() medicoId: string;

  @ApiProperty({ example: '2026-11-03T10:30:00-03:00' })
  @Type(() => Date)
  @IsDate()
  fechaHora: Date;

  @ApiPropertyOptional() @IsOptional() @IsString() @MaxLength(200) motivo?: string;
}
```

Paso 4. Casos de uso del ciclo de vida del turno

Los cuatro (`ConfirmarTurno` , `RegistrarLlegada` , `AtenderTurno` , `CancelarTurno`) hacen lo mismo con una acción distinta. Se escribe una clase base y cada caso de uso queda en cuatro líneas. Cada acción sigue siendo un caso de uso propio (SRP), y se pueden inyectar por separado.

`application/transicionar-turno.base.ts`:

```
import {
  ConflictoDeConcurrenciaException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { Turno } from '../domain/entities/turno.entity';
import { ITurnoRepository } from '../domain/repositories/turno-repository.interface';
import { AccionTurno, siguienteEstado } from '../domain/turno-workflow';

export abstract class TransicionarTurnoBase {
  protected abstract readonly accion: AccionTurno;

  constructor(protected readonly turnos: ITurnoRepository) {}

  async ejecutar(turnoId: string): Promise<Turno> {
    const turno = await this.turnos.findById(turnoId);
    if (!turno) throw new RecursoNoEncontradoException('Turno');

    // El State valida la transición. Si no corresponde, lanza TransicionInvalidaException (422).
    const nuevoEstado = siguienteEstado(turno.estado, this.accion);

    // El UPDATE condicional por versión protege contra dos acciones simultáneas (409).
    const ok = await this.turnos.actualizarEstado(turno.id, turno.version, nuevoEstado);
    if (!ok) throw new ConflictoDeConcurrenciaException('El turno');

    return (await this.turnos.findById(turno.id)) ?? turno;
  }
}
```

`application/confirmar-turno.use-case.ts` (los otros tres son idénticos cambiando clase y acción):

```
import { Inject, Injectable } from '@nestjs/common';
import { ITurnoRepository, TURNO_REPOSITORY } from '../domain/repositories/turno-repository.interface';
import { TransicionarTurnoBase } from './transicionar-turno.base';

@Injectable()
```

```
export class ConfirmarTurnoUseCase extends TransicionarTurnoBase {
  protected readonly accion = 'confirmar' as const;

  constructor(@Inject(TURNO_REPOSITORY) turnos: ITurnoRepository) {
    super(turnos);
  }
}

// RegistrarLlegadaUseCase -> accion = 'registrarLlegada'
// AtenderTurnoUseCase     -> accion = 'atender'
// CancelarTurnoUseCase    -> accion = 'cancelar'
```

`presentation/turnos.controller.ts`:

```
import { Body, Controller, Get, Param, ParseUUIDPipe, Patch, Post, Query, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { Roles } from '../../shared/decorators/roles.decorator';
import { JwtAuthGuard } from '../../shared/guards/jwt-auth.guard';
import { RolesGuard } from '../../shared/guards/roles.guard';
import { RolUsuario } from '../../iam/domain/enums/rol.enum';
import { AtenderTurnoUseCase } from '../application/atender-turno.use-case';
import { CancelarTurnoUseCase } from '../application/cancelar-turno.use-case';
import { ConfirmarTurnoUseCase } from '../application/confirmar-turno.use-case';
import { ListarAgendaUseCase } from '../application/listar-agenda.use-case';
import { RegistrarLlegadaUseCase } from '../application/registrar-llegada.use-case';
import { ReservarTurnoUseCase } from '../application/reservar-turno.use-case';
import { ReservarTurnoDto } from './dto/reservar-turno.dto';

@ApiTags('admisiones')
@ApiBearerAuth()
@UseGuards(JwtAuthGuard, RolesGuard)
@Controller('admisiones/turnos')
export class TurnosController {
  constructor(
    private readonly reservar: ReservarTurnoUseCase,
    private readonly confirmar: ConfirmarTurnoUseCase,
    private readonly llegada: RegistrarLlegadaUseCase,
    private readonly atender: AtenderTurnoUseCase,
    private readonly cancelar: CancelarTurnoUseCase,
    private readonly agenda: ListarAgendaUseCase,
  ) {}

  @Post()
  @Roles(RolUsuario.RECEPCIONISTA, RolUsuario.ADMIN)
  crear(@Body() dto: ReservarTurnoDto) {
    return this.reservar.ejecutar(dto);
  }

  @Get()
  @Roles(RolUsuario.RECEPCIONISTA, RolUsuario.MEDICO, RolUsuario.ADMIN)
  listar(@Query('medicoId', ParseUUIDPipe) medicoId: string, @Query('dia') dia: string) {
    return this.agenda.ejecutar(medicoId, new Date(dia));
  }

  @Patch(':id/confirmar')
  @Roles(RolUsuario.RECEPCIONISTA, RolUsuario.ADMIN)
  confirmarTurno(@Param('id', ParseUUIDPipe) id: string) {
    return this.confirmar.ejecutar(id);
  }

  @Patch(':id/llegada')
  @Roles(RolUsuario.RECEPCIONISTA, RolUsuario.ADMIN)
  registrarLlegada(@Param('id', ParseUUIDPipe) id: string) {
    return this.llegada.ejecutar(id);
  }

  @Patch(':id/atender')
  @Roles(RolUsuario.MEDICO)
  atenderTurno(@Param('id', ParseUUIDPipe) id: string) {
    return this.atender.ejecutar(id);
  }

  @Patch(':id/cancelar')
  @Roles(RolUsuario.RECEPCIONISTA, RolUsuario.ADMIN)
  cancelarTurno(@Param('id', ParseUUIDPipe) id: string) {
    return this.cancelar.ejecutar(id);
  }
}
```

`ListarAgendaUseCase` es una línea: `return this.turnos.listarPorMedicoYDia(medicoId, dia)`.

Paso 5. Cola de triaje y entreturnos

La regla. Código Rojo pasa al frente. Dentro de la misma prioridad, el que llegó primero se atiende primero. El orden es lógica pura: se escribe como función y se prueba sin base de datos.

`domain/cola-prioridad.ts`:

```ts
import { ColaEntreturno } from './entities/cola-entreturno.entity';

export enum PrioridadTriaje {
  CODIGO_ROJO = 'codigo_rojo',
  URGENTE = 'urgente',
  NORMAL = 'normal',
}

const PESO: Record<string, number> = {
  [PrioridadTriaje.CODIGO_ROJO]: 0,
  [PrioridadTriaje.URGENTE]: 1,
  [PrioridadTriaje.NORMAL]: 2,
};

/** Pendientes ordenados: primero por prioridad, después por orden de llegada. */
export function ordenarCola(items: ColaEntreturno[]): ColaEntreturno[] {
  return items
    .filter((i) => !i.atendido)
    .sort(
      (a, b) =>
        (PESO[a.prioridad] ?? 99) - (PESO[b.prioridad] ?? 99) ||
        a.creadoEn.getTime() - b.creadoEn.getTime(),
    );
}
```

`domain/repositories/cola-repository.interface.ts`:

```ts
import { ColaEntreturno } from '../entities/cola-entreturno.entity';

export interface IColaRepository {
  agregar(pacienteId: string, medicoId: string, prioridad: string): Promise<ColaEntreturno>;
  pendientesPorMedico(medicoId: string): Promise<ColaEntreturno[]>;
  /** UPDATE ... WHERE atendido = false. false => otro médico ya lo llamó. */
  marcarAtendido(id: string): Promise<boolean>;
}

export const COLA_REPOSITORY = Symbol('COLA_REPOSITORY');
```

`infrastructure/repositories/typeorm-cola.repository.ts`:

```ts
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { ColaEntreturno } from '../../domain/entities/cola-entreturno.entity';
import { IColaRepository } from '../../domain/repositories/cola-repository.interface';

@Injectable()
export class TypeOrmColaRepository implements IColaRepository {
  constructor(@InjectRepository(ColaEntreturno) private readonly repo: Repository<ColaEntreturno>) {}

  agregar(pacienteId: string, medicoId: string, prioridad: string) {
    return this.repo.save(
      this.repo.create({ paciente: { id: pacienteId }, medico: { id: medicoId }, prioridad }),
    );
  }

  pendientesPorMedico(medicoId: string) {
    return this.repo.find({
      where: { medico: { id: medicoId }, atendido: false },
      relations: { paciente: true },
    });
  }

  async marcarAtendido(id: string) {
    const r = await this.repo.update({ id, atendido: false }, { atendido: true });
    return r.affected === 1;
  }
}
```

`application/llamar-siguiente.use-case.ts`:

```ts
import { Inject, Injectable } from '@nestjs/common';
import {
  ConflictoDeConcurrenciaException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { ordenarCola } from '../domain/cola-prioridad';
import { ColaEntreturno } from '../domain/entities/cola-entreturno.entity';
import { COLA_REPOSITORY, IColaRepository } from '../domain/repositories/cola-repository.interface';

@Injectable()
export class LlamarSiguienteUseCase {
  constructor(@Inject(COLA_REPOSITORY) private readonly cola: IColaRepository) {}
```

```
  async ejecutar(medicoId: string): Promise<ColaEntreturno> {
    const siguiente = ordenarCola(await this.cola.pendientesPorMedico(medicoId))[0];
    if (!siguiente) throw new RecursoNoEncontradoException('Paciente en cola');

    if (!(await this.cola.marcarAtendido(siguiente.id))) {
      throw new ConflictoDeConcurrenciaException('El paciente de la cola');
    }
    return siguiente;
  }
}
```

`AgregarACola` valida paciente y médico, y guarda con la prioridad recibida (`PrioridadTriaje` , validada con `@IsEnum` en el DTO). `VerCola` devuelve `ordenarCola(pendientes)` . El `ColaController` expone:

```
@Post()           // RECEPCIONISTA, MEDICO, ENFERMERO, ADMIN: agrega a la cola
@Get()            // ?medicoId=...  devuelve la cola ya ordenada
@Post('siguiente')// MEDICO: { medicoId } llama al próximo paciente
```

Paso 6. Módulo

`admisiones/admisiones.module.ts` . Exporta los repositorios de paciente y médico para que Camas, Historia Clínica, Atención y Facturación puedan consultarlos:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AgregarAColaUseCase } from './application/agregar-a-cola.use-case';
import { AtenderTurnoUseCase } from './application/atender-turno.use-case';
import { BuscarPacientesUseCase } from './application/buscar-pacientes.use-case';
import { CancelarTurnoUseCase } from './application/cancelar-turno.use-case';
import { ConfirmarTurnoUseCase } from './application/confirmar-turno.use-case';
import { LlamarSiguienteUseCase } from './application/llamar-siguiente.use-case';
import { ListarAgendaUseCase } from './application/listar-agenda.use-case';
import { RegistrarLlegadaUseCase } from './application/registrar-llegada.use-case';
import { RegistrarMedicoUseCase } from './application/registrar-medico.use-case';
import { RegistrarPacienteUseCase } from './application/registrar-paciente.use-case';
import { ReservarTurnoUseCase } from './application/reservar-turno.use-case';
import { VerColaUseCase } from './application/ver-cola.use-case';
import { entidadesAdmisiones } from './domain/entities';
import { COLA_REPOSITORY } from './domain/repositories/cola-repository.interface';
import { MEDICO_REPOSITORY } from './domain/repositories/medico-repository.interface';
import { PACIENTE_REPOSITORY } from './domain/repositories/paciente-repository.interface';
import { TURNO_REPOSITORY } from './domain/repositories/turno-repository.interface';
import { TypeOrmColaRepository } from './infrastructure/repositories/typeorm-cola.repository';
import { TypeOrmMedicoRepository } from './infrastructure/repositories/typeorm-medico.repository';
import { TypeOrmPacienteRepository } from './infrastructure/repositories/typeorm-paciente.repository';
import { TypeOrmTurnoRepository } from './infrastructure/repositories/typeorm-turno.repository';
import { AdmisionesController } from './presentation/admisiones.controller';
import { ColaController } from './presentation/cola.controller';
import { MedicosController } from './presentation/medicos.controller';
import { PacientesController } from './presentation/pacientes.controller';
import { TurnosController } from './presentation/turnos.controller';

@Module({
  imports: [TypeOrmModule.forFeature(entidadesAdmisiones)],
  controllers: [
    AdmisionesController,
    PacientesController,
    MedicosController,
    TurnosController,
    ColaController,
  ],
  providers: [
    { provide: PACIENTE_REPOSITORY, useClass: TypeOrmPacienteRepository },
    { provide: MEDICO_REPOSITORY, useClass: TypeOrmMedicoRepository },
    { provide: TURNO_REPOSITORY, useClass: TypeOrmTurnoRepository },
    { provide: COLA_REPOSITORY, useClass: TypeOrmColaRepository },
    RegistrarPacienteUseCase,
    BuscarPacientesUseCase,
    RegistrarMedicoUseCase,
    ReservarTurnoUseCase,
    ConfirmarTurnoUseCase,
    RegistrarLlegadaUseCase,
    AtenderTurnoUseCase,
    CancelarTurnoUseCase,
    ListarAgendaUseCase,
    AgregarAColaUseCase,
    VerColaUseCase,
    LlamarSiguienteUseCase,
  ],
  exports: [PACIENTE_REPOSITORY, MEDICO_REPOSITORY],
})
export class AdmisionesModule {}
```

Endpoints

MÉTODO	RUTA	ROLES	RESPUESTAS
POST	`/admisiones/pacientes`	Recepcionista, Admin	201, 422 (DNI repetido)
GET	`/admisiones/pacientes?q=`	Personal	200
POST	`/admisiones/medicos`	Admin	201, 422
GET	`/admisiones/medicos`	Personal	200
POST	`/admisiones/turnos`	Recepcionista, Admin	201, **409 horario tomado**, 404, 422
GET	`/admisiones/turnos?medicoId=&dia=`	Recepcionista, Médico, Admin	200
PATCH	`/admisiones/turnos/:id/confirmar`	Recepcionista, Admin	200, **409**, 422 transición inválida
PATCH	`/admisiones/turnos/:id/llegada`	Recepcionista, Admin	200, 409, 422
PATCH	`/admisiones/turnos/:id/atender`	Médico	200, 409, 422
PATCH	`/admisiones/turnos/:id/cancelar`	Recepcionista, Admin	200, 409, 422
POST	`/admisiones/cola`	Personal	201
GET	`/admisiones/cola?medicoId=`	Personal	200, ya ordenada
POST	`/admisiones/cola/siguiente`	Médico	200, 404 cola vacía, 409

Tests

`domain/state/turno-estado.spec.ts`: la máquina de estados completa, sin base de datos:

```
import { TransicionInvalidaException } from '../../../shared/exceptions/domain.exception';
import { siguienteEstado } from '../turno-workflow';
import { EstadoTurno } from './turno-estado';

describe('Máquina de estados del turno', () => {
  it('recorre el camino feliz: programado -> confirmado -> en sala -> atendido', () => {
    expect(siguienteEstado('programado', 'confirmar')).toBe(EstadoTurno.CONFIRMADO);
    expect(siguienteEstado('confirmado', 'registrarLlegada')).toBe(EstadoTurno.EN_SALA_ESPERA);
    expect(siguienteEstado('en_sala_espera', 'atender')).toBe(EstadoTurno.ATENDIDO);
  });

  it('permite cancelar mientras el turno no fue atendido', () => {
    for (const estado of ['programado', 'confirmado', 'en_sala_espera']) {
      expect(siguienteEstado(estado, 'cancelar')).toBe(EstadoTurno.CANCELADO);
    }
  });

  it.each([
    ['atendido', 'cancelar'],
    ['cancelado', 'confirmar'],
    ['programado', 'atender'],        // no se puede saltar la sala de espera
    ['confirmado', 'confirmar'],
  ] as const)('rechaza %s -> %s', (estado, accion) => {
    expect(() => siguienteEstado(estado, accion)).toThrow(TransicionInvalidaException);
  });
});
```

`application/reservar-turno.use-case.spec.ts`: dos reservas simultáneas, solo gana la primera:

```ts
import { ConflictoDeConcurrenciaException } from '../../shared/exceptions/domain.exception';
import { Medico } from '../domain/entities/medico.entity';
import { Paciente } from '../domain/entities/paciente.entity';
import { Turno } from '../domain/entities/turno.entity';
import { ITurnoRepository, NuevoTurno } from '../domain/repositories/turno-repository.interface';
import { ReservarTurnoUseCase } from './reservar-turno.use-case';

/** Simula el índice único (medico + horario) de la base. */
class TurnosEnMemoria implements ITurnoRepository {
  private readonly tomados = new Set<string>();

  async findById() { return null; }
  async actualizarEstado() { return true; }
  async listarPorMedicoYDia() { return []; }

  async crear(d: NuevoTurno): Promise<Turno> {
    const clave = `${d.medicoId}|${d.fechaHora.toISOString()}`;
    if (this.tomados.has(clave)) throw new ConflictoDeConcurrenciaException('El horario');
    this.tomados.add(clave);
    return Object.assign(new Turno(), { id: clave });
  }
}

describe('ReservarTurnoUseCase', () => {
  const medico = Object.assign(new Medico(), { id: 'm1', activo: true });
  const paciente = Object.assign(new Paciente(), { id: 'p1' });
  const manana = new Date(Date.now() + 24 * 3600 * 1000);

  const crear = () =>
    new ReservarTurnoUseCase(
      new TurnosEnMemoria(),
      { findById: async () => paciente } as never,
      { findById: async () => medico } as never,
    );

  it('la segunda reserva del mismo horario responde conflicto', async () => {
    const useCase = crear();
    const cmd = { pacienteId: 'p1', medicoId: 'm1', fechaHora: manana };

    await expect(useCase.ejecutar(cmd)).resolves.toBeDefined();
    await expect(useCase.ejecutar(cmd)).rejects.toBeInstanceOf(ConflictoDeConcurrenciaException);
  });

  it('no permite reservar en el pasado', async () => {
    await expect(
      crear().ejecutar({ pacienteId: 'p1', medicoId: 'm1', fechaHora: new Date(Date.now() - 1000) }),
    ).rejects.toThrow('pasado');
  });
});
```

`domain/cola-prioridad.spec.ts`:

```ts
import { ColaEntreturno } from './entities/cola-entreturno.entity';
import { ordenarCola } from './cola-prioridad';

const item = (id: string, prioridad: string, minuto: number, atendido = false) =>
  Object.assign(new ColaEntreturno(), {
    id, prioridad, atendido, creadoEn: new Date(2026, 10, 3, 9, minuto),
  });

describe('ordenarCola', () => {
  it('pone Código Rojo al frente aunque haya llegado último', () => {
    const cola = ordenarCola([item('a', 'normal', 0), item('b', 'urgente', 1), item('c', 'codigo_rojo', 2)]);
    expect(cola.map((i) => i.id)).toEqual(['c', 'b', 'a']);
  });

  it('a igual prioridad respeta el orden de llegada', () => {
    const cola = ordenarCola([item('b', 'normal', 5), item('a', 'normal', 1)]);
    expect(cola.map((i) => i.id)).toEqual(['a', 'b']);
  });

  it('excluye a los ya atendidos', () => {
    expect(ordenarCola([item('a', 'normal', 0, true)])).toEqual([]);
  });
});
```

Terminado cuando

- [ ] Desde Swagger: crear un paciente, un médico y un turno. Repetir el `POST /admisiones/turnos` con el mismo médico y horario devuelve **409**.
- [ ] Llevar un turno de `programado` a `atendido` paso a paso. Intentar `atender` un turno `programado` devuelve **422**.
- [ ] Con tres personas en la cola (normal, urgente, código rojo) `GET /admisiones/cola` las devuelve en orden de prioridad.

- [ ] Ningún archivo dentro de `admisiones/domain/state/` ni `turno-workflow.ts` importa `@nestjs/*` ni `typeorm`.
- [ ] Los tests nuevos pasan y los anteriores siguen en verde.

Recursos y Camas

Objetivo. Mostrar el mapa de camas, asignarlas a pacientes sin que dos administrativos ocupen la misma, llevar cada cama por su ciclo (libre, ocupada, en limpieza, bloqueada) y controlar el inventario de insumos con descuento de stock sin sobreventa.

Patrones de este módulo: locking optimista y State (reutiliza lo hecho en Admisiones), `UPDATE` atómico para el stock, evento de dominio para el stock bajo.

Qué hay hoy

YA EXISTE	FALTA
Entidades `Cama`, `AsignacionCama`, `Insumo`, `StockInsumo`	Repositorios y casos de uso
Columna `version` en `Cama`	Máquina de estados de la cama
`CamasController` con `GET /camas/ping`	Asignación concurrente y alta del paciente
	Inventario y descuento de stock

Antes de empezar: tener hecho Admisiones, porque este módulo necesita validar que el paciente existe. `Insumo.precioUnitario` ya está en la entidad.

Estados de la cama

DESDE	ACCIÓN	HACIA
`libre`	asignar	`ocupada`
`libre`	bloquear	`bloqueada`
`ocupada`	liberar (alta o traslado)	`en_limpieza`
`en_limpieza`	confirmar limpieza	`libre`
`en_limpieza`	bloquear	`bloqueada`
`bloqueada`	habilitar	`libre`

Una cama ocupada nunca vuelve a libre sin pasar por limpieza, y una ocupada no se puede bloquear.

Estructura final

```
camas/
├── domain/
│   ├── entities/                          (existen)
│   ├── repositories/
│   │   ├── cama-repository.interface.ts       NUEVO
│   │   └── inventario-repository.interface.ts NUEVO
│   ├── state/cama-estado.ts                   NUEVO
│   └── cama-workflow.ts                       NUEVO
├── application/                           NUEVO
├── infrastructure/repositories/           NUEVO
├── presentation/
│   ├── camas.controller.ts                (existe: ping)
│   ├── gestion-camas.controller.ts        NUEVO
│   ├── inventario.controller.ts           NUEVO
│   └── dto/                               NUEVO
└── camas.module.ts                        (se modifica)
```

Paso 1. State de la cama

Es la misma técnica que el turno. `domain/state/cama-estado.ts`:

```
import { TransicionInvalidaException } from '../../../shared/exceptions/domain.exception';

export enum EstadoCama {
  LIBRE = 'libre',
  OCUPADA = 'ocupada',
  EN_LIMPIEZA = 'en_limpieza',
  BLOQUEADA = 'bloqueada',
}

export abstract class CamaEstado {
  abstract readonly nombre: EstadoCama;

  asignar(): CamaEstado { throw this.rechazar('asignar'); }
  liberar(): CamaEstado { throw this.rechazar('liberar'); }
  confirmarLimpieza(): CamaEstado { throw this.rechazar('confirmar la limpieza de'); }
  bloquear(): CamaEstado { throw this.rechazar('bloquear'); }
  habilitar(): CamaEstado { throw this.rechazar('habilitar'); }

  protected rechazar(accion: string): TransicionInvalidaException {
    return new TransicionInvalidaException(
      `No se puede ${accion} una cama en estado "${this.nombre}"`,
    );
  }
}

class Libre extends CamaEstado {
  readonly nombre = EstadoCama.LIBRE;
  asignar() { return new Ocupada(); }
  bloquear() { return new Bloqueada(); }
}

class Ocupada extends CamaEstado {
  readonly nombre = EstadoCama.OCUPADA;
  liberar() { return new EnLimpieza(); }
}

class EnLimpieza extends CamaEstado {
  readonly nombre = EstadoCama.EN_LIMPIEZA;
  confirmarLimpieza() { return new Libre(); }
  bloquear() { return new Bloqueada(); }
}

class Bloqueada extends CamaEstado {
  readonly nombre = EstadoCama.BLOQUEADA;
  habilitar() { return new Libre(); }
}

export function estadoDeCama(nombre: string): CamaEstado {
  switch (nombre) {
    case EstadoCama.LIBRE: return new Libre();
    case EstadoCama.OCUPADA: return new Ocupada();
    case EstadoCama.EN_LIMPIEZA: return new EnLimpieza();
    case EstadoCama.BLOQUEADA: return new Bloqueada();
    default: throw new TransicionInvalidaException(`Estado de cama desconocido: "${nombre}"`);
  }
}
```

`domain/cama-workflow.ts`:

```
import { EstadoCama, estadoDeCama } from './state/cama-estado';

export type AccionCama = 'asignar' | 'liberar' | 'confirmarLimpieza' | 'bloquear' | 'habilitar';

export function siguienteEstadoCama(actual: string, accion: AccionCama): EstadoCama {
  return estadoDeCama(actual)[accion]().nombre;
}
```

Paso 2. Repositorio de camas

La asignación y la liberación tocan dos tablas (`camas` y `asignaciones_cama`) y tienen que ocurrir juntas o no ocurrir. Por eso se hacen dentro de una transacción en el repositorio.

`domain/repositories/cama-repository.interface.ts`:

```
import { AsignacionCama } from '../entities/asignacion-cama.entity';
import { Cama } from '../entities/cama.entity';
import { EstadoCama } from '../state/cama-estado';

export interface ICamaRepository {
```

```
  findById(id: string): Promise<Cama | null>;
  listar(sector?: string): Promise<Cama[]>;
  crear(codigo: string, sector: string): Promise<Cama>;
  findAsignacionVigentePorPaciente(pacienteId: string): Promise<AsignacionCama | null>;

  /** Cambio de estado simple. false => la versión cambió (otra petición ganó). */
  actualizarEstado(id: string, versionLeida: number, nuevoEstado: EstadoCama): Promise<boolean>;

  /** Cama libre -> ocupada + nueva asignación, en una transacción. null => perdió la carrera. */
  asignar(camaId: string, versionLeida: number, pacienteId: string): Promise<AsignacionCama | null>;

  /** Cama ocupada -> en limpieza + cierra la asignación vigente. false => perdió la carrera. */
  liberar(camaId: string, versionLeida: number): Promise<boolean>;
}

export const CAMA_REPOSITORY = Symbol('CAMA_REPOSITORY');
```

`infrastructure/repositories/typeorm-cama.repository.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { DataSource, IsNull, Repository } from 'typeorm';
import { esViolacionDeUnicidad } from '../../../shared/database/errores-pg';
import { DomainException } from '../../../shared/exceptions/domain.exception';
import { AsignacionCama } from '../../domain/entities/asignacion-cama.entity';
import { Cama } from '../../domain/entities/cama.entity';
import { ICamaRepository } from '../../domain/repositories/cama-repository.interface';
import { EstadoCama } from '../../domain/state/cama-estado';

@Injectable()
export class TypeOrmCamaRepository implements ICamaRepository {
  constructor(
    @InjectRepository(Cama) private readonly camas: Repository<Cama>,
    @InjectRepository(AsignacionCama) private readonly asignaciones: Repository<AsignacionCama>,
    private readonly dataSource: DataSource,
  ) {}

  findById(id: string) {
    return this.camas.findOneBy({ id });
  }

  listar(sector?: string) {
    return this.camas.find({
      where: sector ? { sector } : {},
      order: { sector: 'ASC', codigo: 'ASC' },
    });
  }

  async crear(codigo: string, sector: string) {
    try {
      return await this.camas.save(this.camas.create({ codigo, sector }));
    } catch (error) {
      if (esViolacionDeUnicidad(error)) throw new DomainException(`Ya existe la cama ${codigo}`);
      throw error;
    }
  }

  findAsignacionVigentePorPaciente(pacienteId: string) {
    return this.asignaciones.findOne({
      where: { pacienteId, hasta: IsNull() },
      relations: { cama: true },
    });
  }

  async actualizarEstado(id: string, versionLeida: number, nuevoEstado: EstadoCama) {
    const r = await this.camas
      .createQueryBuilder()
      .update(Cama)
      .set({ estado: nuevoEstado, version: () => 'version + 1' })
      .where('id = :id AND version = :versionLeida', { id, versionLeida })
      .execute();
    return r.affected === 1;
  }

  asignar(camaId: string, versionLeida: number, pacienteId: string) {
    return this.dataSource.transaction(async (manager) => {
      const r = await manager
        .createQueryBuilder()
        .update(Cama)
        .set({ estado: EstadoCama.OCUPADA, version: () => 'version + 1' })
        .where('id = :camaId AND version = :versionLeida AND estado = :libre', {
          camaId,
          versionLeida,
          libre: EstadoCama.LIBRE,
        })
        .execute();
      if (r.affected !== 1) return null;

      return manager.save(manager.create(AsignacionCama, { cama: { id: camaId }, pacienteId }));
    });
  }
```

```
  liberar(camaId: string, versionLeida: number) {
    return this.dataSource.transaction(async (manager) => {
      const r = await manager
        .createQueryBuilder()
        .update(Cama)
        .set({ estado: EstadoCama.EN_LIMPIEZA, version: () => 'version + 1' })
        .where('id = :camaId AND version = :versionLeida AND estado = :ocupada', {
          camaId,
          versionLeida,
          ocupada: EstadoCama.OCUPADA,
        })
        .execute();
      if (r.affected !== 1) return false;

      await manager
        .createQueryBuilder()
        .update(AsignacionCama)
        .set({ hasta: () => 'now()' })
        .where('"camaId" = :camaId AND "hasta" IS NULL', { camaId })
        .execute();
      return true;
    });
  }
}
```

Paso 3. Casos de uso de camas

`application/asignar-cama.use-case.ts`:

```
import { Inject, Injectable } from '@nestjs/common';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../../admisiones/domain/repositories/paciente-repository.interface';
import {
  ConflictoDeConcurrenciaException,
  DomainException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { siguienteEstadoCama } from '../domain/cama-workflow';
import { AsignacionCama } from '../domain/entities/asignacion-cama.entity';
import { CAMA_REPOSITORY, ICamaRepository } from '../domain/repositories/cama-repository.interface';

@Injectable()
export class AsignarCamaUseCase {
  constructor(
    @Inject(CAMA_REPOSITORY) private readonly camas: ICamaRepository,
    @Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository,
  ) {}

  async ejecutar(camaId: string, pacienteId: string): Promise<AsignacionCama> {
    const cama = await this.camas.findById(camaId);
    if (!cama) throw new RecursoNoEncontradoException('Cama');
    if (!(await this.pacientes.findById(pacienteId))) {
      throw new RecursoNoEncontradoException('Paciente');
    }
    if (await this.camas.findAsignacionVigentePorPaciente(pacienteId)) {
      throw new DomainException('El paciente ya tiene una cama asignada');
    }

    // El State rechaza (422) si la cama no está libre.
    siguienteEstadoCama(cama.estado, 'asignar');

    // Si otra persona ocupó la cama entre la lectura y este UPDATE, devuelve null (409).
    const asignacion = await this.camas.asignar(cama.id, cama.version, pacienteId);
    if (!asignacion) throw new ConflictoDeConcurrenciaException('La cama');
    return asignacion;
  }
}
```

`application/liberar-cama.use-case.ts` (alta o traslado del paciente):

```
import { Inject, Injectable } from '@nestjs/common';
import {
  ConflictoDeConcurrenciaException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { siguienteEstadoCama } from '../domain/cama-workflow';
import { CAMA_REPOSITORY, ICamaRepository } from '../domain/repositories/cama-repository.interface';

@Injectable()
export class LiberarCamaUseCase {
  constructor(@Inject(CAMA_REPOSITORY) private readonly camas: ICamaRepository) {}

  async ejecutar(camaId: string): Promise<void> {
    const cama = await this.camas.findById(camaId);
    if (!cama) throw new RecursoNoEncontradoException('Cama');
```

```
    siguienteEstadoCama(cama.estado, 'liberar');

    if (!(await this.camas.liberar(cama.id, cama.version))) {
      throw new ConflictoDeConcurrenciaException('La cama');
    }
  }
}
```

Las otras tres transiciones (`ConfirmarLimpieza` , `BloquearCama` , `HabilitarCama`) solo cambian el estado, igual que en turnos. Una clase base evita repetir:

`application/transicionar-cama.base.ts` :

```
import {
  ConflictoDeConcurrenciaException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { AccionCama, siguienteEstadoCama } from '../domain/cama-workflow';
import { Cama } from '../domain/entities/cama.entity';
import { ICamaRepository } from '../domain/repositories/cama-repository.interface';

export abstract class TransicionarCamaBase {
  protected abstract readonly accion: AccionCama;

  constructor(protected readonly camas: ICamaRepository) {}

  async ejecutar(camaId: string): Promise<Cama> {
    const cama = await this.camas.findById(camaId);
    if (!cama) throw new RecursoNoEncontradoException('Cama');

    const nuevoEstado = siguienteEstadoCama(cama.estado, this.accion);
    if (!(await this.camas.actualizarEstado(cama.id, cama.version, nuevoEstado))) {
      throw new ConflictoDeConcurrenciaException('La cama');
    }
    return (await this.camas.findById(cama.id)) ?? cama;
  }
}
```

```
// application/confirmar-limpieza.use-case.ts
@Injectable()
export class ConfirmarLimpiezaUseCase extends TransicionarCamaBase {
  protected readonly accion = 'confirmarLimpieza' as const;
  constructor(@Inject(CAMA_REPOSITORY) camas: ICamaRepository) { super(camas); }
}

// BloquearCamaUseCase  -> accion = 'bloquear'
// HabilitarCamaUseCase -> accion = 'habilitar'
```

Mapa de camas. `application/obtener-mapa-camas.use-case.ts` agrupa las camas por sector para que el frontend dibuje el mapa:

```
import { Inject, Injectable } from '@nestjs/common';
import { CAMA_REPOSITORY, ICamaRepository } from '../domain/repositories/cama-repository.interface';

export interface SectorDeCamas {
  sector: string;
  libres: number;
  camas: { id: string; codigo: string; estado: string }[];
}

@Injectable()
export class ObtenerMapaCamasUseCase {
  constructor(@Inject(CAMA_REPOSITORY) private readonly camas: ICamaRepository) {}

  async ejecutar(): Promise<SectorDeCamas[]> {
    const porSector = new Map<string, SectorDeCamas>();

    for (const cama of await this.camas.listar()) {
      const sector = porSector.get(cama.sector) ?? { sector: cama.sector, libres: 0, camas: [] };
      sector.camas.push({ id: cama.id, codigo: cama.codigo, estado: cama.estado });
      if (cama.estado === 'libre') sector.libres += 1;
      porSector.set(cama.sector, sector);
    }
    return [...porSector.values()];
  }
}
```

`CrearCamaUseCase` valida que `codigo` y `sector` no estén vacíos y llama a `camas.crear(codigo, sector)` .

Paso 4. Inventario de insumos

El problema. Dos enfermeros descuentan la última caja de ampollas al mismo tiempo. Leer el stock, restar en el código y guardar permite stock negativo. La resta tiene que ser atómica en la base: `cantidad = cantidad - n` solo si `cantidad >= n`.

`domain/repositories/inventario-repository.interface.ts`:

```
import { Insumo } from '../entities/insumo.entity';

export interface IInventarioRepository {
  findInsumo(id: string): Promise<Insumo | null>;
  crearInsumo(datos: { nombre: string; unidad: string; umbralMinimo: number; precioUnitario: string }): Promise<Insumo>;
  ingresarStock(insumoId: string, cantidad: number, ubicacion?: string): Promise<void>;
  /** Resta de forma atómica. false => no hay ninguna ubicación con stock suficiente. */
  descontar(insumoId: string, cantidad: number): Promise<boolean>;
  stockTotal(insumoId: string): Promise<number>;
}

export const INVENTARIO_REPOSITORY = Symbol('INVENTARIO_REPOSITORY');
```

`infrastructure/repositories/typeorm-inventario.repository.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Insumo } from '../../domain/entities/insumo.entity';
import { StockInsumo } from '../../domain/entities/stock-insumo.entity';
import { IInventarioRepository } from '../../domain/repositories/inventario-repository.interface';

@Injectable()
export class TypeOrmInventarioRepository implements IInventarioRepository {
  constructor(
    @InjectRepository(Insumo) private readonly insumos: Repository<Insumo>,
    @InjectRepository(StockInsumo) private readonly stock: Repository<StockInsumo>,
  ) {}

  findInsumo(id: string) {
    return this.insumos.findOneBy({ id });
  }

  crearInsumo(datos: { nombre: string; unidad: string; umbralMinimo: number; precioUnitario: string }) {
    return this.insumos.save(this.insumos.create(datos));
  }

  async ingresarStock(insumoId: string, cantidad: number, ubicacion?: string) {
    await this.stock.save(
      this.stock.create({ insumo: { id: insumoId }, cantidad, ubicacion: ubicacion ?? null }),
    );
  }

  async descontar(insumoId: string, cantidad: number) {
    const filas = await this.stock.find({
      where: { insumo: { id: insumoId } },
      order: { cantidad: 'DESC' },
    });

    for (const fila of filas) {
      const r = await this.stock
        .createQueryBuilder()
        .update(StockInsumo)
        .set({ cantidad: () => 'cantidad - :n' })
        .where('id = :id AND cantidad >= :n', { id: fila.id, n: cantidad })
        .execute();
      if (r.affected === 1) return true;   // si otro se llevó el stock, prueba con la siguiente ubicación
    }
    return false;
  }

  async stockTotal(insumoId: string) {
    const fila = await this.stock
      .createQueryBuilder('s')
      .select('COALESCE(SUM(s.cantidad), 0)', 'total')
      .where('s."insumoId" = :insumoId', { insumoId })
      .getRawOne<{ total: string }>();
    return Number(fila?.total ?? 0);
  }
}
```

`application/descontar-insumo.use-case.ts`. Este caso de uso **se exporta del módulo** para que Atención y Enfermería lo use al administrar medicación:

```
import { Inject, Injectable } from '@nestjs/common';
import { EVENTOS, StockBajoUmbralPayload } from '../../shared/events/eventos';
```

```
import { EVENT_PUBLISHER, IEventPublisher } from '../../shared/events/event-publisher.interface';
import {
  DomainException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import {
  IInventarioRepository,
  INVENTARIO_REPOSITORY,
} from '../domain/repositories/inventario-repository.interface';

export interface InsumoDescontado {
  nombre: string;
  precioUnitario: string;
}

@Injectable()
export class DescontarInsumoUseCase {
  constructor(
    @Inject(INVENTARIO_REPOSITORY) private readonly inventario: IInventarioRepository,
    @Inject(EVENT_PUBLISHER) private readonly eventos: IEventPublisher,
  ) {}

  async ejecutar(insumoId: string, cantidad: number): Promise<InsumoDescontado> {
    if (!Number.isInteger(cantidad) || cantidad <= 0) {
      throw new DomainException('La cantidad debe ser un entero positivo');
    }

    const insumo = await this.inventario.findInsumo(insumoId);
    if (!insumo) throw new RecursoNoEncontradoException('Insumo');

    if (!(await this.inventario.descontar(insumoId, cantidad))) {
      throw new DomainException(`Stock insuficiente de ${insumo.nombre}`);
    }

    const stockTotal = await this.inventario.stockTotal(insumoId);
    if (stockTotal < insumo.umbralMinimo) {
      const payload: StockBajoUmbralPayload = {
        insumoId,
        nombre: insumo.nombre,
        stockTotal,
        umbralMinimo: insumo.umbralMinimo,
      };
      this.eventos.publicar(EVENTOS.STOCK_BAJO_UMBRAL, payload);
    }

    return { nombre: insumo.nombre, precioUnitario: insumo.precioUnitario };
  }
}
```

`CrearInsumoUseCase` y `IngresarStockUseCase` son delgados: validan datos (`cantidad > 0`, insumo existente) y llaman al repositorio. Un listener opcional, solo para ver la alerta en consola, queda así:

```
// application/alerta-stock.listener.ts
@Injectable()
export class AlertaStockListener {
  private readonly logger = new Logger(AlertaStockListener.name);

  @OnEvent(EVENTOS.STOCK_BAJO_UMBRAL)
  manejar(p: StockBajoUmbralPayload) {
    this.logger.warn(`Stock bajo: ${p.nombre} (${p.stockTotal} < mínimo ${p.umbralMinimo})`);
  }
}
```

Paso 5. Controllers y módulo

`presentation/gestion-camas.controller.ts`:

```
import { Body, Controller, Get, Param, ParseUUIDPipe, Patch, Post, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { IsNotEmpty, IsString, IsUUID } from 'class-validator';
import { RolUsuario } from '../../iam/domain/enums/rol.enum';
import { Roles } from '../../shared/decorators/roles.decorator';
import { JwtAuthGuard } from '../../shared/guards/jwt-auth.guard';
import { RolesGuard } from '../../shared/guards/roles.guard';
import { AsignarCamaUseCase } from '../application/asignar-cama.use-case';
import { BloquearCamaUseCase } from '../application/bloquear-cama.use-case';
import { ConfirmarLimpiezaUseCase } from '../application/confirmar-limpieza.use-case';
import { CrearCamaUseCase } from '../application/crear-cama.use-case';
import { HabilitarCamaUseCase } from '../application/habilitar-cama.use-case';
import { LiberarCamaUseCase } from '../application/liberar-cama.use-case';
import { ObtenerMapaCamasUseCase } from '../application/obtener-mapa-camas.use-case';

class AsignarCamaDto {
  @IsUUID() pacienteId: string;
}
```

```
class CrearCamaDto {
  @IsString() @IsNotEmpty() codigo: string;
  @IsString() @IsNotEmpty() sector: string;
}

@ApiTags('camas')
@ApiBearerAuth()
@UseGuards(JwtAuthGuard, RolesGuard)
@Controller('camas')
export class GestionCamasController {
  constructor(
    private readonly mapa: ObtenerMapaCamasUseCase,
    private readonly crear: CrearCamaUseCase,
    private readonly asignar: AsignarCamaUseCase,
    private readonly liberar: LiberarCamaUseCase,
    private readonly limpieza: ConfirmarLimpiezaUseCase,
    private readonly bloquear: BloquearCamaUseCase,
    private readonly habilitar: HabilitarCamaUseCase,
  ) {}

  @Get('mapa')
  @Roles(RolUsuario.ADMIN, RolUsuario.RECEPCIONISTA, RolUsuario.MEDICO, RolUsuario.ENFERMERO)
  verMapa() {
    return this.mapa.ejecutar();
  }

  @Post()
  @Roles(RolUsuario.ADMIN)
  crearCama(@Body() dto: CrearCamaDto) {
    return this.crear.ejecutar(dto.codigo, dto.sector);
  }

  @Post(':id/asignar')
  @Roles(RolUsuario.ADMIN, RolUsuario.RECEPCIONISTA)
  asignarCama(@Param('id', ParseUUIDPipe) id: string, @Body() dto: AsignarCamaDto) {
    return this.asignar.ejecutar(id, dto.pacienteId);
  }

  @Patch(':id/liberar')
  @Roles(RolUsuario.ADMIN, RolUsuario.MEDICO, RolUsuario.ENFERMERO)
  liberarCama(@Param('id', ParseUUIDPipe) id: string) {
    return this.liberar.ejecutar(id);
  }

  @Patch(':id/limpieza-confirmada')
  @Roles(RolUsuario.ADMIN, RolUsuario.ENFERMERO)
  confirmarLimpieza(@Param('id', ParseUUIDPipe) id: string) {
    return this.limpieza.ejecutar(id);
  }

  @Patch(':id/bloquear')
  @Roles(RolUsuario.ADMIN)
  bloquearCama(@Param('id', ParseUUIDPipe) id: string) {
    return this.bloquear.ejecutar(id);
  }

  @Patch(':id/habilitar')
  @Roles(RolUsuario.ADMIN)
  habilitarCama(@Param('id', ParseUUIDPipe) id: string) {
    return this.habilitar.ejecutar(id);
  }
}
```

`InventarioController` (`/camas/insumos`): `POST` crea insumo (Admin), `POST :id/stock` ingresa stock (Admin, Enfermero), `GET` lista insumos con su stock total. Las clases DTO de esa carpeta usan `@IsInt() @Min(1)` para `cantidad`.

`camas/camas.module.ts`:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AdmisionesModule } from '../admisiones/admisiones.module';
import { AlertaStockListener } from './application/alerta-stock.listener';
import { AsignarCamaUseCase } from './application/asignar-cama.use-case';
import { BloquearCamaUseCase } from './application/bloquear-cama.use-case';
import { ConfirmarLimpiezaUseCase } from './application/confirmar-limpieza.use-case';
import { CrearCamaUseCase } from './application/crear-cama.use-case';
import { CrearInsumoUseCase } from './application/crear-insumo.use-case';
import { DescontarInsumoUseCase } from './application/descontar-insumo.use-case';
import { HabilitarCamaUseCase } from './application/habilitar-cama.use-case';
import { IngresarStockUseCase } from './application/ingresar-stock.use-case';
import { LiberarCamaUseCase } from './application/liberar-cama.use-case';
import { ObtenerMapaCamasUseCase } from './application/obtener-mapa-camas.use-case';
import { entidadesCamas } from './domain/entities';
import { CAMA_REPOSITORY } from './domain/repositories/cama-repository.interface';
import { INVENTARIO_REPOSITORY } from './domain/repositories/inventario-repository.interface';
import { TypeOrmCamaRepository } from './infrastructure/repositories/typeorm-cama.repository';
import { TypeOrmInventarioRepository } from './infrastructure/repositories/typeorm-inventario.repository';
import { CamasController } from './presentation/camas.controller';
```

```
import { GestionCamasController } from './presentation/gestion-camas.controller';
import { InventarioController } from './presentation/inventario.controller';

@Module({
  imports: [TypeOrmModule.forFeature(entidadesCamas), AdmisionesModule],
  controllers: [CamasController, GestionCamasController, InventarioController],
  providers: [
    { provide: CAMA_REPOSITORY, useClass: TypeOrmCamaRepository },
    { provide: INVENTARIO_REPOSITORY, useClass: TypeOrmInventarioRepository },
    AsignarCamaUseCase,
    LiberarCamaUseCase,
    ConfirmarLimpiezaUseCase,
    BloquearCamaUseCase,
    HabilitarCamaUseCase,
    CrearCamaUseCase,
    ObtenerMapaCamasUseCase,
    CrearInsumoUseCase,
    IngresarStockUseCase,
    DescontarInsumoUseCase,
    AlertaStockListener,
  ],
  // La API pública del módulo: lo que Atención y Enfermería puede usar.
  exports: [DescontarInsumoUseCase],
})
export class CamasModule {}
```

Endpoints

MÉTODO	RUTA	ROLES	RESPUESTAS
GET	`/camas/mapa`	Personal	200, camas agrupadas por sector
POST	`/camas`	Admin	201, 422 código repetido
POST	`/camas/:id/asignar`	Admin, Recepcionista	201, **409 si otra persona la ocupó**, 422 si no está libre
PATCH	`/camas/:id/liberar`	Admin, Médico, Enfermero	200, 409, 422
PATCH	`/camas/:id/limpieza-confirmada`	Admin, Enfermero	200, 409, 422
PATCH	`/camas/:id/bloquear`	Admin	200, 409, 422
PATCH	`/camas/:id/habilitar`	Admin	200, 409, 422
POST	`/camas/insumos`	Admin	201
POST	`/camas/insumos/:id/stock`	Admin, Enfermero	201
GET	`/camas/insumos`	Personal	200 con stock total

Tests

`domain/state/cama-estado.spec.ts` :

```
import { TransicionInvalidaException } from '../../../shared/exceptions/domain.exception';
import { siguienteEstadoCama } from '../cama-workflow';
import { EstadoCama } from './cama-estado';

describe('Máquina de estados de la cama', () => {
  it('recorre el ciclo completo: libre -> ocupada -> en limpieza -> libre', () => {
    expect(siguienteEstadoCama('libre', 'asignar')).toBe(EstadoCama.OCUPADA);
    expect(siguienteEstadoCama('ocupada', 'liberar')).toBe(EstadoCama.EN_LIMPIEZA);
    expect(siguienteEstadoCama('en_limpieza', 'confirmarLimpieza')).toBe(EstadoCama.LIBRE);
  });

  it.each([
    ['ocupada', 'confirmarLimpieza'],   // no salta de ocupada a libre sin limpieza
    ['ocupada', 'bloquear'],
    ['ocupada', 'asignar'],
    ['en_limpieza', 'asignar'],
```

```
    ['bloqueada', 'asignar'],
    ['libre', 'liberar'],
  ] as const)('rechaza %s -> %s', (estado, accion) => {
    expect(() => siguienteEstadoCama(estado, accion)).toThrow(TransicionInvalidaException);
  });
});
```

`application/descontar-insumo.use-case.spec.ts`: sin sobreventa y alerta de stock bajo:

```
import { DomainException } from '../../shared/exceptions/domain.exception';
import { EVENTOS } from '../../shared/events/eventos';
import { Insumo } from '../domain/entities/insumo.entity';
import { IInventarioRepository } from '../domain/repositories/inventario-repository.interface';
import { DescontarInsumoUseCase } from './descontar-insumo.use-case';

class InventarioEnMemoria implements IInventarioRepository {
  constructor(private stock: number, private readonly insumo: Insumo) {}
  async findInsumo() { return this.insumo; }
  async crearInsumo() { return this.insumo; }
  async ingresarStock() {}
  async descontar(_id: string, n: number) {
    if (this.stock < n) return false;
    this.stock -= n;
    return true;
  }
  async stockTotal() { return this.stock; }
}

describe('DescontarInsumoUseCase', () => {
  const insumo = Object.assign(new Insumo(), {
    id: 'i1', nombre: 'Ampolla paracetamol', umbralMinimo: 5, precioUnitario: '120.00',
  });

  it('no permite descontar más de lo que hay', async () => {
    const publicados: string[] = [];
    const useCase = new DescontarInsumoUseCase(new InventarioEnMemoria(3, insumo), {
      publicar: (e) => { publicados.push(e); },
    });

    await expect(useCase.ejecutar('i1', 10)).rejects.toThrow('Stock insuficiente');
    expect(publicados).toEqual([]);
  });

  it('publica una alerta cuando el stock cae bajo el umbral', async () => {
    const publicados: string[] = [];
    const useCase = new DescontarInsumoUseCase(new InventarioEnMemoria(8, insumo), {
      publicar: (e) => { publicados.push(e); },
    });

    const r = await useCase.ejecutar('i1', 4);   // quedan 4, el mínimo es 5

    expect(r).toEqual({ nombre: 'Ampolla paracetamol', precioUnitario: '120.00' });
    expect(publicados).toEqual([EVENTOS.STOCK_BAJO_UMBRAL]);
  });

  it('rechaza cantidades inválidas', async () => {
    const useCase = new DescontarInsumoUseCase(new InventarioEnMemoria(8, insumo), { publicar: () => {} });
    await expect(useCase.ejecutar('i1', 0)).rejects.toBeInstanceOf(DomainException);
    await expect(useCase.ejecutar('i1', 1.5)).rejects.toBeInstanceOf(DomainException);
  });
});
```

Terminado cuando

- [ ] Crear dos camas y un paciente. Asignar la misma cama dos veces seguidas: la primera devuelve 201 y la segunda 422 (ya está ocupada).
- [ ] Liberar la cama la deja `en_limpieza` y cierra la asignación (`hasta` con fecha). Recién después de confirmar la limpieza vuelve a `libre`.
- [ ] `GET /camas/mapa` muestra las camas agrupadas por sector con su estado.
- [ ] Cargar 10 unidades de un insumo y descontar 11 devuelve 422 y el stock sigue en 10.
- [ ] Al bajar del umbral aparece el aviso `Stock bajo` en la consola del backend.
- [ ] Los tests nuevos pasan y los anteriores siguen en verde.

Historia Clínica Electrónica

Objetivo. Una ficha por paciente con sus alergias, antecedentes familiares y evoluciones médicas. Es sobre todo un módulo de datos con reglas de integridad: una sola historia por paciente, evoluciones que no se editan y permisos distintos para leer y para escribir.

Qué hay hoy

YA EXISTE	FALTA
Entidades `HistoriaClinica`, `Alergia`, `AntecedenteFamiliar`, `Evolucion`	Repositorio y casos de uso
`HistoriaClinicaController` con `GET /historia-clinica/ping`	Endpoints para abrir, consultar y escribir
	Permisos por rol

Antes de empezar: Admisiones terminado (se valida que el paciente exista y se resuelve qué médico firma cada evolución) y la columna `Medico.usuarioId` (ya está en la entidad).

Reglas de negocio

1. Cada paciente tiene **una sola** historia clínica (`pacienteId` es único en la tabla).
2. Las evoluciones **no se editan ni se borran**: una nota médica firmada es un registro legal. Solo existe `POST`.
3. Solo un **médico** escribe evoluciones. El `medicoId` sale del usuario logueado, nunca del cuerpo de la petición.
4. Alergias y antecedentes los pueden cargar médicos y enfermeros.
5. Leer la historia: médicos, enfermeros y admin.

Estructura final

```
historia-clinica/
├── domain/
│   ├── entities/                          (existen)
│   ├── enums/severidad-alergia.enum.ts    NUEVO
│   └── repositories/historia-clinica-repository.interface.ts   NUEVO
├── application/                           NUEVO
├── infrastructure/repositories/typeorm-historia-clinica.repository.ts   NUEVO
├── presentation/
│   ├── historia-clinica.controller.ts     (existe: ping)
│   ├── historias.controller.ts            NUEVO
│   └── dto/                               NUEVO
└── historia-clinica.module.ts             (se modifica)
```

Paso 1. Enum y repositorio

`domain/enums/severidad-alergia.enum.ts`:

```ts
export enum SeveridadAlergia {
  LEVE = 'leve',
  MODERADA = 'moderada',
  GRAVE = 'grave',
}
```

`domain/repositories/historia-clinica-repository.interface.ts`:

```ts
import { Alergia } from '../entities/alergia.entity';
import { AntecedenteFamiliar } from '../entities/antecedente-familiar.entity';
import { Evolucion } from '../entities/evolucion.entity';
import { HistoriaClinica } from '../entities/historia-clinica.entity';

export interface IHistoriaClinicaRepository {
```

```
  findByPaciente(pacienteId: string): Promise<HistoriaClinica | null>;
  /** Lanza DomainException si el paciente ya tenía historia. */
  crear(pacienteId: string, grupoSanguineo: string | null): Promise<HistoriaClinica>;

  agregarAlergia(historiaId: string, sustancia: string, severidad: string): Promise<Alergia>;
  agregarAntecedente(historiaId: string, parentesco: string, descripcion: string): Promise<AntecedenteFamiliar>;
  agregarEvolucion(historiaId: string, medicoId: string, nota: string): Promise<Evolucion>;

  listarAlergias(historiaId: string): Promise<Alergia[]>;
  listarAntecedentes(historiaId: string): Promise<AntecedenteFamiliar[]>;
  /** Más recientes primero. */
  listarEvoluciones(historiaId: string, limite: number): Promise<Evolucion[]>;
}

export const HISTORIA_CLINICA_REPOSITORY = Symbol('HISTORIA_CLINICA_REPOSITORY');
```

Paso 2. Implementación TypeORM

Un solo repositorio de dominio se apoya en cuatro repositorios de TypeORM, uno por tabla. El módulo ya registra las cuatro entidades en `forFeature`.

`infrastructure/repositories/typeorm-historia-clinica.repository.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { esViolacionDeUnicidad } from '../../../shared/database/errores-pg';
import { DomainException } from '../../../shared/exceptions/domain.exception';
import { Alergia } from '../../domain/entities/alergia.entity';
import { AntecedenteFamiliar } from '../../domain/entities/antecedente-familiar.entity';
import { Evolucion } from '../../domain/entities/evolucion.entity';
import { HistoriaClinica } from '../../domain/entities/historia-clinica.entity';
import { IHistoriaClinicaRepository } from '../../domain/repositories/historia-clinica-repository.interface';

@Injectable()
export class TypeOrmHistoriaClinicaRepository implements IHistoriaClinicaRepository {
  constructor(
    @InjectRepository(HistoriaClinica) private readonly historias: Repository<HistoriaClinica>,
    @InjectRepository(Alergia) private readonly alergias: Repository<Alergia>,
    @InjectRepository(AntecedenteFamiliar) private readonly antecedentes: Repository<AntecedenteFamiliar>,
    @InjectRepository(Evolucion) private readonly evoluciones: Repository<Evolucion>,
  ) {}

  findByPaciente(pacienteId: string) {
    return this.historias.findOneBy({ pacienteId });
  }

  async crear(pacienteId: string, grupoSanguineo: string | null) {
    try {
      return await this.historias.save(this.historias.create({ pacienteId, grupoSanguineo }));
    } catch (error) {
      // Dos aperturas simultáneas: la base (pacienteId unique) deja pasar solo una.
      if (esViolacionDeUnicidad(error)) throw new DomainException('El paciente ya tiene historia clínica');
      throw error;
    }
  }

  agregarAlergia(historiaId: string, sustancia: string, severidad: string) {
    return this.alergias.save(
      this.alergias.create({ historiaClinica: { id: historiaId }, sustancia, severidad }),
    );
  }

  agregarAntecedente(historiaId: string, parentesco: string, descripcion: string) {
    return this.antecedentes.save(
      this.antecedentes.create({ historiaClinica: { id: historiaId }, parentesco, descripcion }),
    );
  }

  agregarEvolucion(historiaId: string, medicoId: string, nota: string) {
    return this.evoluciones.save(
      this.evoluciones.create({ historiaClinica: { id: historiaId }, medicoId, nota }),
    );
  }

  listarAlergias(historiaId: string) {
    return this.alergias.find({ where: { historiaClinica: { id: historiaId } } });
  }

  listarAntecedentes(historiaId: string) {
    return this.antecedentes.find({ where: { historiaClinica: { id: historiaId } } });
  }

  listarEvoluciones(historiaId: string, limite: number) {
    return this.evoluciones.find({
      where: { historiaClinica: { id: historiaId } },
      order: { fecha: 'DESC' },
      take: limite,
```

```
    });
  }
}
```

Paso 3. Casos de uso

Los dos primeros comparten una búsqueda: "dame la historia de este paciente o falla con 404". Se aísla en una función para no repetirla.

`application/historia-requerida.ts`:

```
import { RecursoNoEncontradoException } from '../../shared/exceptions/domain.exception';
import { HistoriaClinica } from '../domain/entities/historia-clinica.entity';
import { IHistoriaClinicaRepository } from '../domain/repositories/historia-clinica-repository.interface';

export async function historiaRequerida(
  repo: IHistoriaClinicaRepository,
  pacienteId: string,
): Promise<HistoriaClinica> {
  const historia = await repo.findByPaciente(pacienteId);
  if (!historia) throw new RecursoNoEncontradoException('Historia clínica');
  return historia;
}
```

`application/abrir-historia-clinica.use-case.ts`. Es idempotente: si ya existe, devuelve la existente.

```
import { Inject, Injectable } from '@nestjs/common';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../../admisiones/domain/repositories/paciente-repository.interface';
import { RecursoNoEncontradoException } from '../../shared/exceptions/domain.exception';
import { HistoriaClinica } from '../domain/entities/historia-clinica.entity';
import {
  HISTORIA_CLINICA_REPOSITORY,
  IHistoriaClinicaRepository,
} from '../domain/repositories/historia-clinica-repository.interface';

@Injectable()
export class AbrirHistoriaClinicaUseCase {
  constructor(
    @Inject(HISTORIA_CLINICA_REPOSITORY) private readonly historias: IHistoriaClinicaRepository,
    @Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository,
  ) {}

  async ejecutar(pacienteId: string, grupoSanguineo?: string): Promise<HistoriaClinica> {
    if (!(await this.pacientes.findById(pacienteId))) {
      throw new RecursoNoEncontradoException('Paciente');
    }

    const existente = await this.historias.findByPaciente(pacienteId);
    if (existente) return existente;

    return this.historias.crear(pacienteId, grupoSanguineo ?? null);
  }
}
```

`application/registrar-alergia.use-case.ts` (`RegistrarAntecedenteUseCase` es idéntico, con `agregarAntecedente`):

```
import { Inject, Injectable } from '@nestjs/common';
import { Alergia } from '../domain/entities/alergia.entity';
import {
  HISTORIA_CLINICA_REPOSITORY,
  IHistoriaClinicaRepository,
} from '../domain/repositories/historia-clinica-repository.interface';
import { historiaRequerida } from './historia-requerida';

@Injectable()
export class RegistrarAlergiaUseCase {
  constructor(
    @Inject(HISTORIA_CLINICA_REPOSITORY) private readonly historias: IHistoriaClinicaRepository,
  ) {}

  async ejecutar(pacienteId: string, sustancia: string, severidad: string): Promise<Alergia> {
    const historia = await historiaRequerida(this.historias, pacienteId);
    return this.historias.agregarAlergia(historia.id, sustancia.trim(), severidad);
  }
}
```

`application/agregar-evolucion.use-case.ts`. El médico se identifica por el usuario logueado:

```ts
import { Inject, Injectable } from '@nestjs/common';
import {
  IMedicoRepository,
  MEDICO_REPOSITORY,
} from '../../admisiones/domain/repositories/medico-repository.interface';
import {
  DomainException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { Evolucion } from '../domain/entities/evolucion.entity';
import {
  HISTORIA_CLINICA_REPOSITORY,
  IHistoriaClinicaRepository,
} from '../domain/repositories/historia-clinica-repository.interface';
import { historiaRequerida } from './historia-requerida';

@Injectable()
export class AgregarEvolucionUseCase {
  constructor(
    @Inject(HISTORIA_CLINICA_REPOSITORY) private readonly historias: IHistoriaClinicaRepository,
    @Inject(MEDICO_REPOSITORY) private readonly medicos: IMedicoRepository,
  ) {}

  async ejecutar(pacienteId: string, usuarioId: string, nota: string): Promise<Evolucion> {
    if (nota.trim().length === 0) throw new DomainException('La evolución no puede estar vacía');

    const medico = await this.medicos.findByUsuarioId(usuarioId);
    if (!medico) throw new RecursoNoEncontradoException('Médico asociado al usuario');

    const historia = await historiaRequerida(this.historias, pacienteId);
    return this.historias.agregarEvolucion(historia.id, medico.id, nota.trim());
  }
}
```

`application/obtener-historia-completa.use-case.ts`:

```ts
import { Inject, Injectable } from '@nestjs/common';
import { Alergia } from '../domain/entities/alergia.entity';
import { AntecedenteFamiliar } from '../domain/entities/antecedente-familiar.entity';
import { Evolucion } from '../domain/entities/evolucion.entity';
import { HistoriaClinica } from '../domain/entities/historia-clinica.entity';
import {
  HISTORIA_CLINICA_REPOSITORY,
  IHistoriaClinicaRepository,
} from '../domain/repositories/historia-clinica-repository.interface';
import { historiaRequerida } from './historia-requerida';

export interface HistoriaCompleta {
  historia: HistoriaClinica;
  alergias: Alergia[];
  antecedentes: AntecedenteFamiliar[];
  evoluciones: Evolucion[];
}

@Injectable()
export class ObtenerHistoriaCompletaUseCase {
  constructor(
    @Inject(HISTORIA_CLINICA_REPOSITORY) private readonly historias: IHistoriaClinicaRepository,
  ) {}

  async ejecutar(pacienteId: string, limiteEvoluciones = 50): Promise<HistoriaCompleta> {
    const historia = await historiaRequerida(this.historias, pacienteId);

    const [alergias, antecedentes, evoluciones] = await Promise.all([
      this.historias.listarAlergias(historia.id),
      this.historias.listarAntecedentes(historia.id),
      this.historias.listarEvoluciones(historia.id, limiteEvoluciones),
    ]);

    return { historia, alergias, antecedentes, evoluciones };
  }
}
```

Paso 4. DTOs

`presentation/dto/historia.dto.ts`:

```ts
import { ApiProperty, ApiPropertyOptional } from '@nestjs/swagger';
import { IsEnum, IsNotEmpty, IsOptional, IsString, MaxLength } from 'class-validator';
import { SeveridadAlergia } from '../../domain/enums/severidad-alergia.enum';

export class AbrirHistoriaDto {
  @ApiPropertyOptional({ example: 'A+' })
  @IsOptional() @IsString() @MaxLength(5)
```

```
  grupoSanguineo?: string;
}

export class RegistrarAlergiaDto {
  @ApiProperty() @IsString() @IsNotEmpty() sustancia: string;
  @ApiProperty({ enum: SeveridadAlergia }) @IsEnum(SeveridadAlergia) severidad: SeveridadAlergia;
}

export class RegistrarAntecedenteDto {
  @ApiProperty({ example: 'madre' }) @IsString() @IsNotEmpty() parentesco: string;
  @ApiProperty() @IsString() @IsNotEmpty() descripcion: string;
}

export class AgregarEvolucionDto {
  @ApiProperty() @IsString() @IsNotEmpty() nota: string;
}
```

Paso 5. Controller

Todas las rutas cuelgan del paciente (`/historia-clinica/paciente/:pacienteId/...`). Fijarse en que **no hay `PATCH` ni `DELETE` de evoluciones**: es una decisión de diseño, no un olvido.

`presentation/historias.controller.ts`:

```
import { Body, Controller, Get, Param, ParseUUIDPipe, Post, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { RolUsuario } from '../../iam/domain/enums/rol.enum';
import { Roles } from '../../shared/decorators/roles.decorator';
import { UsuarioActual, UsuarioAutenticado } from '../../shared/decorators/usuario-actual.decorator';
import { JwtAuthGuard } from '../../shared/guards/jwt-auth.guard';
import { RolesGuard } from '../../shared/guards/roles.guard';
import { AbrirHistoriaClinicaUseCase } from '../application/abrir-historia-clinica.use-case';
import { AgregarEvolucionUseCase } from '../application/agregar-evolucion.use-case';
import { ObtenerHistoriaCompletaUseCase } from '../application/obtener-historia-completa.use-case';
import { RegistrarAlergiaUseCase } from '../application/registrar-alergia.use-case';
import { RegistrarAntecedenteUseCase } from '../application/registrar-antecedente.use-case';
import {
  AbrirHistoriaDto,
  AgregarEvolucionDto,
  RegistrarAlergiaDto,
  RegistrarAntecedenteDto,
} from './dto/historia.dto';

@ApiTags('historia-clinica')
@ApiBearerAuth()
@UseGuards(JwtAuthGuard, RolesGuard)
@Controller('historia-clinica/paciente/:pacienteId')
export class HistoriasController {
  constructor(
    private readonly abrir: AbrirHistoriaClinicaUseCase,
    private readonly obtener: ObtenerHistoriaCompletaUseCase,
    private readonly alergia: RegistrarAlergiaUseCase,
    private readonly antecedente: RegistrarAntecedenteUseCase,
    private readonly evolucion: AgregarEvolucionUseCase,
  ) {}

  @Post()
  @Roles(RolUsuario.MEDICO, RolUsuario.ADMIN, RolUsuario.RECEPCIONISTA)
  abrirHistoria(@Param('pacienteId', ParseUUIDPipe) pacienteId: string, @Body() dto: AbrirHistoriaDto) {
    return this.abrir.ejecutar(pacienteId, dto.grupoSanguineo);
  }

  @Get()
  @Roles(RolUsuario.MEDICO, RolUsuario.ENFERMERO, RolUsuario.ADMIN)
  verHistoria(@Param('pacienteId', ParseUUIDPipe) pacienteId: string) {
    return this.obtener.ejecutar(pacienteId);
  }

  @Post('alergias')
  @Roles(RolUsuario.MEDICO, RolUsuario.ENFERMERO)
  agregarAlergia(@Param('pacienteId', ParseUUIDPipe) pacienteId: string, @Body() dto: RegistrarAlergiaDto) {
    return this.alergia.ejecutar(pacienteId, dto.sustancia, dto.severidad);
  }

  @Post('antecedentes')
  @Roles(RolUsuario.MEDICO, RolUsuario.ENFERMERO)
  agregarAntecedente(@Param('pacienteId', ParseUUIDPipe) pacienteId: string, @Body() dto: RegistrarAntecedenteDto) {
    return this.antecedente.ejecutar(pacienteId, dto.parentesco, dto.descripcion);
  }

  @Post('evoluciones')
  @Roles(RolUsuario.MEDICO)
  agregarEvolucion(
    @Param('pacienteId', ParseUUIDPipe) pacienteId: string,
    @Body() dto: AgregarEvolucionDto,
    @UsuarioActual() usuario: UsuarioAutenticado,
  ) {
    return this.evolucion.ejecutar(pacienteId, usuario.id, dto.nota);
```

```
  }
}
```

Paso 6. Módulo

`historia-clinica/historia-clinica.module.ts`:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AdmisionesModule } from '../admisiones/admisiones.module';
import { AbrirHistoriaClinicaUseCase } from './application/abrir-historia-clinica.use-case';
import { AgregarEvolucionUseCase } from './application/agregar-evolucion.use-case';
import { ObtenerHistoriaCompletaUseCase } from './application/obtener-historia-completa.use-case';
import { RegistrarAlergiaUseCase } from './application/registrar-alergia.use-case';
import { RegistrarAntecedenteUseCase } from './application/registrar-antecedente.use-case';
import { entidadesHistoriaClinica } from './domain/entities';
import { HISTORIA_CLINICA_REPOSITORY } from './domain/repositories/historia-clinica-repository.interface';
import { TypeOrmHistoriaClinicaRepository } from './infrastructure/repositories/typeorm-historia-clinica.repository';
import { HistoriaClinicaController } from './presentation/historia-clinica.controller';
import { HistoriasController } from './presentation/historias.controller';

@Module({
  imports: [TypeOrmModule.forFeature(entidadesHistoriaClinica), AdmisionesModule],
  controllers: [HistoriaClinicaController, HistoriasController],
  providers: [
    { provide: HISTORIA_CLINICA_REPOSITORY, useClass: TypeOrmHistoriaClinicaRepository },
    AbrirHistoriaClinicaUseCase,
    RegistrarAlergiaUseCase,
    RegistrarAntecedenteUseCase,
    AgregarEvolucionUseCase,
    ObtenerHistoriaCompletaUseCase,
  ],
  exports: [HISTORIA_CLINICA_REPOSITORY],
})
export class HistoriaClinicaModule {}
```

Endpoints

MÉTODO	RUTA	ROLES	RESPUESTAS
POST	`/historia-clinica/paciente/:id`	Médico, Admin, Recepcionista	201 (o la existente), 404 paciente
GET	`/historia-clinica/paciente/:id`	Médico, Enfermero, Admin	200 con alergias, antecedentes y evoluciones; 404 sin historia
POST	`/historia-clinica/paciente/:id/alergias`	Médico, Enfermero	201
POST	`/historia-clinica/paciente/:id/antecedentes`	Médico, Enfermero	201
POST	`/historia-clinica/paciente/:id/evoluciones`	Médico	201, 404 si el usuario no es un médico registrado

Tests

`application/agregar-evolucion.use-case.spec.ts`:

```
import { Medico } from '../../admisiones/domain/entities/medico.entity';
import { Evolucion } from '../domain/entities/evolucion.entity';
import { HistoriaClinica } from '../domain/entities/historia-clinica.entity';
import { IHistoriaClinicaRepository } from '../domain/repositories/historia-clinica-repository.interface';
import { AgregarEvolucionUseCase } from './agregar-evolucion.use-case';

function historiasEnMemoria(guardadas: Evolucion[]): IHistoriaClinicaRepository {
  const historia = Object.assign(new HistoriaClinica(), { id: 'h1', pacienteId: 'p1' });
  return {
    findByPaciente: async (id) => (id === 'p1' ? historia : null),
    agregarEvolucion: async (historiaId, medicoId, nota) => {
```

```
      const e = Object.assign(new Evolucion(), { id: `e${guardadas.length + 1}`, medicoId, nota });
      guardadas.push(e);
      return e;
    },
  } as unknown as IHistoriaClinicaRepository;
}

describe('AgregarEvolucionUseCase', () => {
  const medico = Object.assign(new Medico(), { id: 'm1', usuarioId: 'u1' });
  const medicos = { findByUsuarioId: async (u: string) => (u === 'u1' ? medico : null) } as never;

  it('firma la evolución con el médico del usuario logueado', async () => {
    const guardadas: Evolucion[] = [];
    const useCase = new AgregarEvolucionUseCase(historiasEnMemoria(guardadas), medicos);

    await useCase.ejecutar('p1', 'u1', '  Paciente estable.  ');

    expect(guardadas).toHaveLength(1);
    expect(guardadas[0]).toMatchObject({ medicoId: 'm1', nota: 'Paciente estable.' });
  });

  it('rechaza una nota vacía', async () => {
    const useCase = new AgregarEvolucionUseCase(historiasEnMemoria([]), medicos);
    await expect(useCase.ejecutar('p1', 'u1', '   ')).rejects.toThrow('vacía');
  });

  it('falla si el usuario no es un médico registrado', async () => {
    const useCase = new AgregarEvolucionUseCase(historiasEnMemoria([]), medicos);
    await expect(useCase.ejecutar('p1', 'otro-usuario', 'Nota')).rejects.toThrow('no encontrado');
  });

  it('falla con 404 si el paciente no tiene historia', async () => {
    const useCase = new AgregarEvolucionUseCase(historiasEnMemoria([]), medicos);
    await expect(useCase.ejecutar('sin-historia', 'u1', 'Nota')).rejects.toThrow('Historia clínica no encontrado');
  });
});
```

Terminado cuando

- [ ] Abrir la historia de un paciente dos veces devuelve la misma historia, sin error y sin duplicados.
- [ ] Cargar una alergia y un antecedente; `GET` los devuelve junto a las evoluciones, de la más nueva a la más vieja.
- [ ] Un enfermero que intenta `POST /evoluciones` recibe 403.
- [ ] No existe ninguna ruta para editar o borrar evoluciones.
- [ ] Los tests nuevos pasan y los anteriores siguen en verde.

Extensión opcional: el paciente ve su propia historia

Requiere la columna `Paciente.usuarioId` (ya está en la entidad). En el controller se agrega `RolUsuario.PACIENTE` al `GET`, y en el caso de uso se compara el paciente de la historia con el del usuario logueado:

```
const paciente = await this.pacientes.findByUsuarioId(usuario.id);
if (usuario.rol === RolUsuario.PACIENTE && paciente?.id !== pacienteId) {
  throw new DomainException('Solo podés ver tu propia historia clínica');
}
```

Para eso `IPacienteRepository` necesita el método `findByUsuarioId`.

Atención y Enfermería

Objetivo. Registrar signos vitales con alertas automáticas cuando un valor sale de rango, que los médicos carguen indicaciones, que enfermería las cumpla descontando el insumo del inventario, y armar protocolos de cuidado según el motivo de ingreso.

Patrones de este módulo: Builder (protocolos de cuidado, Épica 2), reglas de umbrales como lógica pura, eventos de dominio (`MedicacionAdministrada` hacia Facturación).

Qué hay hoy

YA EXISTE	FALTA
Entidades `SignoVital` , `IndicacionMedica` , `ProtocoloCuidado` , `TareaProtocolo`	Repositorios y casos de uso
`AtencionEnfermeriaController` con `GET /atencion-enfermeria/ping`	Evaluación de umbrales
	Builder y directores de protocolos
	Cumplimiento de indicaciones con descuento de stock y evento

Antes de empezar: Admisiones y Camas terminados, y las columnas de `IndicacionMedica` (`insumoId` , `cantidad`), que ya están en la entidad. Este módulo importa `AdmisionesModule` (para validar pacientes y médicos) y `CamasModule` (para descontar insumos).

Estructura final

```
atencion-enfermeria/
├── domain/
│   ├── entities/                              (existen)
│   ├── umbrales-signos-vitales.ts             NUEVO  (lógica pura)
│   ├── protocolos/
│   │   ├── protocolo.builder.ts               NUEVO  (Builder)
│   │   └── protocolo.director.ts              NUEVO  (recetas)
│   └── repositories/
│       ├── signo-vital-repository.interface.ts  NUEVO
│       ├── indicacion-repository.interface.ts   NUEVO
│       └── protocolo-repository.interface.ts    NUEVO
├── application/                               NUEVO
├── infrastructure/repositories/               NUEVO
├── presentation/
│   ├── atencion-enfermeria.controller.ts      (existe: ping)
│   ├── signos-vitales.controller.ts           NUEVO
│   ├── indicaciones.controller.ts             NUEVO
│   ├── protocolos.controller.ts               NUEVO
│   └── dto/                                   NUEVO
└── atencion-enfermeria.module.ts              (se modifica)
```

Paso 1. Signos vitales con alertas por umbral

La regla. Cada tipo de signo vital tiene un rango normal y un rango crítico. Fuera del normal es una alerta; fuera del crítico, una alerta crítica. La regla es una función pura: se prueba sin Nest ni base de datos.

`domain/umbrales-signos-vitales.ts` :

```ts
import { DomainException } from '../../shared/exceptions/domain.exception';

export type Severidad = 'normal' | 'alerta' | 'critico';

interface Rango {
  unidad: string;
  normalMin: number;
  normalMax: number;
```

```
  criticoMin: number;
  criticoMax: number;
}

/**
 * Valores de referencia para adultos, pensados para el trabajo práctico.
 * Hay que validarlos con la cátedra; no son criterio clínico.
 */
export const UMBRALES: Record<string, Rango> = {
  frecuencia_cardiaca:     { unidad: 'lpm',  normalMin: 60, normalMax: 100, criticoMin: 40, criticoMax: 130 },
  presion_sistolica:       { unidad: 'mmHg', normalMin: 90, normalMax: 140, criticoMin: 70, criticoMax: 180 },
  temperatura:             { unidad: '°C',   normalMin: 36, normalMax: 37.5, criticoMin: 35, criticoMax: 39.5 },
  saturacion_o2:           { unidad: '%',    normalMin: 95, normalMax: 100, criticoMin: 90, criticoMax: 100 },
  frecuencia_respiratoria: { unidad: 'rpm',  normalMin: 12, normalMax: 20, criticoMin: 8,  criticoMax: 30 },
};

export const TIPOS_SIGNO_VITAL = Object.keys(UMBRALES);

export function unidadDe(tipo: string): string {
  return rangoDe(tipo).unidad;
}

export function evaluarSignoVital(tipo: string, valor: number): Severidad {
  const r = rangoDe(tipo);
  if (valor < r.criticoMin || valor > r.criticoMax) return 'critico';
  if (valor < r.normalMin || valor > r.normalMax) return 'alerta';
  return 'normal';
}

function rangoDe(tipo: string): Rango {
  const rango = UMBRALES[tipo];
  if (!rango) throw new DomainException(`Tipo de signo vital desconocido: "${tipo}"`);
  return rango;
}
```

`domain/repositories/signo-vital-repository.interface.ts`:

```
import { SignoVital } from '../entities/signo-vital.entity';

export interface NuevoSignoVital {
  pacienteId: string;
  registradoPorId: string;
  tipo: string;
  valor: number;
  unidad: string;
}

export interface ISignoVitalRepository {
  registrar(datos: NuevoSignoVital): Promise<SignoVital>;
  historialDePaciente(pacienteId: string, limite: number): Promise<SignoVital[]>;
}

export const SIGNO_VITAL_REPOSITORY = Symbol('SIGNO_VITAL_REPOSITORY');
```

`application/registrar-signo-vital.use-case.ts`:

```
import { Inject, Injectable } from '@nestjs/common';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../../admisiones/domain/repositories/paciente-repository.interface';
import { EVENT_PUBLISHER, IEventPublisher } from '../../shared/events/event-publisher.interface';
import { EVENTOS, SignoVitalFueraDeRangoPayload } from '../../shared/events/eventos';
import {
  DomainException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { SignoVital } from '../domain/entities/signo-vital.entity';
import {
  ISignoVitalRepository,
  SIGNO_VITAL_REPOSITORY,
} from '../domain/repositories/signo-vital-repository.interface';
import { Severidad, evaluarSignoVital, unidadDe } from '../domain/umbrales-signos-vitales';

export interface RegistrarSignoVitalCommand {
  pacienteId: string;
  registradoPorId: string;
  tipo: string;
  valor: number;
}

export interface SignoVitalRegistrado {
  signoVital: SignoVital;
  severidad: Severidad;
}

@Injectable()
export class RegistrarSignoVitalUseCase {
  constructor(
```

```
    @Inject(SIGNO_VITAL_REPOSITORY) private readonly signos: ISignoVitalRepository,
    @Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository,
    @Inject(EVENT_PUBLISHER) private readonly eventos: IEventPublisher,
  ) {}

  async ejecutar(cmd: RegistrarSignoVitalCommand): Promise<SignoVitalRegistrado> {
    if (!Number.isFinite(cmd.valor)) throw new DomainException('El valor debe ser un número');
    if (!(await this.pacientes.findById(cmd.pacienteId))) {
      throw new RecursoNoEncontradoException('Paciente');
    }

    const severidad = evaluarSignoVital(cmd.tipo, cmd.valor);   // lanza si el tipo no existe
    const signoVital = await this.signos.registrar({ ...cmd, unidad: unidadDe(cmd.tipo) });

    if (severidad !== 'normal') {
      const payload: SignoVitalFueraDeRangoPayload = {
        pacienteId: cmd.pacienteId,
        tipo: cmd.tipo,
        valor: cmd.valor,
        severidad,
      };
      this.eventos.publicar(EVENTOS.SIGNO_VITAL_FUERA_DE_RANGO, payload);
    }

    return { signoVital, severidad };
  }
}
```

La respuesta del endpoint incluye `severidad`, así el frontend puede pintar la alerta sin recalcular nada. Para el historial, `HistorialSignosVitalesUseCase` llama a `historialDePaciente(pacienteId, 100)`.

La implementación TypeORM guarda el valor como texto numérico (`numeric` de Postgres llega como `string`):

```
// infrastructure/repositories/typeorm-signo-vital.repository.ts
registrar(datos: NuevoSignoVital) {
  return this.repo.save(
    this.repo.create({ ...datos, valor: String(datos.valor) }),
  );
}

historialDePaciente(pacienteId: string, limite: number) {
  return this.repo.find({ where: { pacienteId }, order: { registradoEn: 'DESC' }, take: limite });
}
```

Paso 2. Builder de protocolos de cuidado

La idea. Un protocolo es una lista ordenada de tareas, y la lista cambia según el motivo de ingreso. El Builder arma el protocolo paso a paso y valida que quede completo. Los "directores" son recetas fijas: uno por motivo. Agregar un motivo nuevo es agregar un método, sin tocar el resto.

`domain/protocolos/protocolo.builder.ts`:

```
import { DomainException } from '../../../shared/exceptions/domain.exception';

export interface TareaPlan {
  descripcion: string;
  orden: number;
}

export interface PlanProtocolo {
  motivoIngreso: string;
  tareas: TareaPlan[];
}

export class ProtocoloBuilder {
  private motivo = '';
  private readonly tareas: string[] = [];

  conMotivo(motivo: string): this {
    this.motivo = motivo;
    return this;
  }

  agregarTarea(descripcion: string): this {
    this.tareas.push(descripcion);
    return this;
  }

  build(): PlanProtocolo {
```

```
    if (!this.motivo) throw new DomainException('El protocolo necesita un motivo de ingreso');
    if (this.tareas.length === 0) throw new DomainException('El protocolo necesita al menos una tarea');

    return {
      motivoIngreso: this.motivo,
      tareas: this.tareas.map((descripcion, i) => ({ descripcion, orden: i + 1 })),
    };
  }
}
```

`domain/protocolos/protocolo.director.ts`:

```
import { DomainException } from '../../../shared/exceptions/domain.exception';
import { PlanProtocolo, ProtocoloBuilder } from './protocolo.builder';

export enum MotivoIngreso {
  PRE_OPERATORIO = 'pre_operatorio',
  RESPIRATORIO = 'respiratorio',
}

/** Recetas: cada una decide qué tareas lleva el protocolo. Contenido de ejemplo para el TP. */
export class ProtocoloDirector {
  preOperatorio(): PlanProtocolo {
    return new ProtocoloBuilder()
      .conMotivo(MotivoIngreso.PRE_OPERATORIO)
      .agregarTarea('Verificar ayuno de al menos 8 horas')
      .agregarTarea('Confirmar consentimiento informado firmado')
      .agregarTarea('Controlar signos vitales basales')
      .agregarTarea('Retirar prótesis, joyas y esmalte de uñas')
      .agregarTarea('Realizar higiene y preparación de la zona quirúrgica')
      .agregarTarea('Canalizar vía periférica')
      .agregarTarea('Administrar la premedicación indicada')
      .build();
  }

  respiratorio(): PlanProtocolo {
    return new ProtocoloBuilder()
      .conMotivo(MotivoIngreso.RESPIRATORIO)
      .agregarTarea('Controlar saturación de oxígeno y frecuencia respiratoria')
      .agregarTarea('Colocar al paciente en posición semisentada')
      .agregarTarea('Administrar oxigenoterapia según indicación')
      .agregarTarea('Realizar nebulización indicada')
      .agregarTarea('Enseñar técnica de tos asistida')
      .agregarTarea('Registrar la evolución respiratoria cada 4 horas')
      .build();
  }

  construir(motivo: string): PlanProtocolo {
    switch (motivo) {
      case MotivoIngreso.PRE_OPERATORIO: return this.preOperatorio();
      case MotivoIngreso.RESPIRATORIO: return this.respiratorio();
      default: throw new DomainException(`No hay protocolo definido para "${motivo}"`);
    }
  }
}
```

`domain/repositories/protocolo-repository.interface.ts`:

```
import { PlanProtocolo } from '../protocolos/protocolo.builder';
import { ProtocoloCuidado } from '../entities/protocolo-cuidado.entity';
import { TareaProtocolo } from '../entities/tarea-protocolo.entity';

export interface IProtocoloRepository {
  /** Guarda el protocolo y todas sus tareas en una transacción. */
  crearConTareas(pacienteId: string, plan: PlanProtocolo): Promise<ProtocoloCuidado>;
  findById(id: string): Promise<ProtocoloCuidado | null>;
  tareasDe(protocoloId: string): Promise<TareaProtocolo[]>;
  /** UPDATE ... WHERE completada = false. false => ya estaba completada (o no existe). */
  completarTarea(tareaId: string): Promise<boolean>;
  protocoloDeTarea(tareaId: string): Promise<string | null>;
  contarPendientes(protocoloId: string): Promise<number>;
  finalizar(protocoloId: string): Promise<void>;
}

export const PROTOCOLO_REPOSITORY = Symbol('PROTOCOLO_REPOSITORY');
```

`infrastructure/repositories/typeorm-protocolo.repository.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { DataSource, Repository } from 'typeorm';
import { ProtocoloCuidado } from '../../domain/entities/protocolo-cuidado.entity';
import { TareaProtocolo } from '../../domain/entities/tarea-protocolo.entity';
import { IProtocoloRepository } from '../../domain/repositories/protocolo-repository.interface';
import { PlanProtocolo } from '../../domain/protocolos/protocolo.builder';

@Injectable()
```

```
export class TypeOrmProtocoloRepository implements IProtocoloRepository {
  constructor(
    @InjectRepository(ProtocoloCuidado) private readonly protocolos: Repository<ProtocoloCuidado>,
    @InjectRepository(TareaProtocolo) private readonly tareas: Repository<TareaProtocolo>,
    private readonly dataSource: DataSource,
  ) {}

  crearConTareas(pacienteId: string, plan: PlanProtocolo) {
    return this.dataSource.transaction(async (manager) => {
      const protocolo = await manager.save(
        manager.create(ProtocoloCuidado, { pacienteId, motivoIngreso: plan.motivoIngreso }),
      );
      await manager.save(
        plan.tareas.map((t) =>
          manager.create(TareaProtocolo, { protocolo: { id: protocolo.id }, ...t }),
        ),
      );
      return protocolo;
    });
  }

  findById(id: string) {
    return this.protocolos.findOneBy({ id });
  }

  tareasDe(protocoloId: string) {
    return this.tareas.find({ where: { protocolo: { id: protocoloId } }, order: { orden: 'ASC' } });
  }

  async completarTarea(tareaId: string) {
    const r = await this.tareas
      .createQueryBuilder()
      .update(TareaProtocolo)
      .set({ completada: true, completadaEn: () => 'now()' })
      .where('id = :tareaId AND completada = false', { tareaId })
      .execute();
    return r.affected === 1;
  }

  async protocoloDeTarea(tareaId: string) {
    const fila = await this.tareas
      .createQueryBuilder('t')
      .select('t."protocoloId"', 'protocoloId')
      .where('t.id = :tareaId', { tareaId })
      .getRawOne<{ protocoloId: string }>();
    return fila?.protocoloId ?? null;
  }

  contarPendientes(protocoloId: string) {
    return this.tareas.count({ where: { protocolo: { id: protocoloId }, completada: false } });
  }

  async finalizar(protocoloId: string) {
    await this.protocolos.update({ id: protocoloId }, { estado: 'finalizado' });
  }
}
```

`application/iniciar-protocolo.use-case.ts`:

```
import { Inject, Injectable } from '@nestjs/common';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../../admisiones/domain/repositories/paciente-repository.interface';
import { RecursoNoEncontradoException } from '../../shared/exceptions/domain.exception';
import { ProtocoloCuidado } from '../domain/entities/protocolo-cuidado.entity';
import { ProtocoloDirector } from '../domain/protocolos/protocolo.director';
import {
  IProtocoloRepository,
  PROTOCOLO_REPOSITORY,
} from '../domain/repositories/protocolo-repository.interface';

@Injectable()
export class IniciarProtocoloUseCase {
  private readonly director = new ProtocoloDirector();

  constructor(
    @Inject(PROTOCOLO_REPOSITORY) private readonly protocolos: IProtocoloRepository,
    @Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository,
  ) {}

  async ejecutar(pacienteId: string, motivoIngreso: string): Promise<ProtocoloCuidado> {
    if (!(await this.pacientes.findById(pacienteId))) {
      throw new RecursoNoEncontradoException('Paciente');
    }

    const plan = this.director.construir(motivoIngreso);   // el Builder valida que el plan esté completo
    return this.protocolos.crearConTareas(pacienteId, plan);
  }
}
```

`application/completar-tarea.use-case.ts`. Cuando no quedan tareas pendientes, el protocolo se finaliza solo:

```ts
import { Inject, Injectable } from '@nestjs/common';
import {
  ConflictoDeConcurrenciaException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import {
  IProtocoloRepository,
  PROTOCOLO_REPOSITORY,
} from '../domain/repositories/protocolo-repository.interface';

@Injectable()
export class CompletarTareaUseCase {
  constructor(@Inject(PROTOCOLO_REPOSITORY) private readonly protocolos: IProtocoloRepository) {}

  async ejecutar(tareaId: string): Promise<{ protocoloFinalizado: boolean }> {
    const protocoloId = await this.protocolos.protocoloDeTarea(tareaId);
    if (!protocoloId) throw new RecursoNoEncontradoException('Tarea');

    // Si dos enfermeros marcan la misma tarea a la vez, solo uno gana.
    if (!(await this.protocolos.completarTarea(tareaId))) {
      throw new ConflictoDeConcurrenciaException('La tarea');
    }

    const pendientes = await this.protocolos.contarPendientes(protocoloId);
    if (pendientes === 0) await this.protocolos.finalizar(protocoloId);

    return { protocoloFinalizado: pendientes === 0 };
  }
}
```

Paso 3. Indicaciones médicas y administración de medicación

Es el flujo que conecta tres módulos:

1. El **médico** crea una indicación de tipo `medicacion` con el insumo y la cantidad.
2. El **enfermero** la cumple. Eso marca la indicación, **descuenta el stock** (módulo Camas) y **publica el evento** `MedicacionAdministrada`.
3. **Facturación** escucha el evento y genera el cargo (se implementa en el módulo siguiente).

`domain/repositories/indicacion-repository.interface.ts`:

```ts
import { IndicacionMedica } from '../entities/indicacion-medica.entity';

export interface NuevaIndicacion {
  pacienteId: string;
  medicoId: string;
  tipo: string;
  descripcion: string;
  insumoId?: string;
  cantidad?: number;
}

export interface IIndicacionRepository {
  crear(datos: NuevaIndicacion): Promise<IndicacionMedica>;
  findById(id: string): Promise<IndicacionMedica | null>;
  listarPendientes(pacienteId: string): Promise<IndicacionMedica[]>;
  /** UPDATE ... WHERE estado = 'pendiente'. false => otro enfermero ya la cumplió o fue cancelada. */
  marcarCumplida(id: string): Promise<boolean>;
  /** Compensación: vuelve a 'pendiente' si falló el descuento de stock. */
  reabrir(id: string): Promise<void>;
}

export const INDICACION_REPOSITORY = Symbol('INDICACION_REPOSITORY');
```

`infrastructure/repositories/typeorm-indicacion.repository.ts`:

```ts
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { IndicacionMedica } from '../../domain/entities/indicacion-medica.entity';
import {
  IIndicacionRepository,
  NuevaIndicacion,
} from '../../domain/repositories/indicacion-repository.interface';

@Injectable()
export class TypeOrmIndicacionRepository implements IIndicacionRepository {
  constructor(@InjectRepository(IndicacionMedica) private readonly repo: Repository<IndicacionMedica>) {}
```

```
  crear(d: NuevaIndicacion) {
    return this.repo.save(
      this.repo.create({
        pacienteId: d.pacienteId,
        medicoId: d.medicoId,
        tipo: d.tipo,
        descripcion: d.descripcion,
        insumoId: d.insumoId ?? null,
        cantidad: d.cantidad ?? null,
      }),
    );
  }

  findById(id: string) {
    return this.repo.findOneBy({ id });
  }

  listarPendientes(pacienteId: string) {
    return this.repo.find({ where: { pacienteId, estado: 'pendiente' }, order: { creadaEn: 'ASC' } });
  }

  async marcarCumplida(id: string) {
    const r = await this.repo.update({ id, estado: 'pendiente' }, { estado: 'cumplida' });
    return r.affected === 1;
  }

  async reabrir(id: string) {
    await this.repo.update({ id }, { estado: 'pendiente' });
  }
}
```

`application/indicar-medicacion.use-case.ts`:

```
import { Inject, Injectable } from '@nestjs/common';
import {
  IMedicoRepository,
  MEDICO_REPOSITORY,
} from '../../admisiones/domain/repositories/medico-repository.interface';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../../admisiones/domain/repositories/paciente-repository.interface';
import {
  DomainException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { IndicacionMedica } from '../domain/entities/indicacion-medica.entity';
import {
  IIndicacionRepository,
  INDICACION_REPOSITORY,
} from '../domain/repositories/indicacion-repository.interface';

export interface IndicarCommand {
  pacienteId: string;
  usuarioMedicoId: string;
  tipo: string;
  descripcion: string;
  insumoId?: string;
  cantidad?: number;
}

@Injectable()
export class IndicarUseCase {
  constructor(
    @Inject(INDICACION_REPOSITORY) private readonly indicaciones: IIndicacionRepository,
    @Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository,
    @Inject(MEDICO_REPOSITORY) private readonly medicos: IMedicoRepository,
  ) {}

  async ejecutar(cmd: IndicarCommand): Promise<IndicacionMedica> {
    if (!(await this.pacientes.findById(cmd.pacienteId))) {
      throw new RecursoNoEncontradoException('Paciente');
    }
    const medico = await this.medicos.findByUsuarioId(cmd.usuarioMedicoId);
    if (!medico) throw new RecursoNoEncontradoException('Médico asociado al usuario');

    if (cmd.tipo === 'medicacion' && (!cmd.insumoId || !cmd.cantidad)) {
      throw new DomainException('Una indicación de medicación necesita insumo y cantidad');
    }

    return this.indicaciones.crear({ ...cmd, medicoId: medico.id });
  }
}
```

`application/cumplir-indicacion.use-case.ts`. El corazón del módulo:

```
import { Inject, Injectable } from '@nestjs/common';
import {
```

```
  DescontarInsumoUseCase,
  InsumoDescontado,
} from '../../camas/application/descontar-insumo.use-case';
import { EVENT_PUBLISHER, IEventPublisher } from '../../shared/events/event-publisher.interface';
import { EVENTOS, MedicacionAdministradaPayload } from '../../shared/events/eventos';
import {
  ConflictoDeConcurrenciaException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import {
  IIndicacionRepository,
  INDICACION_REPOSITORY,
} from '../domain/repositories/indicacion-repository.interface';

@Injectable()
export class CumplirIndicacionUseCase {
  constructor(
    @Inject(INDICACION_REPOSITORY) private readonly indicaciones: IIndicacionRepository,
    private readonly descontarInsumo: DescontarInsumoUseCase,
    @Inject(EVENT_PUBLISHER) private readonly eventos: IEventPublisher,
  ) {}

  async ejecutar(indicacionId: string): Promise<void> {
    const indicacion = await this.indicaciones.findById(indicacionId);
    if (!indicacion) throw new RecursoNoEncontradoException('Indicación');

    // 1. Reclamar la indicación: si dos enfermeros llegan juntos, solo uno pasa (409 al otro).
    if (!(await this.indicaciones.marcarCumplida(indicacion.id))) {
      throw new ConflictoDeConcurrenciaException('La indicación');
    }

    if (indicacion.tipo !== 'medicacion' || !indicacion.insumoId || !indicacion.cantidad) return;

    // 2. Descontar el stock. Si no alcanza, se revierte el paso 1 y el error llega al cliente.
    let insumo: InsumoDescontado;
    try {
      insumo = await this.descontarInsumo.ejecutar(indicacion.insumoId, indicacion.cantidad);
    } catch (error) {
      await this.indicaciones.reabrir(indicacion.id);
      throw error;
    }

    // 3. Avisar a Facturación sin bloquear: el evento se emite y el flujo sigue.
    const payload: MedicacionAdministradaPayload = {
      indicacionId: indicacion.id,
      pacienteId: indicacion.pacienteId,
      insumoId: indicacion.insumoId,
      concepto: `Medicación: ${insumo.nombre}`,
      cantidad: indicacion.cantidad,
      precioUnitario: insumo.precioUnitario,
    };
    this.eventos.publicar(EVENTOS.MEDICACION_ADMINISTRADA, payload);
  }
}
```

Por qué el paso 1 va primero. Reclamar la indicación antes de descontar evita descontar dos veces el mismo medicamento. Si el descuento falla, se compensa reabriendo la indicación. La alternativa sería una transacción que abarque los dos módulos, que obliga a compartir la conexión entre ellos; para este alcance, la compensación es más simple y se entiende mejor.

Paso 4. Controllers

`presentation/dto/atencion.dto.ts`:

```
import { ApiProperty, ApiPropertyOptional } from '@nestjs/swagger';
import { IsEnum, IsIn, IsInt, IsNotEmpty, IsNumber, IsOptional, IsString, IsUUID, Min } from 'class-validator';
import { TIPOS_SIGNO_VITAL } from '../../domain/umbrales-signos-vitales';
import { MotivoIngreso } from '../../domain/protocolos/protocolo.director';

export class RegistrarSignoVitalDto {
  @ApiProperty({ enum: TIPOS_SIGNO_VITAL }) @IsIn(TIPOS_SIGNO_VITAL) tipo: string;
  @ApiProperty({ example: 98 }) @IsNumber() valor: number;
}

export class CrearIndicacionDto {
  @ApiProperty({ example: 'medicacion' }) @IsIn(['medicacion', 'estudio', 'dieta', 'otro']) tipo: string;
  @ApiProperty() @IsString() @IsNotEmpty() descripcion: string;
  @ApiPropertyOptional() @IsOptional() @IsUUID() insumoId?: string;
  @ApiPropertyOptional() @IsOptional() @IsInt() @Min(1) cantidad?: number;
}

export class IniciarProtocoloDto {
  @ApiProperty({ enum: MotivoIngreso }) @IsEnum(MotivoIngreso) motivoIngreso: MotivoIngreso;
}
```

`presentation/signos-vitales.controller.ts`:

```
import { Body, Controller, Get, Param, ParseUUIDPipe, Post, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { RolUsuario } from '../../iam/domain/enums/rol.enum';
import { Roles } from '../../shared/decorators/roles.decorator';
import { UsuarioActual, UsuarioAutenticado } from '../../shared/decorators/usuario-actual.decorator';
import { JwtAuthGuard } from '../../shared/guards/jwt-auth.guard';
import { RolesGuard } from '../../shared/guards/roles.guard';
import { HistorialSignosVitalesUseCase } from '../application/historial-signos-vitales.use-case';
import { RegistrarSignoVitalUseCase } from '../application/registrar-signo-vital.use-case';
import { RegistrarSignoVitalDto } from './dto/atencion.dto';

@ApiTags('atencion-enfermeria')
@ApiBearerAuth()
@UseGuards(JwtAuthGuard, RolesGuard)
@Controller('atencion-enfermeria/pacientes/:pacienteId/signos-vitales')
export class SignosVitalesController {
  constructor(
    private readonly registrar: RegistrarSignoVitalUseCase,
    private readonly historial: HistorialSignosVitalesUseCase,
  ) {}

  @Post()
  @Roles(RolUsuario.ENFERMERO, RolUsuario.MEDICO)
  registrarSigno(
    @Param('pacienteId', ParseUUIDPipe) pacienteId: string,
    @Body() dto: RegistrarSignoVitalDto,
    @UsuarioActual() usuario: UsuarioAutenticado,
  ) {
    return this.registrar.ejecutar({ pacienteId, registradoPorId: usuario.id, ...dto });
  }

  @Get()
  @Roles(RolUsuario.ENFERMERO, RolUsuario.MEDICO, RolUsuario.ADMIN)
  verHistorial(@Param('pacienteId', ParseUUIDPipe) pacienteId: string) {
    return this.historial.ejecutar(pacienteId);
  }
}
```

`IndicacionesController` (`/atencion-enfermeria`):

```
@Post('pacientes/:pacienteId/indicaciones')   // MEDICO: crea la indicación
@Get('pacientes/:pacienteId/indicaciones')    // ENFERMERO, MEDICO: pendientes del paciente
@Patch('indicaciones/:id/cumplir')            // ENFERMERO: llama a CumplirIndicacionUseCase
```

`ProtocolosController`:

```
@Post('pacientes/:pacienteId/protocolos')     // ENFERMERO, MEDICO: { motivoIngreso }
@Get('protocolos/:id/tareas')                 // ENFERMERO, MEDICO
@Patch('tareas/:id/completar')                // ENFERMERO
```

Todos siguen el mismo molde de `SignosVitalesController`: `@UseGuards(JwtAuthGuard, RolesGuard)`, `@Roles(...)` por handler y el caso de uso inyectado por constructor.

Paso 5. Módulo

`atencion-enfermeria/atencion-enfermeria.module.ts`:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AdmisionesModule } from '../admisiones/admisiones.module';
import { CamasModule } from '../camas/camas.module';
import { CompletarTareaUseCase } from './application/completar-tarea.use-case';
import { CumplirIndicacionUseCase } from './application/cumplir-indicacion.use-case';
import { HistorialSignosVitalesUseCase } from './application/historial-signos-vitales.use-case';
import { IndicarUseCase } from './application/indicar-medicacion.use-case';
import { IniciarProtocoloUseCase } from './application/iniciar-protocolo.use-case';
import { RegistrarSignoVitalUseCase } from './application/registrar-signo-vital.use-case';
import { entidadesAtencionEnfermeria } from './domain/entities';
import { INDICACION_REPOSITORY } from './domain/repositories/indicacion-repository.interface';
import { PROTOCOLO_REPOSITORY } from './domain/repositories/protocolo-repository.interface';
import { SIGNO_VITAL_REPOSITORY } from './domain/repositories/signo-vital-repository.interface';
import { TypeOrmIndicacionRepository } from './infrastructure/repositories/typeorm-indicacion.repository';
import { TypeOrmProtocoloRepository } from './infrastructure/repositories/typeorm-protocolo.repository';
import { TypeOrmSignoVitalRepository } from './infrastructure/repositories/typeorm-signo-vital.repository';
import { AtencionEnfermeriaController } from './presentation/atencion-enfermeria.controller';
import { IndicacionesController } from './presentation/indicaciones.controller';
import { ProtocolosController } from './presentation/protocolos.controller';
import { SignosVitalesController } from './presentation/signos-vitales.controller';
```

```
@Module({
  imports: [TypeOrmModule.forFeature(entidadesAtencionEnfermeria), AdmisionesModule, CamasModule],
  controllers: [
    AtencionEnfermeriaController,
    SignosVitalesController,
    IndicacionesController,
    ProtocolosController,
  ],
  providers: [
    { provide: SIGNO_VITAL_REPOSITORY, useClass: TypeOrmSignoVitalRepository },
    { provide: INDICACION_REPOSITORY, useClass: TypeOrmIndicacionRepository },
    { provide: PROTOCOLO_REPOSITORY, useClass: TypeOrmProtocoloRepository },
    RegistrarSignoVitalUseCase,
    HistorialSignosVitalesUseCase,
    IndicarUseCase,
    CumplirIndicacionUseCase,
    IniciarProtocoloUseCase,
    CompletarTareaUseCase,
  ],
})
export class AtencionEnfermeriaModule {}
```

Endpoints

MÉTODO	RUTA	ROLES	RESPUESTAS
POST	`/atencion-enfermeria/pacientes/:id/signos-vitales`	Enfermero, Médico	201 con `severidad`
GET	`/atencion-enfermeria/pacientes/:id/signos-vitales`	Enfermero, Médico, Admin	200
POST	`/atencion-enfermeria/pacientes/:id/indicaciones`	Médico	201, 422 si falta insumo o cantidad
GET	`/atencion-enfermeria/pacientes/:id/indicaciones`	Enfermero, Médico	200, pendientes
PATCH	`/atencion-enfermeria/indicaciones/:id/cumplir`	Enfermero	200, **409 ya cumplida**, 422 stock insuficiente
POST	`/atencion-enfermeria/pacientes/:id/protocolos`	Enfermero, Médico	201 con sus tareas
GET	`/atencion-enfermeria/protocolos/:id/tareas`	Enfermero, Médico	200
PATCH	`/atencion-enfermeria/tareas/:id/completar`	Enfermero	200, 409

Tests

`domain/umbrales-signos-vitales.spec.ts` :

```
import { DomainException } from '../../shared/exceptions/domain.exception';
import { evaluarSignoVital } from './umbrales-signos-vitales';

describe('evaluarSignoVital', () => {
  it.each([
    ['frecuencia_cardiaca', 75, 'normal'],
    ['frecuencia_cardiaca', 110, 'alerta'],
    ['frecuencia_cardiaca', 35, 'critico'],
    ['temperatura', 36.8, 'normal'],
    ['temperatura', 38.2, 'alerta'],
    ['temperatura', 40, 'critico'],
    ['saturacion_o2', 97, 'normal'],
    ['saturacion_o2', 92, 'alerta'],
```

```
    ['saturacion_o2', 85, 'critico'],
  ] as const)('%s = %s es %s', (tipo, valor, esperado) => {
    expect(evaluarSignoVital(tipo, valor)).toBe(esperado);
  });

  it('los bordes del rango normal todavía son normales', () => {
    expect(evaluarSignoVital('frecuencia_cardiaca', 60)).toBe('normal');
    expect(evaluarSignoVital('frecuencia_cardiaca', 100)).toBe('normal');
  });

  it('rechaza un tipo desconocido', () => {
    expect(() => evaluarSignoVital('azucar', 90)).toThrow(DomainException);
  });
});
```

`domain/protocolos/protocolo.builder.spec.ts`:

```
import { DomainException } from '../../../shared/exceptions/domain.exception';
import { ProtocoloBuilder } from './protocolo.builder';
import { MotivoIngreso, ProtocoloDirector } from './protocolo.director';

describe('ProtocoloBuilder', () => {
  it('numera las tareas en el orden en que se agregan', () => {
    const plan = new ProtocoloBuilder().conMotivo('x').agregarTarea('A').agregarTarea('B').build();
    expect(plan.tareas).toEqual([
      { descripcion: 'A', orden: 1 },
      { descripcion: 'B', orden: 2 },
    ]);
  });

  it('no arma un protocolo sin motivo o sin tareas', () => {
    expect(() => new ProtocoloBuilder().agregarTarea('A').build()).toThrow(DomainException);
    expect(() => new ProtocoloBuilder().conMotivo('x').build()).toThrow(DomainException);
  });
});

describe('ProtocoloDirector', () => {
  const director = new ProtocoloDirector();

  it('el protocolo pre-operatorio y el respiratorio tienen tareas distintas', () => {
    const pre = director.construir(MotivoIngreso.PRE_OPERATORIO);
    const resp = director.construir(MotivoIngreso.RESPIRATORIO);

    expect(pre.motivoIngreso).toBe('pre_operatorio');
    expect(resp.motivoIngreso).toBe('respiratorio');
    expect(pre.tareas.map((t) => t.descripcion)).not.toEqual(resp.tareas.map((t) => t.descripcion));
  });

  it('rechaza un motivo sin receta', () => {
    expect(() => director.construir('desconocido')).toThrow(DomainException);
  });
});
```

`application/cumplir-indicacion.use-case.spec.ts`: lo importante es la compensación y el evento.

```
import { DomainException } from '../../shared/exceptions/domain.exception';
import { EVENTOS } from '../../shared/events/eventos';
import { IndicacionMedica } from '../domain/entities/indicacion-medica.entity';
import { CumplirIndicacionUseCase } from './cumplir-indicacion.use-case';

function armar(opciones: { stockAlcanza: boolean }) {
  const indicacion = Object.assign(new IndicacionMedica(), {
    id: 'ind1', pacienteId: 'p1', tipo: 'medicacion', insumoId: 'i1', cantidad: 2, estado: 'pendiente',
  });
  const estado = { estado: 'pendiente' as string };
  const publicados: Array<{ evento: string; payload: object }> = [];

  const indicaciones = {
    findById: async () => indicacion,
    marcarCumplida: async () => {
      if (estado.estado !== 'pendiente') return false;
      estado.estado = 'cumplida';
      return true;
    },
    reabrir: async () => { estado.estado = 'pendiente'; },
  };
  const descontar = {
    ejecutar: async () => {
      if (!opciones.stockAlcanza) throw new DomainException('Stock insuficiente de Ampolla');
      return { nombre: 'Ampolla', precioUnitario: '120.00' };
    },
  };
  const eventos = { publicar: (evento: string, payload: object) => { publicados.push({ evento, payload }); } };

  const useCase = new CumplirIndicacionUseCase(indicaciones as never, descontar as never, eventos);
  return { useCase, estado, publicados };
}

describe('CumplirIndicacionUseCase', () => {
```

```
  it('marca la indicación, descuenta el stock y publica MedicacionAdministrada', async () => {
    const { useCase, estado, publicados } = armar({ stockAlcanza: true });

    await useCase.ejecutar('ind1');

    expect(estado.estado).toBe('cumplida');
    expect(publicados).toHaveLength(1);
    expect(publicados[0].evento).toBe(EVENTOS.MEDICACION_ADMINISTRADA);
    expect(publicados[0].payload).toMatchObject({ indicacionId: 'ind1', cantidad: 2, precioUnitario: '120.00' });
  });

  it('si no hay stock, deja la indicación pendiente y no publica nada', async () => {
    const { useCase, estado, publicados } = armar({ stockAlcanza: false });

    await expect(useCase.ejecutar('ind1')).rejects.toThrow('Stock insuficiente');

    expect(estado.estado).toBe('pendiente');
    expect(publicados).toEqual([]);
  });

  it('la segunda persona que la cumple recibe conflicto', async () => {
    const { useCase } = armar({ stockAlcanza: true });

    await useCase.ejecutar('ind1');
    await expect(useCase.ejecutar('ind1')).rejects.toThrow('acaba de ser modificado');
  });
});
```

Terminado cuando

- [ ] Registrar una frecuencia cardíaca de 75 devuelve `severidad: "normal"`; una de 110, `"alerta"`; una de 35, `"critico"`, y en este último caso se publica el evento.
- [ ] Iniciar un protocolo `pre_operatorio` crea el protocolo con sus siete tareas numeradas; completar todas lo pasa a `finalizado`.
- [ ] Cumplir una indicación de medicación descuenta el stock en Camas y deja el evento publicado (se ve en Facturación).
- [ ] Cumplir la misma indicación dos veces devuelve 409 la segunda.
- [ ] Con stock insuficiente, cumplir la indicación devuelve 422 y la indicación sigue `pendiente`.
- [ ] Los tests nuevos pasan y los anteriores siguen en verde.

Facturación

Objetivo. Calcular cuánto paga el paciente y cuánto absorbe su cobertura (pública, prepaga o particular), generar cargos automáticamente cuando enfermería administra medicación, y llevar cada factura por su ciclo (pendiente, emitida, pagada o anulada).

Patrones de este módulo: Strategy (cálculo por cobertura, Épica 1) y Observer con eventos de dominio y outbox (Épica 2).

Qué hay hoy

YA EXISTE	FALTA
Entidades `Cobertura`, `Factura`, `LineaFactura`, `EventoDominio`	Estrategias de cálculo
`FacturacionController` con `GET /facturacion/ping`	Repositorios y casos de uso
	Listener del evento `MedicacionAdministrada`
	Ciclo de vida de la factura

Antes de empezar: Admisiones terminado (el paciente trae su `coberturaId`), el módulo de Atención publicando `MedicacionAdministrada`, y la columna `Factura.montoCubierto`, que ya está en la entidad.

Cómo se calcula

El dinero se maneja en **centavos enteros**, nunca con decimales de punto flotante: `0.1 + 0.2` no da `0.3` en JavaScript. Las columnas `numeric` de Postgres llegan como texto (`"1250.50"`); se convierten a centavos al entrar y a texto al guardar.

COBERTURA (`TIPO`)	CUBRE	EL PACIENTE PAGA
`publica`	100%	0
`prepaga`	`porcentajeCobertura` de la cobertura (ej. 70%)	el resto
`particular`	0%	todo

Ejemplo: un cargo de $10.000 con una prepaga al 70% da $7.000 cubiertos y $3.000 a cargo del paciente.

Estructura final

```
facturacion/
├── domain/
│   ├── entities/                              (existen)
│   ├── dinero.ts                              NUEVO
│   ├── estrategias/
│   │   ├── estrategia-cobertura.interface.ts    NUEVO  (puerto del Strategy)
│   │   ├── cobertura-publica.ts                 NUEVO
│   │   ├── cobertura-prepaga.ts                 NUEVO
│   │   ├── cobertura-particular.ts              NUEVO
│   │   └── resolver-estrategia.ts               NUEVO
│   ├── factura-workflow.ts                    NUEVO
│   └── repositories/                          NUEVO
├── application/                               NUEVO
├── infrastructure/
│   ├── repositories/                          NUEVO
│   └── listeners/medicacion-administrada.listener.ts   NUEVO
├── presentation/
│   ├── facturacion.controller.ts              (existe: ping)
```

```
│   ├── facturas.controller.ts                NUEVO
│   └── dto/                                  NUEVO
└── facturacion.module.ts                     (se modifica)
```

Paso 1. Dinero

`domain/dinero.ts`:

```ts
/** "1250.50" (o 1250.5) -> 125050 centavos. */
export function aCentavos(valor: string | number): number {
  return Math.round(Number(valor) * 100);
}

/** 125050 centavos -> "1250.50", listo para una columna numeric. */
export function deCentavos(centavos: number): string {
  return (centavos / 100).toFixed(2);
}
```

Paso 2. Strategy de cobertura

`domain/estrategias/estrategia-cobertura.interface.ts`:

```ts
export interface CalculoCargo {
  montoCubiertoCentavos: number;
  montoPacienteCentavos: number;
}

export interface IEstrategiaCobertura {
  /** Coincide con Cobertura.tipo: 'publica' | 'prepaga' | 'particular' | ... */
  readonly tipo: string;
  calcular(baseCentavos: number, porcentajeCobertura: number): CalculoCargo;
}

export const ESTRATEGIAS_COBERTURA = Symbol('ESTRATEGIAS_COBERTURA');
```

`domain/estrategias/cobertura-publica.ts`, `cobertura-prepaga.ts` y `cobertura-particular.ts`.
Cada una es una clase de pocas líneas:

```ts
import { Injectable } from '@nestjs/common';
import { CalculoCargo, IEstrategiaCobertura } from './estrategia-cobertura.interface';

@Injectable()
export class CoberturaPublica implements IEstrategiaCobertura {
  readonly tipo = 'publica';

  calcular(base: number): CalculoCargo {
    return { montoCubiertoCentavos: base, montoPacienteCentavos: 0 };
  }
}
```

```ts
@Injectable()
export class CoberturaParticular implements IEstrategiaCobertura {
  readonly tipo = 'particular';

  calcular(base: number): CalculoCargo {
    return { montoCubiertoCentavos: 0, montoPacienteCentavos: base };
  }
}
```

```ts
@Injectable()
export class CoberturaPrepaga implements IEstrategiaCobertura {
  readonly tipo = 'prepaga';

  calcular(base: number, porcentajeCobertura: number): CalculoCargo {
    const cubierto = Math.round((base * porcentajeCobertura) / 100);
    return { montoCubiertoCentavos: cubierto, montoPacienteCentavos: base - cubierto };
  }
}
```

Estas tres clases usan `@Injectable()` de Nest. Es el único import de framework que se permite en esta carpeta, porque las estrategias se registran como providers. La lógica interna sigue sin depender de nada.

`domain/estrategias/resolver-estrategia.ts`. Busca la estrategia que corresponde al tipo de cobertura:

```ts
import { Inject, Injectable } from '@nestjs/common';
import { DomainException } from '../../../shared/exceptions/domain.exception';
import { ESTRATEGIAS_COBERTURA, IEstrategiaCobertura } from './estrategia-cobertura.interface';

@Injectable()
export class ResolverEstrategiaCobertura {
  constructor(@Inject(ESTRATEGIAS_COBERTURA) private readonly estrategias: IEstrategiaCobertura[]) {}

  para(tipo: string): IEstrategiaCobertura {
    const estrategia = this.estrategias.find((e) => e.tipo === tipo);
    if (!estrategia) throw new DomainException(`No hay estrategia de cálculo para la cobertura "${tipo}"`);
    return estrategia;
  }
}
```

Agregar una cobertura nueva (por ejemplo ART) es crear `cobertura-art.ts` con su clase y sumarla al módulo en dos lugares (lista de providers y lista `inject` del factory, ver el módulo más abajo). No se toca ninguna estrategia ni ningún caso de uso existente: es el principio abierto/cerrado.

Paso 3. Repositorios

`domain/repositories/factura-repository.interface.ts`:

```ts
import { Cobertura } from '../entities/cobertura.entity';
import { Factura } from '../entities/factura.entity';
import { LineaFactura } from '../entities/linea-factura.entity';

export interface IFacturaRepository {
  findById(id: string): Promise<Factura | null>;
  /** La factura 'pendiente' (abierta) del paciente, si existe. */
  findAbiertaDePaciente(pacienteId: string): Promise<Factura | null>;
  listarDePaciente(pacienteId: string): Promise<Factura[]>;
  crear(pacienteId: string, coberturaId: string): Promise<Factura>;

  agregarLinea(facturaId: string, concepto: string, cantidad: number, precioUnitario: string): Promise<LineaFactura>;
  lineasDe(facturaId: string): Promise<LineaFactura[]>;
  actualizarTotales(facturaId: string, total: string, montoCubierto: string): Promise<void>;

  /** UPDATE ... WHERE estado = desde. false => otro cambió el estado antes. */
  cambiarEstado(id: string, desde: string, hasta: string): Promise<boolean>;

  findCobertura(id: string): Promise<Cobertura | null>;
  findCoberturaPorTipo(tipo: string): Promise<Cobertura | null>;
  listarCoberturas(): Promise<Cobertura[]>;
  crearCobertura(nombre: string, tipo: string, porcentaje: string): Promise<Cobertura>;
}

export const FACTURA_REPOSITORY = Symbol('FACTURA_REPOSITORY');
```

`domain/repositories/evento-dominio-repository.interface.ts`:

```ts
import { EventoDominio } from '../entities/evento-dominio.entity';

export interface IEventoDominioRepository {
  registrar(tipo: string, payload: Record<string, unknown>): Promise<EventoDominio>;
  marcarProcesado(id: string): Promise<void>;
  /** ¿Ya se procesó un evento de este tipo para esta clave? Evita cobrar dos veces. */
  existeProcesado(tipo: string, claveIdempotencia: string): Promise<boolean>;
}

export const EVENTO_DOMINIO_REPOSITORY = Symbol('EVENTO_DOMINIO_REPOSITORY');
```

`infrastructure/repositories/typeorm-factura.repository.ts`:

```ts
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Cobertura } from '../../domain/entities/cobertura.entity';
import { Factura } from '../../domain/entities/factura.entity';
import { LineaFactura } from '../../domain/entities/linea-factura.entity';
import { IFacturaRepository } from '../../domain/repositories/factura-repository.interface';

@Injectable()
export class TypeOrmFacturaRepository implements IFacturaRepository {
  constructor(
```

```
    @InjectRepository(Factura) private readonly facturas: Repository<Factura>,
    @InjectRepository(LineaFactura) private readonly lineas: Repository<LineaFactura>,
    @InjectRepository(Cobertura) private readonly coberturas: Repository<Cobertura>,
  ) {}

  findById(id: string) {
    return this.facturas.findOne({ where: { id }, relations: { cobertura: true } });
  }

  findAbiertaDePaciente(pacienteId: string) {
    return this.facturas.findOne({
      where: { pacienteId, estado: 'pendiente' },
      relations: { cobertura: true },
    });
  }

  listarDePaciente(pacienteId: string) {
    return this.facturas.find({
      where: { pacienteId },
      relations: { cobertura: true },
      order: { emitidaEn: 'DESC' },
    });
  }

  crear(pacienteId: string, coberturaId: string) {
    return this.facturas.save(this.facturas.create({ pacienteId, cobertura: { id: coberturaId } }));
  }

  agregarLinea(facturaId: string, concepto: string, cantidad: number, precioUnitario: string) {
    return this.lineas.save(
      this.lineas.create({ factura: { id: facturaId }, concepto, cantidad, precioUnitario }),
    );
  }

  lineasDe(facturaId: string) {
    return this.lineas.find({ where: { factura: { id: facturaId } } });
  }

  async actualizarTotales(facturaId: string, total: string, montoCubierto: string) {
    await this.facturas.update({ id: facturaId }, { total, montoCubierto });
  }

  async cambiarEstado(id: string, desde: string, hasta: string) {
    const r = await this.facturas.update({ id, estado: desde }, { estado: hasta });
    return r.affected === 1;
  }

  findCobertura(id: string) {
    return this.coberturas.findOneBy({ id });
  }

  findCoberturaPorTipo(tipo: string) {
    return this.coberturas.findOneBy({ tipo });
  }

  listarCoberturas() {
    return this.coberturas.find({ order: { nombre: 'ASC' } });
  }

  crearCobertura(nombre: string, tipo: string, porcentaje: string) {
    return this.coberturas.save(
      this.coberturas.create({ nombre, tipo, porcentajeCobertura: porcentaje }),
    );
  }
}
```

`infrastructure/repositories/typeorm-evento-dominio.repository.ts`:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { EventoDominio } from '../../domain/entities/evento-dominio.entity';
import { IEventoDominioRepository } from '../../domain/repositories/evento-dominio-repository.interface';

@Injectable()
export class TypeOrmEventoDominioRepository implements IEventoDominioRepository {
  constructor(@InjectRepository(EventoDominio) private readonly repo: Repository<EventoDominio>) {}

  registrar(tipo: string, payload: Record<string, unknown>) {
    return this.repo.save(this.repo.create({ tipo, payload }));
  }

  async marcarProcesado(id: string) {
    await this.repo.update({ id }, { procesado: true });
  }

  async existeProcesado(tipo: string, claveIdempotencia: string) {
    const cantidad = await this.repo
      .createQueryBuilder('e')
      .where('e.tipo = :tipo', { tipo })
      .andWhere('e.procesado = true')
      .andWhere(`e.payload ->> 'indicacionId' = :clave`, { clave: claveIdempotencia })
```

```
      .getCount();
    return cantidad > 0;
  }
}
```

Paso 4. Casos de uso

Registrar un cargo. Es la pieza central: la usa el listener del evento y también la carga manual de líneas.

`application/registrar-cargo.use-case.ts`:

```
import { Inject, Injectable } from '@nestjs/common';
import {
  IPacienteRepository,
  PACIENTE_REPOSITORY,
} from '../../admisiones/domain/repositories/paciente-repository.interface';
import {
  DomainException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { aCentavos, deCentavos } from '../domain/dinero';
import { Cobertura } from '../domain/entities/cobertura.entity';
import { Factura } from '../domain/entities/factura.entity';
import { ResolverEstrategiaCobertura } from '../domain/estrategias/resolver-estrategia';
import {
  FACTURA_REPOSITORY,
  IFacturaRepository,
} from '../domain/repositories/factura-repository.interface';

export interface RegistrarCargoCommand {
  pacienteId: string;
  concepto: string;
  cantidad: number;
  precioUnitario: string;
}

@Injectable()
export class RegistrarCargoUseCase {
  constructor(
    @Inject(FACTURA_REPOSITORY) private readonly facturas: IFacturaRepository,
    @Inject(PACIENTE_REPOSITORY) private readonly pacientes: IPacienteRepository,
    private readonly estrategias: ResolverEstrategiaCobertura,
  ) {}

  async ejecutar(cmd: RegistrarCargoCommand): Promise<Factura> {
    if (!Number.isInteger(cmd.cantidad) || cmd.cantidad <= 0) {
      throw new DomainException('La cantidad debe ser un entero positivo');
    }

    const paciente = await this.pacientes.findById(cmd.pacienteId);
    if (!paciente) throw new RecursoNoEncontradoException('Paciente');

    // Factura abierta del paciente, o una nueva con su cobertura (sin cobertura => particular).
    let factura = await this.facturas.findAbiertaDePaciente(paciente.id);
    if (!factura) {
      const cobertura = await this.coberturaDe(paciente.coberturaId);
      factura = await this.facturas.crear(paciente.id, cobertura.id);
      factura.cobertura = cobertura;
    }

    await this.facturas.agregarLinea(factura.id, cmd.concepto, cmd.cantidad, cmd.precioUnitario);
    await this.recalcular(factura);

    return (await this.facturas.findById(factura.id)) ?? factura;
  }

  private async coberturaDe(coberturaId: string | null): Promise<Cobertura> {
    const cobertura = coberturaId
      ? await this.facturas.findCobertura(coberturaId)
      : await this.facturas.findCoberturaPorTipo('particular');
    if (!cobertura) throw new DomainException('No hay una cobertura configurada para el paciente');
    return cobertura;
  }

  /** Recalcula siempre desde las líneas guardadas: el total nunca se arrastra de un valor viejo. */
  private async recalcular(factura: Factura): Promise<void> {
    const lineas = await this.facturas.lineasDe(factura.id);
    const baseCentavos = lineas.reduce((acc, l) => acc + l.cantidad * aCentavos(l.precioUnitario), 0);

    const estrategia = this.estrategias.para(factura.cobertura.tipo);
    const calculo = estrategia.calcular(baseCentavos, Number(factura.cobertura.porcentajeCobertura));

    await this.facturas.actualizarTotales(
      factura.id,
      deCentavos(calculo.montoPacienteCentavos),
      deCentavos(calculo.montoCubiertoCentavos),
    );
```

```
  }
}
```

Ciclo de vida de la factura. Una tabla de transiciones alcanza (son cuatro estados y no hay comportamiento propio por estado):

`domain/factura-workflow.ts`:

```ts
import { TransicionInvalidaException } from '../../shared/exceptions/domain.exception';

export type AccionFactura = 'emitir' | 'pagar' | 'anular';

const TRANSICIONES: Record<AccionFactura, { desde: string[]; hacia: string }> = {
  emitir: { desde: ['pendiente'], hacia: 'emitida' },
  pagar: { desde: ['emitida'], hacia: 'pagada' },
  anular: { desde: ['pendiente', 'emitida'], hacia: 'anulada' },
};

export function siguienteEstadoFactura(actual: string, accion: AccionFactura): string {
  const regla = TRANSICIONES[accion];
  if (!regla.desde.includes(actual)) {
    throw new TransicionInvalidaException(`No se puede ${accion} una factura en estado "${actual}"`);
  }
  return regla.hacia;
}
```

`application/cambiar-estado-factura.base.ts`, con los tres casos de uso derivados (`EmitirFacturaUseCase`, `PagarFacturaUseCase`, `AnularFacturaUseCase`) igual que en turnos y camas:

```ts
import {
  ConflictoDeConcurrenciaException,
  DomainException,
  RecursoNoEncontradoException,
} from '../../shared/exceptions/domain.exception';
import { AccionFactura, siguienteEstadoFactura } from '../domain/factura-workflow';
import { Factura } from '../domain/entities/factura.entity';
import { IFacturaRepository } from '../domain/repositories/factura-repository.interface';

export abstract class CambiarEstadoFacturaBase {
  protected abstract readonly accion: AccionFactura;

  constructor(protected readonly facturas: IFacturaRepository) {}

  async ejecutar(facturaId: string): Promise<Factura> {
    const factura = await this.facturas.findById(facturaId);
    if (!factura) throw new RecursoNoEncontradoException('Factura');

    if (this.accion === 'emitir' && (await this.facturas.lineasDe(factura.id)).length === 0) {
      throw new DomainException('No se puede emitir una factura sin líneas');
    }

    const nuevo = siguienteEstadoFactura(factura.estado, this.accion);
    if (!(await this.facturas.cambiarEstado(factura.id, factura.estado, nuevo))) {
      throw new ConflictoDeConcurrenciaException('La factura');
    }
    return (await this.facturas.findById(factura.id)) ?? factura;
  }
}
```

Paso 5. Listener del evento `MedicacionAdministrada`

Este es el Observer. Facturación **se entera** de que se administró una medicación sin que Atención y Enfermería sepa que existe Facturación. El orden dentro del listener es la clave de la trazabilidad (outbox):

1. Si el evento ya se procesó, salir (idempotencia: si el evento se emite dos veces, no se cobra dos veces).
2. Guardar el evento en `eventos_dominio` como **no procesado**.
3. Registrar el cargo.
4. Marcar el evento como **procesado**.

Si el paso 3 falla, queda una fila `procesado = false`: se sabe exactamente qué evento no llegó a facturarse y se puede reintentar.

`infrastructure/listeners/medicacion-administrada.listener.ts`:

```ts
import { Inject, Injectable, Logger } from '@nestjs/common';
import { OnEvent } from '@nestjs/event-emitter';
import { EVENTOS, MedicacionAdministradaPayload } from '../../../shared/events/eventos';
import { RegistrarCargoUseCase } from '../../application/registrar-cargo.use-case';
import {
  EVENTO_DOMINIO_REPOSITORY,
  IEventoDominioRepository,
} from '../../domain/repositories/evento-dominio-repository.interface';

@Injectable()
export class MedicacionAdministradaListener {
  private readonly logger = new Logger(MedicacionAdministradaListener.name);

  constructor(
    private readonly registrarCargo: RegistrarCargoUseCase,
    @Inject(EVENTO_DOMINIO_REPOSITORY) private readonly eventos: IEventoDominioRepository,
  ) {}

  @OnEvent(EVENTOS.MEDICACION_ADMINISTRADA)
  async manejar(payload: MedicacionAdministradaPayload): Promise<void> {
    if (await this.eventos.existeProcesado(EVENTOS.MEDICACION_ADMINISTRADA, payload.indicacionId)) {
      return;
    }

    const evento = await this.eventos.registrar(EVENTOS.MEDICACION_ADMINISTRADA, { ...payload });

    try {
      await this.registrarCargo.ejecutar({
        pacienteId: payload.pacienteId,
        concepto: payload.concepto,
        cantidad: payload.cantidad,
        precioUnitario: payload.precioUnitario,
      });
      await this.eventos.marcarProcesado(evento.id);
    } catch (error) {
      // El evento queda sin procesar para poder reintentarlo; no se propaga al módulo emisor.
      this.logger.error(`No se pudo facturar el evento ${evento.id}`, error as Error);
    }
  }
}
```

Paso 6. Datos iniciales de coberturas

Sin coberturas cargadas no se puede facturar. Se crean al arrancar, igual que el administrador de IAM:

`infrastructure/sembrar-coberturas.service.ts`:

```ts
import { Inject, Injectable, OnApplicationBootstrap } from '@nestjs/common';
import { FACTURA_REPOSITORY, IFacturaRepository } from '../domain/repositories/factura-repository.interface';

@Injectable()
export class SembrarCoberturasService implements OnApplicationBootstrap {
  constructor(@Inject(FACTURA_REPOSITORY) private readonly facturas: IFacturaRepository) {}

  async onApplicationBootstrap(): Promise<void> {
    if ((await this.facturas.listarCoberturas()).length > 0) return;

    await this.facturas.crearCobertura('Particular', 'particular', '0');
    await this.facturas.crearCobertura('Obra social pública', 'publica', '100');
    await this.facturas.crearCobertura('Prepaga (ejemplo 70%)', 'prepaga', '70');
  }
}
```

Paso 7. Controller y módulo

`presentation/facturas.controller.ts`:

```ts
import { Body, Controller, Get, Param, ParseUUIDPipe, Patch, Post, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { IsInt, IsNotEmpty, IsNumberString, IsString, Min } from 'class-validator';
import { RolUsuario } from '../../iam/domain/enums/rol.enum';
import { Roles } from '../../shared/decorators/roles.decorator';
import { JwtAuthGuard } from '../../shared/guards/jwt-auth.guard';
import { RolesGuard } from '../../shared/guards/roles.guard';
import { AnularFacturaUseCase } from '../application/anular-factura.use-case';
import { EmitirFacturaUseCase } from '../application/emitir-factura.use-case';
import { ListarFacturasPacienteUseCase } from '../application/listar-facturas-paciente.use-case';
import { ObtenerFacturaUseCase } from '../application/obtener-factura.use-case';
import { PagarFacturaUseCase } from '../application/pagar-factura.use-case';
import { RegistrarCargoUseCase } from '../application/registrar-cargo.use-case';

class AgregarLineaDto {
```

```
  @IsString() @IsNotEmpty() concepto: string;
  @IsInt() @Min(1) cantidad: number;
  @IsNumberString() precioUnitario: string;
}

@ApiTags('facturacion')
@ApiBearerAuth()
@UseGuards(JwtAuthGuard, RolesGuard)
@Controller('facturacion')
export class FacturasController {
  constructor(
    private readonly obtener: ObtenerFacturaUseCase,
    private readonly delPaciente: ListarFacturasPacienteUseCase,
    private readonly cargo: RegistrarCargoUseCase,
    private readonly emitir: EmitirFacturaUseCase,
    private readonly pagar: PagarFacturaUseCase,
    private readonly anular: AnularFacturaUseCase,
  ) {}

  @Get('facturas/:id')
  @Roles(RolUsuario.ADMIN, RolUsuario.RECEPCIONISTA)
  ver(@Param('id', ParseUUIDPipe) id: string) {
    return this.obtener.ejecutar(id);
  }

  @Get('pacientes/:pacienteId/facturas')
  @Roles(RolUsuario.ADMIN, RolUsuario.RECEPCIONISTA)
  listar(@Param('pacienteId', ParseUUIDPipe) pacienteId: string) {
    return this.delPaciente.ejecutar(pacienteId);
  }

  @Post('pacientes/:pacienteId/cargos')
  @Roles(RolUsuario.ADMIN)
  cargoManual(@Param('pacienteId', ParseUUIDPipe) pacienteId: string, @Body() dto: AgregarLineaDto) {
    return this.cargo.ejecutar({ pacienteId, ...dto });
  }

  @Patch('facturas/:id/emitir')
  @Roles(RolUsuario.ADMIN, RolUsuario.RECEPCIONISTA)
  emitirFactura(@Param('id', ParseUUIDPipe) id: string) {
    return this.emitir.ejecutar(id);
  }

  @Patch('facturas/:id/pagar')
  @Roles(RolUsuario.ADMIN, RolUsuario.RECEPCIONISTA)
  pagarFactura(@Param('id', ParseUUIDPipe) id: string) {
    return this.pagar.ejecutar(id);
  }

  @Patch('facturas/:id/anular')
  @Roles(RolUsuario.ADMIN)
  anularFactura(@Param('id', ParseUUIDPipe) id: string) {
    return this.anular.ejecutar(id);
  }
}
```

`CoberturasController` (`/facturacion/coberturas`): `GET` lista (Admin, Recepcionista) y `POST` crea una nueva (Admin).

`facturacion/facturacion.module.ts`:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AdmisionesModule } from '../admisiones/admisiones.module';
import { AnularFacturaUseCase } from './application/anular-factura.use-case';
import { EmitirFacturaUseCase } from './application/emitir-factura.use-case';
import { ListarFacturasPacienteUseCase } from './application/listar-facturas-paciente.use-case';
import { ObtenerFacturaUseCase } from './application/obtener-factura.use-case';
import { PagarFacturaUseCase } from './application/pagar-factura.use-case';
import { RegistrarCargoUseCase } from './application/registrar-cargo.use-case';
import { CoberturaParticular } from './domain/estrategias/cobertura-particular';
import { CoberturaPrepaga } from './domain/estrategias/cobertura-prepaga';
import { CoberturaPublica } from './domain/estrategias/cobertura-publica';
import { ESTRATEGIAS_COBERTURA, IEstrategiaCobertura } from './domain/estrategias/estrategia-cobertura.interface';
import { ResolverEstrategiaCobertura } from './domain/estrategias/resolver-estrategia';
import { entidadesFacturacion } from './domain/entities';
import { EVENTO_DOMINIO_REPOSITORY } from './domain/repositories/evento-dominio-repository.interface';
import { FACTURA_REPOSITORY } from './domain/repositories/factura-repository.interface';
import { MedicacionAdministradaListener } from './infrastructure/listeners/medicacion-administrada.listener';
import { TypeOrmEventoDominioRepository } from './infrastructure/repositories/typeorm-evento-dominio.repository';
import { TypeOrmFacturaRepository } from './infrastructure/repositories/typeorm-factura.repository';
import { SembrarCoberturasService } from './infrastructure/sembrar-coberturas.service';
import { CoberturasController } from './presentation/coberturas.controller';
import { FacturacionController } from './presentation/facturacion.controller';
import { FacturasController } from './presentation/facturas.controller';

@Module({
  imports: [TypeOrmModule.forFeature(entidadesFacturacion), AdmisionesModule],
  controllers: [FacturacionController, FacturasController, CoberturasController],
  providers: [
    { provide: FACTURA_REPOSITORY, useClass: TypeOrmFacturaRepository },
```

```
    { provide: EVENTO_DOMINIO_REPOSITORY, useClass: TypeOrmEventoDominioRepository },

    // Strategy: para sumar una cobertura nueva, agregar su clase acá y en `inject`.
    CoberturaPublica,
    CoberturaPrepaga,
    CoberturaParticular,
    {
      provide: ESTRATEGIAS_COBERTURA,
      useFactory: (...estrategias: IEstrategiaCobertura[]) => estrategias,
      inject: [CoberturaPublica, CoberturaPrepaga, CoberturaParticular],
    },
    ResolverEstrategiaCobertura,

    RegistrarCargoUseCase,
    ObtenerFacturaUseCase,
    ListarFacturasPacienteUseCase,
    EmitirFacturaUseCase,
    PagarFacturaUseCase,
    AnularFacturaUseCase,

    MedicacionAdministradaListener,
    SembrarCoberturasService,
  ],
})
export class FacturacionModule {}
```

Endpoints

MÉTODO	RUTA	ROLES	RESPUESTAS
GET	`/facturacion/coberturas`	Admin, Recepcionista	200
POST	`/facturacion/coberturas`	Admin	201
GET	`/facturacion/facturas/:id`	Admin, Recepcionista	200, 404
GET	`/facturacion/pacientes/:id/facturas`	Admin, Recepcionista	200
POST	`/facturacion/pacientes/:id/cargos`	Admin	201, carga manual de una línea
PATCH	`/facturacion/facturas/:id/emitir`	Admin, Recepcionista	200, 422 sin líneas o estado inválido
PATCH	`/facturacion/facturas/:id/pagar`	Admin, Recepcionista	200, 422
PATCH	`/facturacion/facturas/:id/anular`	Admin	200, 422

La factura de medicaciones **no tiene endpoint de alta**: aparece sola cuando enfermería cumple una indicación.

Tests

`domain/estrategias/estrategias-cobertura.spec.ts` :

```
import { DomainException } from '../../../shared/exceptions/domain.exception';
import { CoberturaParticular } from './cobertura-particular';
import { CoberturaPrepaga } from './cobertura-prepaga';
import { CoberturaPublica } from './cobertura-publica';
import { ResolverEstrategiaCobertura } from './resolver-estrategia';

describe('Estrategias de cobertura', () => {
  const base = 1_000_000; // $10.000,00 en centavos

  it('pública: lo cubre todo', () => {
    expect(new CoberturaPublica().calcular(base)).toEqual({
      montoCubiertoCentavos: 1_000_000,
      montoPacienteCentavos: 0,
    });
```

```
  });

  it('particular: el paciente paga todo', () => {
    expect(new CoberturaParticular().calcular(base)).toEqual({
      montoCubiertoCentavos: 0,
      montoPacienteCentavos: 1_000_000,
    });
  });

  it('prepaga al 70%: $7.000 cubiertos y $3.000 a cargo del paciente', () => {
    expect(new CoberturaPrepaga().calcular(base, 70)).toEqual({
      montoCubiertoCentavos: 700_000,
      montoPacienteCentavos: 300_000,
    });
  });

  it('prepaga: cubierto + paciente siempre suman la base, incluso con redondeo', () => {
    const r = new CoberturaPrepaga().calcular(333, 70);   // 233,1 se redondea a 233
    expect(r.montoCubiertoCentavos + r.montoPacienteCentavos).toBe(333);
  });
});

describe('ResolverEstrategiaCobertura', () => {
  const resolver = new ResolverEstrategiaCobertura([new CoberturaPublica(), new CoberturaParticular()]);

  it('devuelve la estrategia que coincide con el tipo', () => {
    expect(resolver.para('publica')).toBeInstanceOf(CoberturaPublica);
  });

  it('falla si el tipo no tiene estrategia (ej. una cobertura nueva sin implementar)', () => {
    expect(() => resolver.para('art')).toThrow(DomainException);
  });
});
```

`application/registrar-cargo.use-case.spec.ts`: con repositorios en memoria, verifica que el total se recalcula y se acumula:

```
import { Cobertura } from '../domain/entities/cobertura.entity';
import { Factura } from '../domain/entities/factura.entity';
import { LineaFactura } from '../domain/entities/linea-factura.entity';
import { CoberturaParticular } from '../domain/estrategias/cobertura-particular';
import { CoberturaPrepaga } from '../domain/estrategias/cobertura-prepaga';
import { ResolverEstrategiaCobertura } from '../domain/estrategias/resolver-estrategia';
import { RegistrarCargoUseCase } from './registrar-cargo.use-case';

function armar(cobertura: Cobertura) {
  const factura = Object.assign(new Factura(), { id: 'f1', pacienteId: 'p1', estado: 'pendiente', cobertura });
  const lineas: LineaFactura[] = [];
  let existe = false;

  const facturas = {
    findAbiertaDePaciente: async () => (existe ? factura : null),
    crear: async () => { existe = true; return factura; },
    findCobertura: async () => cobertura,
    findCoberturaPorTipo: async () => cobertura,
    agregarLinea: async (_f: string, concepto: string, cantidad: number, precioUnitario: string) => {
      const l = Object.assign(new LineaFactura(), { concepto, cantidad, precioUnitario });
      lineas.push(l);
      return l;
    },
    lineasDe: async () => lineas,
    actualizarTotales: async (_id: string, total: string, cubierto: string) => {
      factura.total = total;
      factura.montoCubierto = cubierto;
    },
    findById: async () => factura,
  };
  const pacientes = { findById: async () => ({ id: 'p1', coberturaId: cobertura.id }) };
  const resolver = new ResolverEstrategiaCobertura([new CoberturaPrepaga(), new CoberturaParticular()]);

  return new RegistrarCargoUseCase(facturas as never, pacientes as never, resolver);
}

describe('RegistrarCargoUseCase', () => {
  it('acumula líneas y recalcula con la cobertura del paciente (prepaga 70%)', async () => {
    const prepaga = Object.assign(new Cobertura(), { id: 'c1', tipo: 'prepaga', porcentajeCobertura: '70.00' });
    const useCase = armar(prepaga);

    await useCase.ejecutar({ pacienteId: 'p1', concepto: 'Medicación: Ampolla', cantidad: 2, precioUnitario: '1500.00' });
    const factura = await useCase.ejecutar({ pacienteId: 'p1', concepto: 'Medicación: Suero', cantidad: 1, precioUnitario: '1000.00'
});

    // base = 2 x 1500 + 1000 = 4000 -> cubierto 2800, paciente 1200
    expect(factura.montoCubierto).toBe('2800.00');
    expect(factura.total).toBe('1200.00');
  });

  it('rechaza cantidades inválidas', async () => {
    const particular = Object.assign(new Cobertura(), { id: 'c2', tipo: 'particular', porcentajeCobertura: '0.00' });
    await expect(
      armar(particular).ejecutar({ pacienteId: 'p1', concepto: 'x', cantidad: 0, precioUnitario: '10.00' }),
```

```
    ).rejects.toThrow('entero positivo');
  });
});
```

Terminado cuando

- [ ] Existen las tres coberturas iniciales (`GET /facturacion/coberturas`).
- [ ] Un paciente con cobertura prepaga al 70%: cumplir una indicación de medicación en Atención genera sola una factura `pendiente` con una línea, con el total y el monto cubierto correctos.
- [ ] Cumplir una segunda medicación agrega una segunda línea a la **misma** factura y recalcula los montos.
- [ ] La tabla `eventos_dominio` tiene una fila por cada medicación, con `procesado = true`.
- [ ] La factura pasa de `pendiente` a `emitida` y a `pagada`. Intentar pagar una factura `pendiente` devuelve 422.
- [ ] Agregar una cobertura nueva de prueba (por ejemplo ART) no requiere modificar ninguna estrategia ni caso de uso existente.
- [ ] Los tests nuevos pasan y los anteriores siguen en verde.

Frontend: sesión, rutas por rol y vistas

Objetivo. Que la aplicación Vue hable con el backend real: iniciar sesión, guardar el token, proteger rutas según el rol y tener una vista por módulo que consuma los endpoints de las secciones anteriores. Se hace al final, cuando el backend ya responde.

Qué hay hoy

YA EXISTE	FALTA
`shared/http/http-client.ts` : instancia de axios con `baseURL` y un interceptor que agrega el `Bearer` desde el store	Manejo del 401 (cerrar sesión y volver al login)
`modules/auth/stores/auth.store.ts` con `token` , `usuario` , `estaAutenticado` , `setSesion` , `cerrarSesion`	Llamar a `POST /iam/login` y persistir el token
`router/index.ts` con las rutas `/login` , `/admisiones` , `/historia-clinica` , `/atencion` , `/camas` , `/facturacion`	Guard `beforeEach` y `meta.roles`
Vistas de cada módulo (vacías)	Contenido real de cada vista

Estructura final

```
src/
├── shared/
│   ├── http/http-client.ts                (se modifica)
│   └── types/roles.ts                     NUEVO
├── router/index.ts                        (se modifica)
└── modules/
    ├── auth/
    │   ├── api/auth.api.ts                NUEVO
    │   ├── stores/auth.store.ts           (se modifica)
    │   └── views/LoginView.vue            (se completa)
    └── admisiones/
        ├── api/turnos.api.ts              NUEVO
        ├── stores/turnos.store.ts         NUEVO
        └── views/AgendaView.vue           (se completa)
```

Los demás módulos repiten exactamente el mismo molde: `api/` (llamadas HTTP, sin estado), `stores/` (estado y acciones) y `views/` (pantallas).

Paso 1. Tipos de roles

Archivo: `src/shared/types/roles.ts` . Tiene que coincidir con el enum del backend.

```ts
export type Rol = 'admin' | 'medico' | 'enfermero' | 'recepcionista' | 'paciente';

export interface UsuarioSesion {
  id: string;
  nombre: string;
  rol: Rol;
}
```

Paso 2. API de autenticación

Archivo: `src/modules/auth/api/auth.api.ts` :

```ts
import { httpClient } from '@/shared/http/http-client';
import type { UsuarioSesion } from '@/shared/types/roles';

export interface LoginRespuesta {
  accessToken: string;
  usuario: UsuarioSesion;
}
```

```ts
export const authApi = {
  async login(email: string, password: string): Promise<LoginRespuesta> {
    const { data } = await httpClient.post<LoginRespuesta>('/iam/login', { email, password });
    return data;
  },
};
```

Paso 3. Store de sesión

Archivo: `src/modules/auth/stores/auth.store.ts`. Reemplaza el stub. El token se guarda en `localStorage` para que la sesión sobreviva a recargar la página.

```ts
import { defineStore } from 'pinia';
import { authApi } from '../api/auth.api';
import type { Rol, UsuarioSesion } from '@/shared/types/roles';

const CLAVE = 'his.sesion';

interface Estado {
  token: string | null;
  usuario: UsuarioSesion | null;
}

function leerSesion(): Estado {
  try {
    const crudo = localStorage.getItem(CLAVE);
    if (crudo) return JSON.parse(crudo) as Estado;
  } catch {
    /* localStorage no disponible o JSON roto: se arranca sin sesión */
  }
  return { token: null, usuario: null };
}

export const useAuthStore = defineStore('auth', {
  state: (): Estado => leerSesion(),

  getters: {
    estaAutenticado: (s) => !!s.token,
    rol: (s): Rol | null => s.usuario?.rol ?? null,
  },

  actions: {
    async iniciarSesion(email: string, password: string) {
      const { accessToken, usuario } = await authApi.login(email, password);
      this.setSesion(accessToken, usuario);
    },

    setSesion(token: string, usuario: UsuarioSesion) {
      this.token = token;
      this.usuario = usuario;
      try {
        localStorage.setItem(CLAVE, JSON.stringify({ token, usuario }));
      } catch {
        /* sin persistencia: la sesión dura hasta recargar */
      }
    },

    cerrarSesion() {
      this.token = null;
      this.usuario = null;
      try {
        localStorage.removeItem(CLAVE);
      } catch {
        /* nada que limpiar */
      }
    },

    tieneRol(...roles: Rol[]) {
      return !!this.usuario && roles.includes(this.usuario.rol);
    },
  },
});
```

Paso 4. Cliente HTTP: manejar el 401

Archivo: `src/shared/http/http-client.ts`. Se conserva el interceptor de request que ya existe y se agrega uno de respuesta. Si el token venció, se limpia la sesión y se vuelve al login.

```ts
import axios from 'axios';
import { useAuthStore } from '@/modules/auth/stores/auth.store';
import { router } from '@/router';

export const httpClient = axios.create({
```

```
  baseURL: import.meta.env.VITE_API_URL ?? 'http://localhost:3000',
});

httpClient.interceptors.request.use((config) => {
  const auth = useAuthStore();
  if (auth.token) {
    config.headers.Authorization = `Bearer ${auth.token}`;
  }
  return config;
});

httpClient.interceptors.response.use(
  (respuesta) => respuesta,
  (error) => {
    const url: string = error.config?.url ?? '';
    // El 401 del propio login es "credenciales incorrectas", no sesión vencida.
    if (error.response?.status === 401 && !url.includes('/iam/login')) {
      useAuthStore().cerrarSesion();
      router.push({ name: 'login' });
    }
    return Promise.reject(error);
  },
);

/** Saca el mensaje legible de un error de la API para mostrarlo en pantalla. */
export function mensajeDeError(error: unknown): string {
  const e = error as { response?: { data?: { message?: string | string[] } } };
  const msg = e.response?.data?.message;
  if (Array.isArray(msg)) return msg.join('. ');
  return msg ?? 'No se pudo completar la operación';
}
```

Si `router` importa el store y el store importa `http-client`, puede aparecer una importación circular. Si pasa, en el interceptor se usa `const { router } = await import('@/router')` (importación dinámica) en lugar del `import` de arriba.

Paso 5. Router con control de acceso por rol

Archivo: `src/router/index.ts`. Cada ruta declara en `meta.roles` quién puede entrar; si no declara nada, alcanza con estar logueado.

```
import { createRouter, createWebHistory } from 'vue-router';
import { useAuthStore } from '@/modules/auth/stores/auth.store';
import type { Rol } from '@/shared/types/roles';

declare module 'vue-router' {
  interface RouteMeta {
    publica?: boolean;
    roles?: Rol[];
  }
}

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/login',
      name: 'login',
      component: () => import('@/modules/auth/views/LoginView.vue'),
      meta: { publica: true },
    },
    {
      path: '/admisiones',
      name: 'admisiones',
      component: () => import('@/modules/admisiones/views/AgendaView.vue'),
      meta: { roles: ['recepcionista', 'medico', 'admin'] },
    },
    {
      path: '/historia-clinica',
      name: 'historia-clinica',
      component: () => import('@/modules/historia-clinica/views/HistoriaClinicaView.vue'),
      meta: { roles: ['medico', 'enfermero', 'admin'] },
    },
    {
      path: '/atencion',
      name: 'atencion',
      component: () => import('@/modules/atencion/views/AtencionView.vue'),
      meta: { roles: ['enfermero', 'medico'] },
    },
    {
      path: '/camas',
      name: 'camas',
      component: () => import('@/modules/camas/views/CamasView.vue'),
      meta: { roles: ['admin', 'recepcionista', 'enfermero', 'medico'] },
    },
    {
      path: '/facturacion',
```

```
      name: 'facturacion',
      component: () => import('@/modules/facturacion/views/FacturacionView.vue'),
      meta: { roles: ['admin', 'recepcionista'] },
    },
    { path: '/:pathMatch(.*)*', redirect: '/admisiones' },
  ],
});

router.beforeEach((to) => {
  const auth = useAuthStore();

  if (to.meta.publica) {
    // Si ya hay sesión, no tiene sentido volver al login.
    return auth.estaAutenticado ? { path: '/admisiones' } : true;
  }

  if (!auth.estaAutenticado) {
    return { name: 'login', query: { redirect: to.fullPath } };
  }

  const permitidos = to.meta.roles;
  if (permitidos && !auth.tieneRol(...permitidos)) {
    return { path: '/admisiones' }; // o una vista "sin permiso"
  }
  return true;
});
```

Ajustar los nombres de los archivos de vista a los que ya existen en el repo (el stub de cada módulo ya tiene su `View.vue`). El guard solo mejora la experiencia: la seguridad real está en el backend, que devuelve 403 si el rol no corresponde.

Paso 6. Vista de login

Archivo: `src/modules/auth/views/LoginView.vue`:

```
<script setup lang="ts">
import { ref } from 'vue';
import { useRoute, useRouter } from 'vue-router';
import { useAuthStore } from '../stores/auth.store';
import { mensajeDeError } from '@/shared/http/http-client';

const auth = useAuthStore();
const router = useRouter();
const route = useRoute();

const email = ref('');
const password = ref('');
const error = ref('');
const cargando = ref(false);

async function enviar() {
  error.value = '';
  cargando.value = true;
  try {
    await auth.iniciarSesion(email.value, password.value);
    const destino = (route.query.redirect as string) || '/admisiones';
    await router.push(destino);
  } catch (e) {
    error.value = mensajeDeError(e);
  } finally {
    cargando.value = false;
  }
}
</script>

<template>
  <main class="login">
    <form @submit.prevent="enviar">
      <h1>Ingresar al HIS</h1>

      <label>
        Email
        <input v-model="email" type="email" autocomplete="username" required />
      </label>

      <label>
        Contraseña
        <input v-model="password" type="password" autocomplete="current-password" required />
      </label>

      <p v-if="error" role="alert" class="error">{{ error }}</p>

      <button type="submit" :disabled="cargando">
        {{ cargando ? 'Ingresando…' : 'Ingresar' }}
      </button>
    </form>
  </main>
```

```
</template>

<style scoped>
.login { display: grid; place-items: center; min-height: 100vh; padding: 16px; }
form { display: grid; gap: 12px; width: min(360px, 100%); }
label { display: grid; gap: 4px; font-size: 14px; }
input { padding: 8px; font-size: 16px; }
button { padding: 10px; font-size: 16px; cursor: pointer; }
.error { color: #b3261e; margin: 0; }
</style>
```

Paso 7. Un módulo completo como molde: Admisiones

Los otros módulos se copian de este. Primero la capa `api`, que solo sabe de HTTP:

Archivo: `src/modules/admisiones/api/turnos.api.ts`:

```
import { httpClient } from '@/shared/http/http-client';

export type EstadoTurno = 'programado' | 'confirmado' | 'en_sala_espera' | 'atendido' | 'cancelado';

export interface Turno {
  id: string;
  pacienteId: string;
  medicoId: string;
  fechaHora: string;
  estado: EstadoTurno;
  motivo?: string;
}

export interface NuevoTurno {
  pacienteId: string;
  medicoId: string;
  fechaHora: string; // ISO 8601
  motivo?: string;
}

export const turnosApi = {
  async listar(medicoId: string, dia: string): Promise<Turno[]> {
    const { data } = await httpClient.get<Turno[]>('/admisiones/turnos', {
      params: { medicoId, dia },
    });
    return data;
  },

  async reservar(turno: NuevoTurno): Promise<Turno> {
    const { data } = await httpClient.post<Turno>('/admisiones/turnos', turno);
    return data;
  },

  async cambiarEstado(
    id: string,
    accion: 'confirmar' | 'llegada' | 'atender' | 'cancelar',
  ): Promise<Turno> {
    const { data } = await httpClient.patch<Turno>(`/admisiones/turnos/${id}/${accion}`);
    return data;
  },
};
```

Después el store, que guarda el estado y traduce los errores del backend a mensajes:

Archivo: `src/modules/admisiones/stores/turnos.store.ts`:

```
import { defineStore } from 'pinia';
import { turnosApi, type NuevoTurno, type Turno } from '../api/turnos.api';
import { mensajeDeError } from '@/shared/http/http-client';

export const useTurnosStore = defineStore('turnos', {
  state: () => ({
    turnos: [] as Turno[],
    cargando: false,
    error: '' as string,
  }),

  actions: {
    async cargar(medicoId: string, dia: string) {
      this.cargando = true;
      this.error = '';
      try {
        this.turnos = await turnosApi.listar(medicoId, dia);
      } catch (e) {
        this.error = mensajeDeError(e);
      } finally {
        this.cargando = false;
      }
    },
```

```
    async reservar(nuevo: NuevoTurno) {
      this.error = '';
      try {
        const creado = await turnosApi.reservar(nuevo);
        this.turnos.push(creado);
        return true;
      } catch (e) {
        // 409 = otra recepcionista tomó el horario un instante antes.
        this.error = mensajeDeError(e);
        return false;
      }
    },

    async cambiarEstado(id: string, accion: 'confirmar' | 'llegada' | 'atender' | 'cancelar') {
      this.error = '';
      try {
        const actualizado = await turnosApi.cambiarEstado(id, accion);
        const i = this.turnos.findIndex((t) => t.id === id);
        if (i >= 0) this.turnos[i] = actualizado;
      } catch (e) {
        this.error = mensajeDeError(e);
      }
    },
  },
});
```

Y la vista, que muestra la agenda y solo ofrece los botones que corresponden al estado y al rol:

Archivo: `src/modules/admisiones/views/AgendaView.vue`:

```
<script setup lang="ts">
import { ref, watch } from 'vue';
import { useTurnosStore } from '../stores/turnos.store';
import { useAuthStore } from '@/modules/auth/stores/auth.store';
import type { EstadoTurno } from '../api/turnos.api';

const turnos = useTurnosStore();
const auth = useAuthStore();

const medicoId = ref('');
const dia = ref(new Date().toISOString().slice(0, 10));

watch([medicoId, dia], ([m, d]) => {
  if (m) turnos.cargar(m, d);
}, { immediate: true });

/** Qué acciones se ofrecen según el estado actual del turno. */
const ACCIONES: Record<EstadoTurno, Array<'confirmar' | 'llegada' | 'atender' | 'cancelar'>> = {
  programado: ['confirmar', 'cancelar'],
  confirmado: ['llegada', 'cancelar'],
  en_sala_espera: ['atender', 'cancelar'],
  atendido: [],
  cancelado: [],
};

/** El rol decide qué botón se muestra (el backend lo vuelve a validar). */
function puede(accion: string) {
  if (accion === 'atender') return auth.tieneRol('medico');
  return auth.tieneRol('recepcionista', 'admin');
}
</script>

<template>
  <section>
    <h1>Agenda</h1>

    <label>Médico (id) <input v-model="medicoId" /></label>
    <label>Día <input v-model="dia" type="date" /></label>

    <p v-if="turnos.error" role="alert">{{ turnos.error }}</p>
    <p v-if="turnos.cargando">Cargando…</p>

    <table v-else>
      <thead>
        <tr><th>Hora</th><th>Paciente</th><th>Estado</th><th></th></tr>
      </thead>
      <tbody>
        <tr v-for="t in turnos.turnos" :key="t.id">
          <td>{{ new Date(t.fechaHora).toLocaleTimeString('es-AR', { hour: '2-digit', minute: '2-digit' }) }}</td>
          <td>{{ t.pacienteId }}</td>
          <td>{{ t.estado }}</td>
          <td>
            <button
              v-for="a in ACCIONES[t.estado].filter(puede)"
              :key="a"
              @click="turnos.cambiarEstado(t.id, a)"
            >
              {{ a }}
            </button>
          </td>
```

```
        </tr>
      </tbody>
    </table>
  </section>
</template>
```

Este es el mínimo funcional. Después se mejora: un selector de médicos usando `GET /admisiones/medicos`, un buscador de pacientes con `GET /admisiones/pacientes?q=`, y un formulario para reservar que llame a `turnos.reservar`. Cuando el backend responde 409 el mensaje aparece solo gracias a `mensajeDeError`.

Paso 8. Vistas del resto de los módulos

Cada una sigue el molde del paso 7 (`api` → `store` → `view`). Esta tabla dice qué pantalla armar y qué endpoints usa.

MÓDULO	VISTA	QUÉ MUESTRA	ENDPOINTS QUE CONSUME
Admisiones	Pacientes	Alta y búsqueda por DNI o nombre	`POST` y `GET /admisiones/pacientes`
Admisiones	Agenda	Turnos del día por médico, con botones de estado	`GET/POST /admisiones/turnos`, `PATCH .../confirmar`, `llegada`, `atender`, `cancelar`
Admisiones	Sala de espera	Cola ya ordenada por prioridad y botón "Llamar siguiente" (médico)	`GET/POST /admisiones/cola`, `POST /admisiones/cola/siguiente`
Camas	Mapa de camas	Camas por sector, coloreadas por estado	`GET /camas/mapa`
Camas	Acciones sobre una cama	Asignar, liberar, confirmar limpieza, bloquear, habilitar	`POST /camas/:id/asignar`, `PATCH .../liberar`, `limpieza-confirmada`, `bloquear`, `habilitar`
Camas	Inventario	Insumos con stock total, alta de insumo y de stock	`GET/POST /camas/insumos`, `POST /camas/insumos/:id/stock`
Historia clínica	Historia del paciente	Alergias, antecedentes y evoluciones en orden cronológico	`GET /historia-clinica/paciente/:id`
Historia clínica	Formularios	Abrir historia, agregar alergia, antecedente y evolución (la evolución solo para médico)	`POST /historia-clinica/paciente/:id`, `.../alergias`, `.../antecedentes`, `.../evoluciones`
Atención	Signos vitales	Formulario y gráfico o tabla; marca en color si la severidad es alerta o crítica	`POST/GET /atencion-enfermeria/pacientes/:id/signos-vitales`
Atención	Indicaciones	Pendientes del paciente y botón "Cumplir" (enfermero)	`GET/POST .../indicaciones`, `PATCH /atencion-enfermeria/indicaciones/:id/cumplir`
Atención	Protocolos	Alta de protocolo con checklist de tareas	`POST .../protocolos`, `GET .../protocolos/:id/tareas`, `PATCH .../tareas/:id/completar`

MÓDULO	VISTA	QUÉ MUESTRA	ENDPOINTS QUE CONSUME
Facturación	Facturas del paciente	Listado con estado, líneas y totales	`GET /facturacion/pacientes/:id/facturas`, `GET /facturacion/facturas/:id`
Facturación	Acciones de factura	Emitir, pagar, anular	`PATCH /facturacion/facturas/:id/emitir`, `pagar`, `anular`
Facturación	Coberturas	Listado y alta (admin)	`GET/POST /facturacion/coberturas`

Detalles que conviene respetar

- **Dinero.** Los montos llegan en centavos (enteros). Se formatean en un solo lugar:

`ts export const pesos = (centavos: number) => new Intl.NumberFormat('es-AR', { style: 'currency', currency: 'ARS' }).format(centavos / 100);`

Ponerlo en `src/shared/format/dinero.ts` y usarlo en todas las vistas de facturación. - **Errores 409 y 422.** Son errores esperables del negocio (horario tomado, stock insuficiente, transición no permitida). Se muestran como mensaje al usuario, no como pantalla de error. - **Botones según el estado.** Mostrar solo las acciones válidas para el estado actual, como en `ACCIONES` del paso 7. El backend igual rechaza las inválidas con 422. - **Fechas.** Enviar siempre ISO 8601 con zona horaria (`new Date().toISOString()`); mostrar con `toLocaleString('es-AR')`. - **CORS.** Si el navegador bloquea las llamadas, revisar que `main.ts` del backend tenga `app.enableCors({ origin: 'http://localhost:5173' })`.

Tests

Un test del store de sesión alcanza para fijar el comportamiento que más importa. Archivo: `src/modules/auth/stores/auth.store.spec.ts` (con Vitest, si el proyecto ya lo tiene configurado):

```
import { beforeEach, describe, expect, it, vi } from 'vitest';
import { createPinia, setActivePinia } from 'pinia';
import { useAuthStore } from './auth.store';

vi.mock('../api/auth.api', () => ({
  authApi: {
    login: vi.fn().mockResolvedValue({
      accessToken: 'token-1',
      usuario: { id: 'u1', nombre: 'Dra. Pérez', rol: 'medico' },
    }),
  },
}));

describe('auth store', () => {
  beforeEach(() => {
    localStorage.clear();
    setActivePinia(createPinia());
  });

  it('inicia sesión y guarda el token', async () => {
    const auth = useAuthStore();
    await auth.iniciarSesion('dra@hospital.test', 'clave');

    expect(auth.estaAutenticado).toBe(true);
    expect(auth.rol).toBe('medico');
    expect(localStorage.getItem('his.sesion')).toContain('token-1');
  });

  it('cierra sesión y borra el token', async () => {
    const auth = useAuthStore();
    await auth.iniciarSesion('dra@hospital.test', 'clave');
    auth.cerrarSesion();

    expect(auth.estaAutenticado).toBe(false);
    expect(localStorage.getItem('his.sesion')).toBeNull();
  });

  it('tieneRol respeta el rol del usuario', async () => {
    const auth = useAuthStore();
```

```
    await auth.iniciarSesion('dra@hospital.test', 'clave');

    expect(auth.tieneRol('medico', 'admin')).toBe(true);
    expect(auth.tieneRol('recepcionista')).toBe(false);
  });
});
```

Si el frontend todavía no tiene Vitest: `npm i -D vitest jsdom @vue/test-utils` y en `vite.config.ts` agregar `test: { environment: 'jsdom' }`.

Terminado cuando

- Entrar con el admin sembrado (`ADMIN_EMAIL` / `ADMIN_PASSWORD`) lleva a `/admisiones` y recargar la página mantiene la sesión.
- Entrar a `/facturacion` con un usuario `enfermero` redirige y no muestra la pantalla.
- Con el token borrado o vencido, cualquier llamada que dé 401 lleva al login.
- La agenda lista turnos reales y los botones cambian el estado; reservar dos veces el mismo horario muestra el mensaje del 409.
- Cada módulo tiene su vista conectada a los endpoints de la tabla del paso 8.